npu-compute性能优化

1. 功能概述

1.1. 背景描述

- 面向PTO编程的命令行工具，支持性能优化
- 面向Agent的提供友好性显示输出
- 面向下一代芯片验证工具

1.2. 功能范围

为了更好的满足PTO/ASC调优能力, 支持用户、Agent更友好的使用NPU调优能力，需要提供对应的PTO/ASC公共的调优能力。

工具分类	主要功能特性
精度验证	精度比对
	误差分析
异常分析	内存检测
	竞争检测
	未初始化检测
	同步检测
	溢出检测
	Aicpu内存检测
	芯片状态分析
benchmark	SOL等
性能分析	计算负载
	内存负载
	时序流水（仿真/上板）
	内存带宽
	热点指令（仿真/上板）
	Roofline
运行&复跑	单算子运行/多端复跑

交付时间	交付特性	
930	Metrics性能指标	
	Pipeline流水（biuperf）	
1230	SIMD 热点代码/SIMT热点代码	

1.3. 场景分析

1.3.1. aclnn算子采集

auto

Full Screen

```
1
```

1.3.2. <<<>>>算子采集

cpp

Full Screen

```cpp
#include "utils/debug/asc_printf.h"
#include "acl/acl.h"

__global__ __vector__ void hello_world() {
    printf("Hello World!!!\n");
}

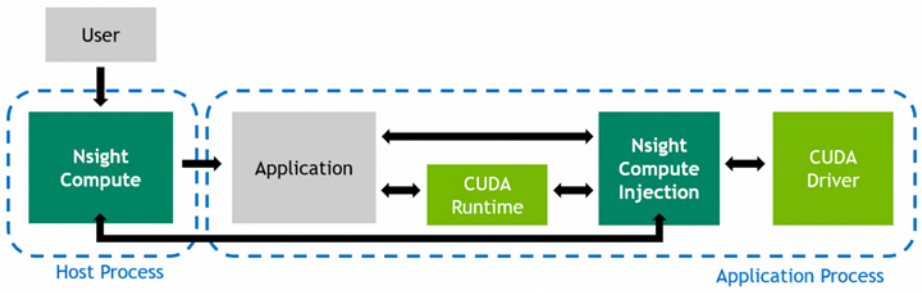
int main(int argc, char const* argv[])
{
    aclrtSetDevice(0); // 运行管理资源申请。
    aclrtStream stream = nullptr;
    aclrtCreateStream(&stream);
    hello_world<<<8, 0, stream>>>();
    aclrtSynchronizeStream(stream);
    aclrtDestroyStream(stream);
    aclrtResetDevice(0); // 销毁运行资源。
    return 0;
}
```

2. 竞品方案借鉴点（以竞品为参考）

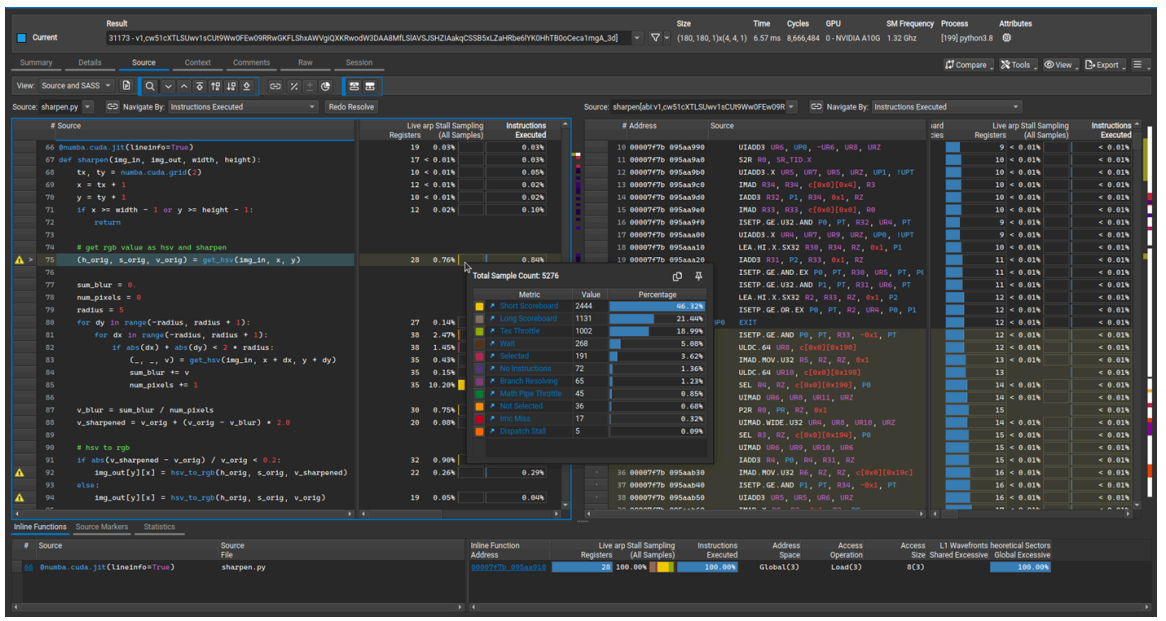
2.1. Nsight System

2.2. Nsight Compute

2.2.1. 采集方式



2.2.2. **warp sampling**



2.2.3. **命令行参数**

2.2.3.1. **--list-sets**

```
(base) root@huawei:/home/w00441123/nvcc/printf# ncu --list-sets
---------- ------------------------------------------------------------------------------ ------- -----------------
Identifier Sections                                                                       Enabled Estimated Metrics
---------- ------------------------------------------------------------------------------ ------- -----------------
basic      LaunchStats, Occupancy, SpeedOfLight, WorkloadDistribution                     yes     213
detailed   ComputeWorkloadAnalysis, LaunchStats, MemoryWorkloadAnalysis, MemoryWorkloa    no      1071
           dAnalysis_Chart, Occupancy, SourceCounters, SpeedOfLight, SpeedOfLight_Roof
           lineChart, Tile, WorkloadDistribution
full       ComputeWorkloadAnalysis, InstructionStats, LaunchStats, MemoryWorkloadAnaly    no      7381
           sis, MemoryWorkloadAnalysis_Chart, MemoryWorkloadAnalysis_Tables, NumaAffin
           ity, Nvlink_Tables, Nvlink_Topology, Occupancy, PmSampling, SchedulerStats,
            SourceCounters, SpeedOfLight, SpeedOfLight_HierarchicalDoubleRooflineChart
           , SpeedOfLight_HierarchicalHalfRooflineChart, SpeedOfLight_HierarchicalSing
           leRooflineChart, SpeedOfLight_HierarchicalTensorRooflineChart, SpeedOfLight
           _RooflineChart, Tile, WarpStateStats, WorkloadDistribution
nvlink     Nvlink, Nvlink_Tables, Nvlink_Topology                                         no      122
pmsampling PmSampling, PmSampling_WarpStates                                              no      554
roofline   SpeedOfLight, SpeedOfLight_HierarchicalDoubleRooflineChart, SpeedOfLight_Hi    no      5919
           erarchicalHalfRooflineChart, SpeedOfLight_HierarchicalSingleRooflineChart,
           SpeedOfLight_HierarchicalTensorRooflineChart, SpeedOfLight_RooflineChart, W
           orkloadDistribution
(base) root@huawei:/home/w00441123/nvcc/printf#
```

2.2.3.2. **--list-sections**

```
(base) root@huawei:/home/w00441123/nvcc/printf# ncu --list-sections
----------------------------------------- ------------------------------------------------------------------------ ------- ------------------------------------------------------
Identifier                                Display Name                                                             Enabled Filename
----------------------------------------- ------------------------------------------------------------------------ ------- ------------------------------------------------------
C2CLink                                   C2CLink                                                                  no      ...sight Compute/2026.2.0/Sections/C2CLink.section
ComputeWorkloadAnalysis                   Compute Workload Analysis                                                no      ...26.2.0/Sections/ComputeWorkloadAnalysis.section
InstructionStats                          Instruction Statistics                                                   no      ...2026.2.0/Sections/InstructionStatistics.section
LaunchStats                               Launch Statistics                                                        yes     ...pute/2026.2.0/Sections/LaunchStatistics.section
MemoryWorkloadAnalysis                    Memory Workload Analysis                                                 no      ...026.2.0/Sections/MemoryWorkloadAnalysis.section
MemoryWorkloadAnalysis_Chart              Memory Workload Analysis Chart                                           no      ...0/Sections/MemoryWorkloadAnalysis_Chart.section
MemoryWorkloadAnalysis_Tables             Memory Workload Analysis Tables                                          no      .../Sections/MemoryWorkloadAnalysis_Tables.section
NumaAffinity                              NUMA Affinity                                                            no      ... Compute/2026.2.0/Sections/NumaAffinity.section
Nvlink                                    NVLink                                                                   no      ...Nsight Compute/2026.2.0/Sections/Nvlink.section
Nvlink_Tables                             NVLink Tables                                                            no      ...Compute/2026.2.0/Sections/Nvlink_Tables.section
Nvlink_Topology                           NVLink Topology                                                          no      ...mpute/2026.2.0/Sections/Nvlink_Topology.section
Occupancy                                 Occupancy                                                                yes     ...ght Compute/2026.2.0/Sections/Occupancy.section
PmSampling                                PM Sampling                                                              no      ...ht Compute/2026.2.0/Sections/PmSampling.section
PmSampling_WarpStates                     PM Sampling: Warp States                                                 no      ...2026.2.0/Sections/PmSampling_WarpStates.section
SchedulerStats                            Scheduler Statistics                                                     no      ...e/2026.2.0/Sections/SchedulerStatistics.section
SourceCounters                            Source Counters                                                          no      ...ompute/2026.2.0/Sections/SourceCounters.section
SpeedOfLight                              GPU Speed Of Light Throughput                                            yes     ... Compute/2026.2.0/Sections/SpeedOfLight.section
SpeedOfLight_HierarchicalDoubleRooflineChart GPU Speed Of Light Hierarchical Roofline Chart (Double Precision)    no      ...OfLight_HierarchicalDoubleRooflineChart.section
SpeedOfLight_HierarchicalHalfRooflineChart   GPU Speed Of Light Hierarchical Roofline Chart (Half Precision)      no      ...edOfLight_HierarchicalHalfRooflineChart.section
SpeedOfLight_HierarchicalSingleRooflineChart GPU Speed Of Light Hierarchical Roofline Chart (Single Precision)    no      ...OfLight_HierarchicalSingleRooflineChart.section
SpeedOfLight_HierarchicalTensorRooflineChart GPU Speed Of Light Hierarchical Roofline Chart (Tensor Core)         no      ...OfLight_HierarchicalTensorRooflineChart.section
SpeedOfLight_RooflineChart                GPU Speed Of Light Roofline Chart (Overview)                             no      ...2.0/Sections/SpeedOfLight_RooflineChart.section
Tile                                      Tile Statistics                                                          no      ...A Nsight Compute/2026.2.0/Sections/Tile.section
WarpStateStats                            Warp State Statistics                                                    no      ...e/2026.2.0/Sections/WarpStateStatistics.section
WorkloadDistribution                      GPU and Memory Workload Distribution                                     yes     .../2026.2.0/Sections/WorkloadDistribution.section
(base) root@huawei:/home/w00441123/nvcc/printf#
```

2.2.3.3. **ncu --list-metrics**

```
(base) root@huawei:/home/w00441123/nvcc/printf# ncu --list-metrics
smsp__warps_active.sum.peak_sustained
 -smsp__warps_active.sum.peak_sustained
smsp__warps_active.min.peak_sustained
 -smsp__warps_active.min.peak_sustained
smsp__warps_active.max.peak_sustained
 -smsp__warps_active.max.peak_sustained
smsp__warps_active.avg.peak_sustained
 -smsp__warps_active.avg.peak_sustained
smsp__maximum_warps_avg_per_active_cycle
 -smsp__maximum_warps_avg_per_active_cycle
smsp__cycles_elapsed.sum
 -smsp__cycles_elapsed.sum
smsp__cycles_elapsed.min
 -smsp__cycles_elapsed.min
```

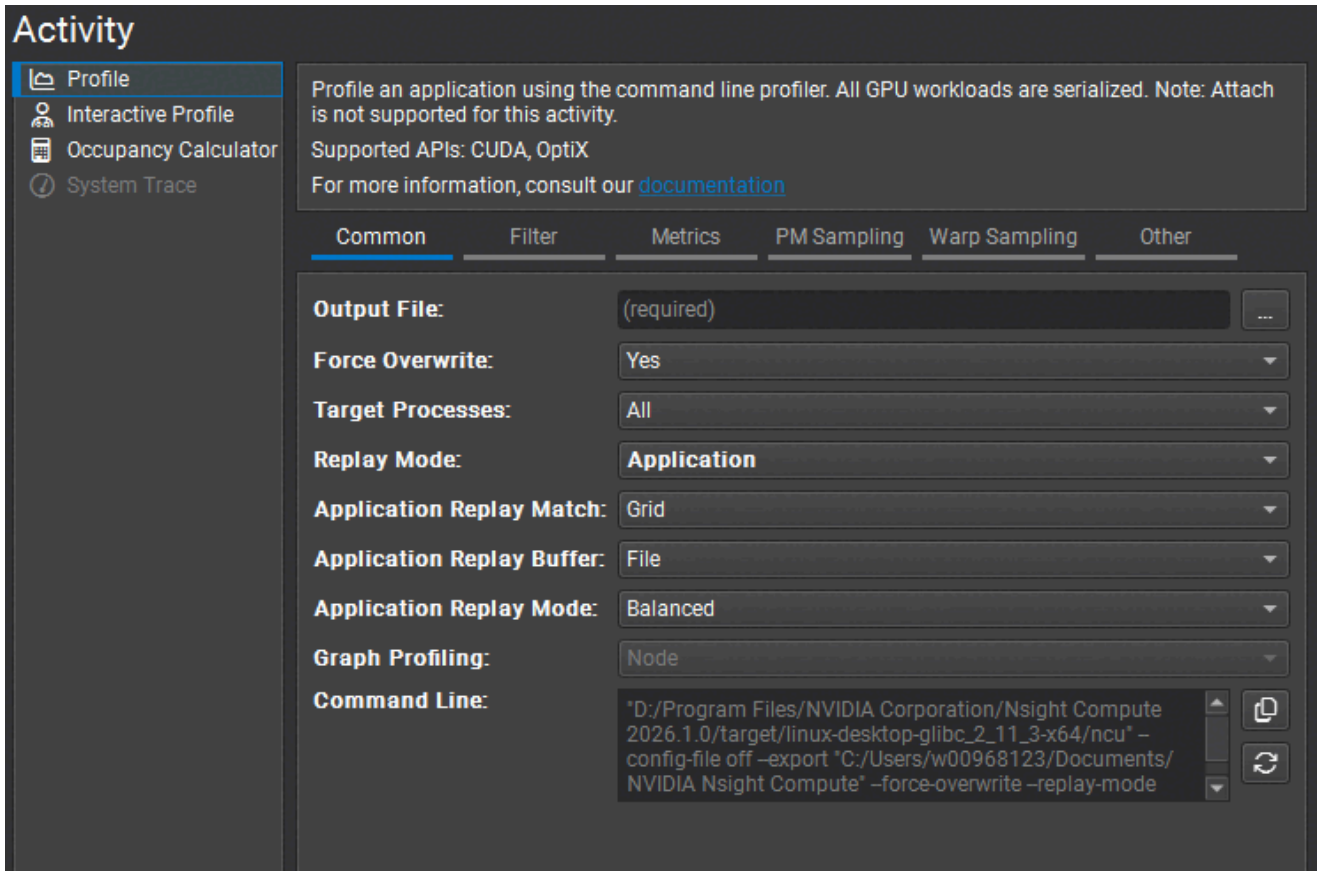
2.2.3.4. **ncu --query-metrics**

```
(base) root@huawei:/home/w00441123/nvcc/printf# ncu --query-metrics
Device NVIDIA A100-SXM4-80GB (GA100)
```

Metric Name	Metric Type	Metric Unit	Metric Description
dram__bytes	Counter	byte	# of bytes accessed in DRAM
dram__bytes_read	Counter	byte	# of bytes read from DRAM
dram__bytes_write	Counter	byte	# of bytes written to DRAM
dram__cycles_active	Counter	cycle	# of cycles where DRAM was active
dram__cycles_active_read	Counter	cycle	# of cycles where DRAM was active for reads
dram__cycles_active_write	Counter	cycle	# of cycles where DRAM was active for writes
dram__cycles_elapsed	Counter	cycle	# of elapsed DRAM memory clock cycles
dram__cycles_in_frame	Counter	cycle	# of cycles in user-defined frame
dram__cycles_in_region	Counter	cycle	# of cycles in user-defined region
dram__sectors	Counter	sector	# of sectors accessed in DRAM
dram__sectors_read	Counter	sector	# of sectors read from DRAM
dram__sectors_write	Counter	sector	# of sectors written to DRAM
dram__throughput	Throughput	%	DRAM throughput
fbpa__cycles_active	Counter	cycle	# of cycles where FBPA
fbpa__cycles_elapsed	Counter	cycle	# of cycles elapsed on FBPA
fbpa__cycles_in_frame	Counter	cycle	# of cycles in user-defined frame
fbpa__cycles_in_region	Counter	cycle	# of cycles in user-defined region
fbpa__dram_cycles_elapsed	Counter	cycle	# of cycles elapsed on DRAM

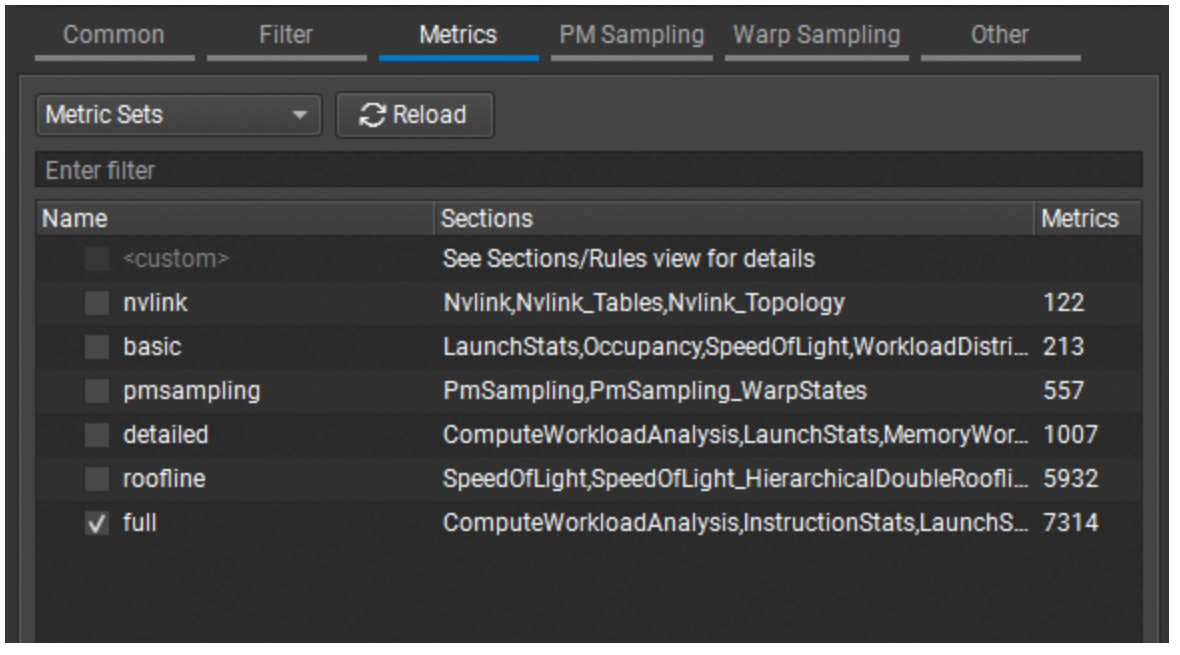
2.2.4. **界面采集配置**

- 通用配置

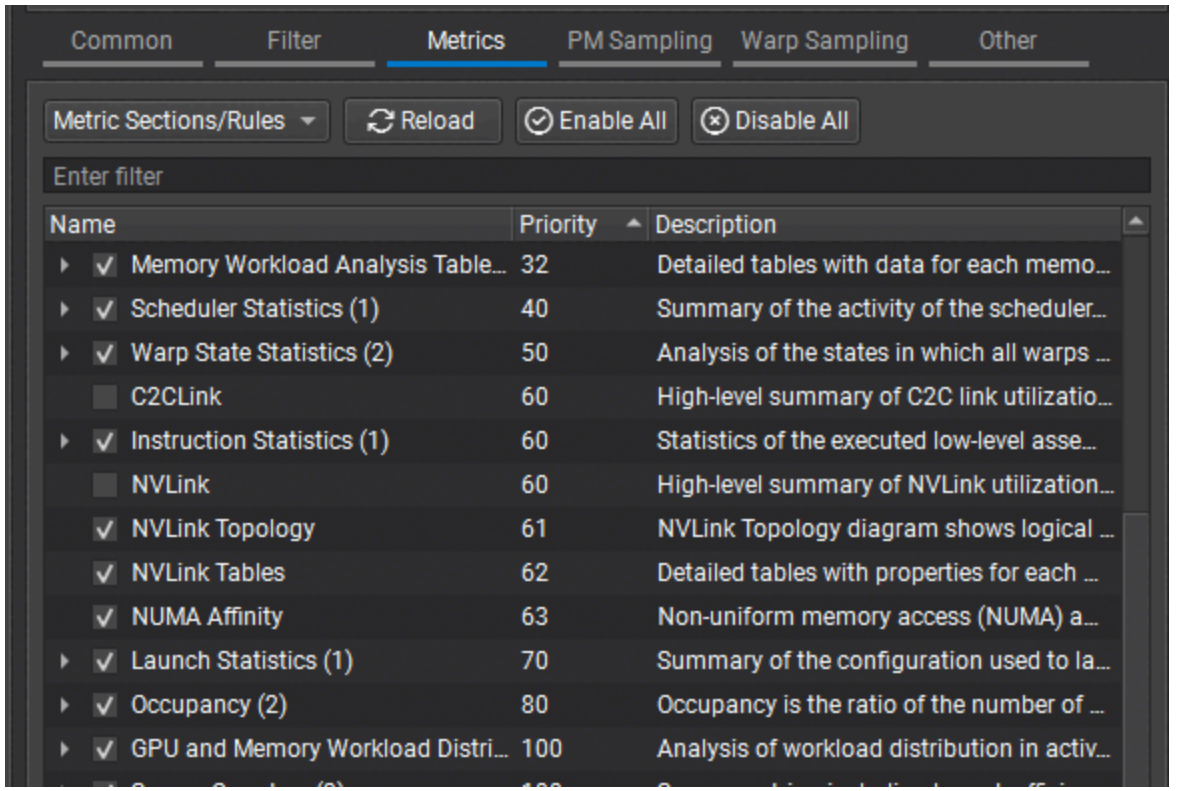


- 采集参数配置

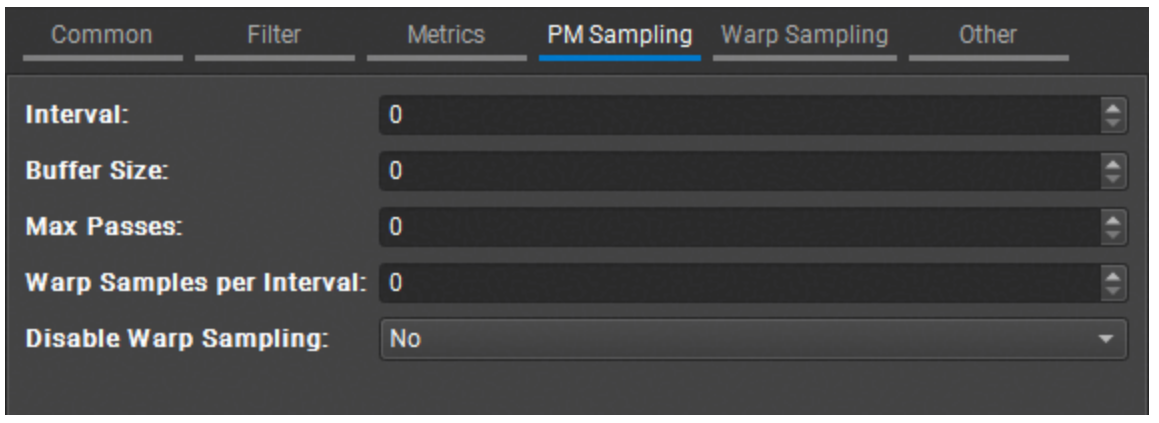
1. Set方式采集



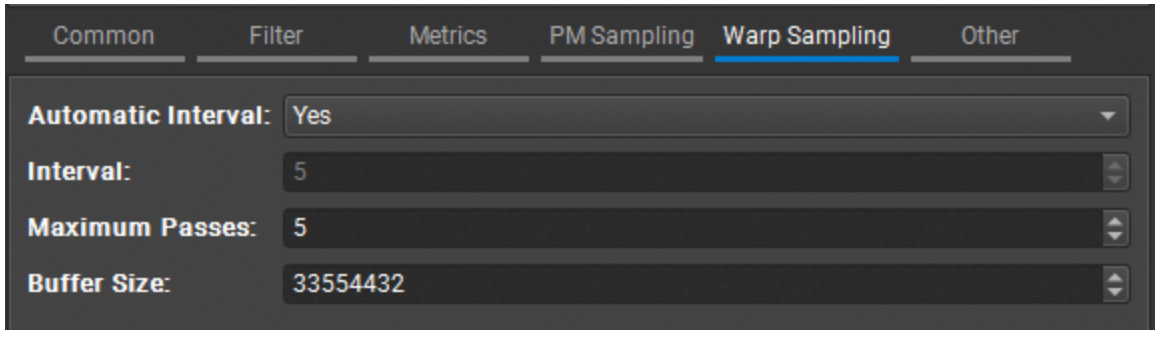
2. Section + Metrics采集



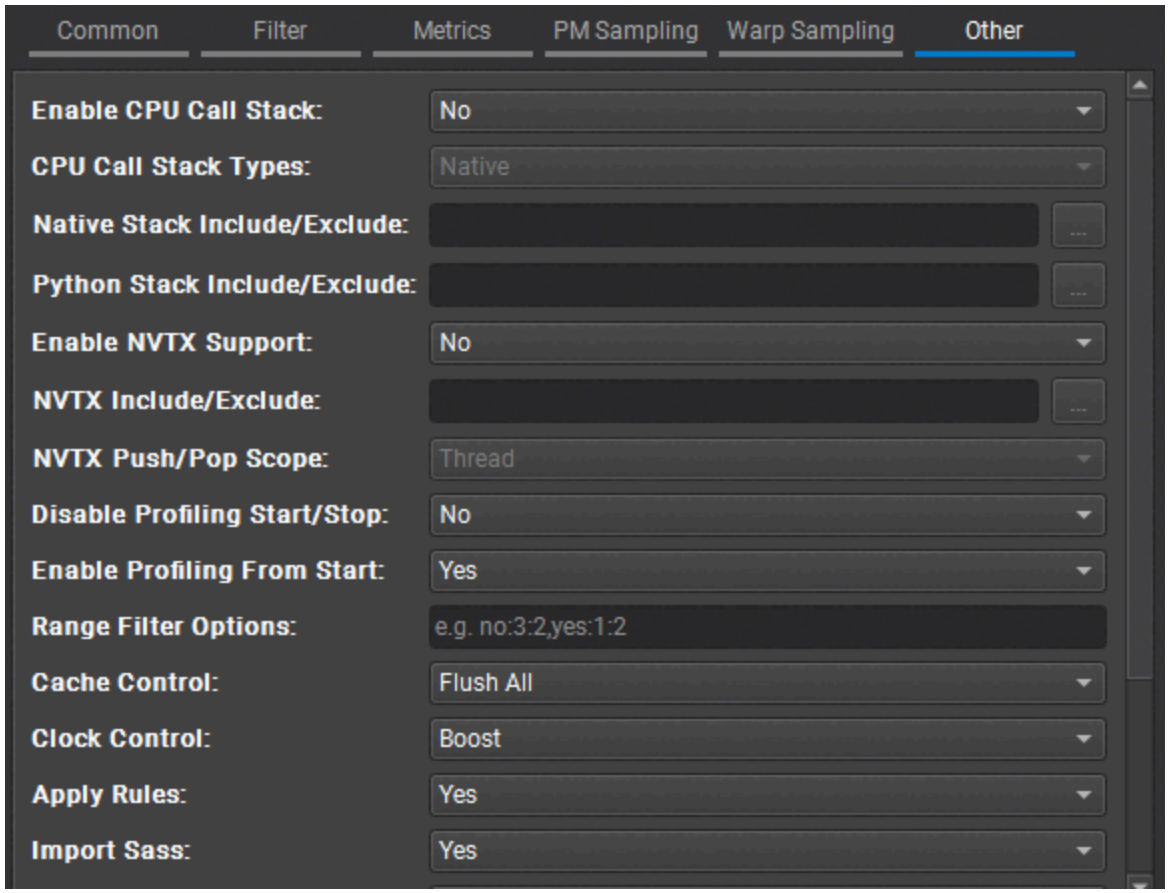
3. PM Sampling



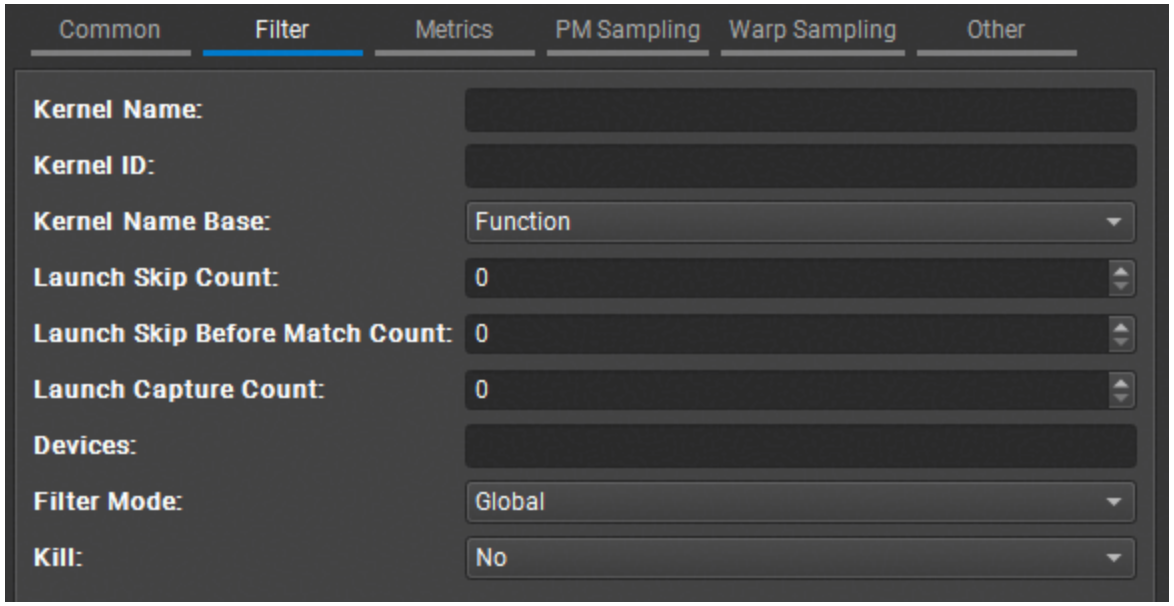
4. Warp Sampling



5. Other



6. Filter



2.3. **CUPTI**

2.3.1. **RangeProfiler API**

cpp

Full Screen

```
#include <cuda_runtime.h>
#include <cuda.h>
#include <stdio.h>
#include <stdint.h>
#include <vector>
#include <cupti_profiler_host.h>
#include <cupti_range_profiler.h>
#include <cupti_target.h>

// Global variables for Range Profiler
CUpti_RangeProfiler_Object* g_pRangeProfilerObject = NULL;

std::vector<uint8_t> g_counterDataImage;
std::vector<uint8_t> g_configImage;

CUcontext g_cuContext;
std::string g_chipName;

std::vector<const char*> g_metrics =
{
    "sm__warps_launched.sum",     // Number of warps launched
    "sm__ctas_launched.sum"
};

// CUDA kernel for vector addition
__global__ void VectorAdd(const float *A, const float *B, float *C, int N)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N)
        C[idx] = A[idx] + B[idx];
}

// Helper function declarations
void InitializeAndEnableRangeProfiler(CUcontext cuContext);
void ConfigureRangeProfiler(CUpti_ProfilerRange range, CUpti_ProfilerReplayMode replayMode, size_t numOfRanges);
void StartRangeProfiler();
void StopRangeProfiler();
void DecodeAndPrintProfilingData();
void CleanupRangeProfiler();

int main()
{
    const int vectorLen = 1024 * 1024;
    const size_t size = vectorLen * sizeof(float);

    // Initialize CUDA and create context
    cuInit(0);
    cuCtxCreate(&g_cuContext, (CUctxCreateParams*)0, 0, 0);

    // Initialize and Enable Range Profiler
    InitializeAndEnableRangeProfiler(g_cuContext);
```

```
52
53      // Configure Range Profiler
54      constexpr size_t numOfRanges = 10;
55      ConfigureRangeProfiler(CUPTI_AutoRange, CUPTI_KernelReplay, numOfRanges);
56
57      // Setup CUDA workload
58      float *h_A = (float*)malloc(size);
59      float *h_B = (float*)malloc(size);
60      float *h_C = (float*)malloc(size);
61
62      for (int i = 0; i < vectorLen; ++i) {
63          h_A[i] = rand() / (float)RAND_MAX;
64          h_B[i] = rand() / (float)RAND_MAX;
65      }
66
67      float *d_A, *d_B, *d_C;
68      cudaMalloc((void **)&d_A, size);
69      cudaMalloc((void **)&d_B, size);
70      cudaMalloc((void **)&d_C, size);
71
72      cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
73      cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);
74
75      int threadsPerBlock = 128;
76      int blocksPerGrid = (vectorLen + threadsPerBlock - 1) / threadsPerBlock;
77
78      // Start Range Profiling
79      StartRangeProfiler();
80
81      // Launch CUDA workload
82      VectorAdd<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, vectorLen);
83
84      // Stop Range Profiling
85      StopRangeProfiler();
86
87      // Decode and evaluate profiling data
88      DecodeAndPrintProfilingData();
89
90      // Cleanup Range Profiler
91      CleanupRangeProfiler();
92
93      cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);
94      cudaDeviceSynchronize();
95
96      // Cleanup
97      cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
98      free(h_A); free(h_B); free(h_C);
99      return 0;
100 }
101
```

```
102 void CreateConfigImage()
103 {
104     CUpti_Profiler_Host_Initialize_Params hostInitializeParams = {CUpti_Profiler_Host_Initialize_Params_STRUCT_SIZE};
105     hostInitializeParams.profilerType = CUPTI_PROFILER_TYPE_RANGE_PROFILER;
106     hostInitializeParams.pChipName = g_chipName.c_str();
107     hostInitializeParams.pCounterAvailabilityImage = nullptr;
108     cuptiProfilerHostInitialize(&hostInitializeParams);
109     CUpti_Profiler_Host_Object* pHostObject = hostInitializeParams.pHostObject;
110 
111     CUpti_Profiler_Host_ConfigAddMetrics_Params configAddMetricsParams {CUpti_Profiler_Host_ConfigAddMetrics_Params_STRUCT_SIZE};
112     configAddMetricsParams.pHostObject = pHostObject;
113     configAddMetricsParams.ppMetricNames = g_metrics.data();
114     configAddMetricsParams.numMetrics = g_metrics.size();
115     cuptiProfilerHostConfigAddMetrics(&configAddMetricsParams);
116 
117     CUpti_Profiler_Host_GetConfigImageSize_Params getConfigImageSizeParams {CUpti_Profiler_Host_GetConfigImageSize_Params_STRUCT_SIZE};
118     getConfigImageSizeParams.pHostObject = pHostObject;
119     cuptiProfilerHostGetConfigImageSize(&getConfigImageSizeParams);
120     g_configImage.resize(getConfigImageSizeParams.configImageSize);
121 
122     CUpti_Profiler_Host_GetConfigImage_Params getConfigImageParams = {CUpti_Profiler_Host_GetConfigImage_Params_STRUCT_SIZE};
123     getConfigImageParams.pHostObject = pHostObject;
124     getConfigImageParams.pConfigImage = g_configImage.data();
125     getConfigImageParams.configImageSize = g_configImage.size();
126     cuptiProfilerHostGetConfigImage(&getConfigImageParams);
127 
128     CUpti_Profiler_Host_GetNumOfPasses_Params getNumOfPassesParam {CUpti_Profiler_Host_GetNumOfPasses_Params_STRUCT_SIZE};
129     getNumOfPassesParam.pConfigImage = g_configImage.data();
130     getNumOfPassesParam.configImageSize = g_configImage.size();
131     cuptiProfilerHostGetNumOfPasses(&getNumOfPassesParam);
132     printf("Num of Passes: %d\n", getNumOfPassesParam.numOfPasses);
133 
134     CUpti_Profiler_Host_Deinitialize_Params deinitializeParams = {CUpti_Profiler_Host_Deinitialize_Params_STRUCT_SIZE};
135     deinitializeParams.pHostObject = pHostObject;
136     cuptiProfilerHostDeinitialize(&deinitializeParams);
137     pHostObject = nullptr;
138 }
139 
140 void EvaluateAndPrintForRange(size_t rangeIndex, CUpti_Profiler_Host_Object* pHostObject)
141 {
142     std::vector<double> metricValues(g_metrics.size());
143     CUpti_Profiler_Host_EvaluateToGpuValues_Params evalauateToGpuValuesParams {CUpti_Profiler_Host_EvaluateToGpuValues_Params_STRUCT_SIZ
144     evalauateToGpuValuesParams.pHostObject = pHostObject;
145     evalauateToGpuValuesParams.pCounterDataImage = g_counterDataImage.data();
146     evalauateToGpuValuesParams.counterDataImageSize = g_counterDataImage.size();
147     evalauateToGpuValuesParams.ppMetricNames = g_metrics.data();
148     evalauateToGpuValuesParams.numMetrics = g_metrics.size();
149     evalauateToGpuValuesParams.rangeIndex = rangeIndex;
150     evalauateToGpuValuesParams.pMetricValues = metricValues.data();
151     cuptiProfilerHostEvaluateToGpuValues(&evalauateToGpuValuesParams);
152 
```

```
    for (size_t i = 0; i < g_metrics.size(); ++i) {
        printf("\t%s: %f\n", g_metrics[i], metricValues[i]);
    }
    printf("\n");
}

void EvaluateAndPrintAllRanges(size_t numOfRanges)
{
    CUpti_Profiler_Host_Initialize_Params hostInitializeParams = {CUpti_Profiler_Host_Initialize_Params_STRUCT_SIZE};
    hostInitializeParams.profilerType = CUPTI_PROFILER_TYPE_RANGE_PROFILER;
    hostInitializeParams.pChipName = g_chipName.c_str();
    hostInitializeParams.pCounterAvailabilityImage = nullptr;
    cuptiProfilerHostInitialize(&hostInitializeParams);
    CUpti_Profiler_Host_Object* pHostObject = hostInitializeParams.pHostObject;

    for (size_t i = 0; i < numOfRanges; ++i)
    {
        CUpti_RangeProfiler_CounterData_GetRangeInfo_Params getRangeInfoParams = {CUpti_RangeProfiler_CounterData_GetRangeInfo_Params_STI
        getRangeInfoParams.counterDataImageSize = g_counterDataImage.size();
        getRangeInfoParams.pCounterDataImage = g_counterDataImage.data();
        getRangeInfoParams.rangeIndex = i;
        getRangeInfoParams.rangeDelimiter = "/";
        cuptiRangeProfilerCounterDataGetRangeInfo(&getRangeInfoParams);

        printf("Range: %s\n", getRangeInfoParams.rangeName);
        printf("Metric Values:\n");
        EvaluateAndPrintForRange(i, pHostObject);
    }

    CUpti_Profiler_Host_Deinitialize_Params deinitializeParams = {CUpti_Profiler_Host_Deinitialize_Params_STRUCT_SIZE};
    deinitializeParams.pHostObject = pHostObject;
    cuptiProfilerHostDeinitialize(&deinitializeParams);
    pHostObject = nullptr;
}

void InitializeAndEnableRangeProfiler(CUcontext cuContext)
{
    // Initialize CUPTI Profiler
    CUpti_Profiler_Initialize_Params profilerInitializeParams = { CUpti_Profiler_Initialize_Params_STRUCT_SIZE };
    cuptiProfilerInitialize(&profilerInitializeParams);

    CUdevice device;
    cuCtxGetDevice(&device);
    CUpti_Device_GetChipName_Params getChipNameParams = { CUpti_Device_GetChipName_Params_STRUCT_SIZE };
    getChipNameParams.deviceIndex = (size_t)device;
    cuptiDeviceGetChipName(&getChipNameParams);
    g_chipName = std::string(getChipNameParams.pChipName);
    printf("Chip Name: %s\n", g_chipName.c_str());

    // Enable Range profiler
```

```
    CUpti_RangeProfiler_Enable_Params enableRange = { CUpti_RangeProfiler_Enable_Params_STRUCT_SIZE };
    enableRange.ctx = cuContext;
    cuptiRangeProfilerEnable(&enableRange);
    g_pRangeProfilerObject = enableRange.pRangeProfilerObject;
}

void CreateCounterDataImage(size_t maxNumOfRangesInCounterDataImage)
{
    // Get counter data size
    CUpti_RangeProfiler_GetCounterDataSize_Params ctDataSize = { CUpti_RangeProfiler_GetCounterDataSize_Params_STRUCT_SIZE };
    ctDataSize.pRangeProfilerObject = g_pRangeProfilerObject;
    ctDataSize.pMetricNames = g_metrics.data();
    ctDataSize.numMetrics = g_metrics.size();
    ctDataSize.maxNumOfRanges = maxNumOfRangesInCounterDataImage;
    ctDataSize.maxNumRangeTreeNodes = maxNumOfRangesInCounterDataImage;
    cuptiRangeProfilerGetCounterDataSize(&ctDataSize);

    // Initialize counter data image
    g_counterDataImage.resize(ctDataSize.counterDataSize);
    CUpti_RangeProfiler_CounterDataImage_Initialize_Params initCtImg = { CUpti_RangeProfiler_CounterDataImage_Initialize_Params_STRUCT_S
    initCtImg.pRangeProfilerObject = g_pRangeProfilerObject;
    initCtImg.pCounterData = g_counterDataImage.data();
    initCtImg.counterDataSize = g_counterDataImage.size();
    cuptiRangeProfilerCounterDataImageInitialize(&initCtImg);
}

void ConfigureRangeProfiler(CUpti_ProfilerRange range, CUpti_ProfilerReplayMode replayMode, size_t numOfRanges)
{
    // Create config image
    CreateConfigImage();

    // Create counter data image
    CreateCounterDataImage(numOfRanges);

    CUpti_RangeProfiler_SetConfig_Params setConfig = { CUpti_RangeProfiler_SetConfig_Params_STRUCT_SIZE };
    setConfig.pRangeProfilerObject = g_pRangeProfilerObject;
    setConfig.configSize = g_configImage.size();
    setConfig.pConfig = g_configImage.data();
    setConfig.counterDataImageSize = g_counterDataImage.size();
    setConfig.pCounterDataImage = g_counterDataImage.data();
    setConfig.range = range;
    setConfig.replayMode = replayMode;
    setConfig.maxRangesPerPass = numOfRanges;
    setConfig.numNestingLevels = 1;
    setConfig.minNestingLevel = 1;
    setConfig.passIndex = 0;
    setConfig.targetNestingLevel = 0;
    cuptiRangeProfilerSetConfig(&setConfig);
}

void StartRangeProfiler()
```

```
void StartRangeProfiler()
{
    CUpti_RangeProfiler_Start_Params startRangeProfiler = { CUpti_RangeProfiler_Start_Params_STRUCT_SIZE };
    startRangeProfiler.pRangeProfilerObject = g_pRangeProfilerObject;
    cuptiRangeProfilerStart(&startRangeProfiler);
}

void StopRangeProfiler()
{
    CUpti_RangeProfiler_Stop_Params stopRangeProfiler = { CUpti_RangeProfiler_Stop_Params_STRUCT_SIZE };
    stopRangeProfiler.pRangeProfilerObject = g_pRangeProfilerObject;
    cuptiRangeProfilerStop(&stopRangeProfiler);
}

void DecodeAndPrintProfilingData()
{
    // Decode profiling data
    CUpti_RangeProfiler_DecodeData_Params decodeData = { CUpti_RangeProfiler_DecodeData_Params_STRUCT_SIZE };
    decodeData.pRangeProfilerObject = g_pRangeProfilerObject;
    cuptiRangeProfilerDecodeData(&decodeData);

    // Get information about profiled ranges
    CUpti_RangeProfiler_GetCounterDataInfo_Params cdiParams = { CUpti_RangeProfiler_GetCounterDataInfo_Params_STRUCT_SIZE };
    cdiParams.pCounterDataImage = g_counterDataImage.data();
    cdiParams.counterDataImageSize = g_counterDataImage.size();
    cuptiRangeProfilerGetCounterDataInfo(&cdiParams);
    printf("Number of profiled ranges: %zu\n", cdiParams.numTotalRanges);

    // Evaluate and print profiling data
    const size_t numRangesToPrint = cdiParams.numTotalRanges > 10 ? 10 : cdiParams.numTotalRanges;
    EvaluateAndPrintAllRanges(numRangesToPrint);
}

void CleanupRangeProfiler()
{
    // Disable Range profiler
    CUpti_RangeProfiler_Disable_Params disableRangeProfiler = { CUpti_RangeProfiler_Disable_Params_STRUCT_SIZE };
    disableRangeProfiler.pRangeProfilerObject = g_pRangeProfilerObject;
    cuptiRangeProfilerDisable(&disableRangeProfiler);

    // Deinitialize profiler
    CUpti_Profiler_DeInitialize_Params profilerDeInitializeParams = { CUpti_Profiler_DeInitialize_Params_STRUCT_SIZE };
    cuptiProfilerDeInitialize(&profilerDeInitializeParams);
}
```

2.3.2. **PM Sampling**

cpp

Full Screen

```
#include <cuda_runtime.h>
```

```
#include <cuda.h>
#include <stdio.h>
#include <vector>
#include <cupti_target.h>
#include <cupti_pmsampling.h>
#include <cupti_profiler_target.h>
#include <cupti_profiler_host.h>

// Global variables for PM Sampling
CUpti_PmSampling_Object* g_pPmSamplingObject = NULL;

std::string g_chipName;
std::vector<uint8_t> g_configImage;
std::vector<uint8_t> g_counterDataImage;

std::vector<const char*> g_metrics =
{
    "gr__cycles_active.avg",     // GPU Active Cycles
    "gr__cycles_elapsed.max",     // GPU Elapsed Cycles
    "sm__cycles_active.avg"     // SM Active Cycles
};

// CUDA kernel for vector addition
__global__ void VectorAdd(const int *A, const int *B, int *C, int N)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N)
        C[idx] = A[idx] + B[idx];
}

// Helper function to initialize PM Sampling
void InitializeAndEnablePmSampling(int deviceIndex);
void ConfigurePmSampling(uint64_t hardwareBufferSize, uint64_t samplingInterval);
void StartPmSampling();
void StopPmSampling();
void DecodeAndPrintSamplingData();
void CleanupPmSampling();

int main()
{
    const int vectorLen = 4096 * 4096 * 2;
    size_t size = vectorLen * sizeof(int);

    // Initialize CUDA
    cuInit(0);

    // Setup CUDA workload
    int *h_A = (int*)malloc(size);
    int *h_B = (int*)malloc(size);
    int *h_C = (int*)malloc(size);
```

```
52
53      for (int i = 0; i < vectorLen; ++i) {
54          h_A[i] = i;
55          h_B[i] = i * 2;
56      }
57
58      int *d_A, *d_B, *d_C;
59      cudaMalloc((void **)&d_A, size);
60      cudaMalloc((void **)&d_B, size);
61      cudaMalloc((void **)&d_C, size);
62
63      cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
64      cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);
65
66      // Initalize and Enable PM Sampling
67      constexpr int deviceIndex = 0;
68      InitializeAndEnablePmSampling(deviceIndex);
69
70      // Configure PM Sampling
71      constexpr size_t hardwareBufferSize = 512 * 1024 * 1024; // 512MB buffer
72      constexpr uint64_t samplingInterval = 100000; // 100us interval
73      ConfigurePmSampling(hardwareBufferSize, samplingInterval);
74
75      // Start PM Sampling
76      StartPmSampling();
77
78      // Launch CUDA workload
79      int threadsPerBlock = 512;
80      int blocksPerGrid = (vectorLen + threadsPerBlock - 1) / threadsPerBlock;
81      for (int i = 0; i < 100; ++i) {
82          VectorAdd<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, vectorLen);
83      }
84      cudaDeviceSynchronize();
85
86      // Stop PM Sampling
87      StopPmSampling();
88
89      // Decode and print sampling data
90      DecodeAndPrintSamplingData();
91
92      // Cleanup PM Sampling
93      CleanupPmSampling();
94
95      // Cleanup
96      cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);
97      cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
98      free(h_A); free(h_B); free(h_C);
99      return 0;
100 }
101
102 void CreateConfigImage()
```

```
void CreateConfigImage()
{
    CUpti_Profiler_Host_Initialize_Params hostInitializeParams = {CUpti_Profiler_Host_Initialize_Params_STRUCT_SIZE};
    hostInitializeParams.profilerType = CUPTI_PROFILER_TYPE_PM_SAMPLING;
    hostInitializeParams.pChipName = g_chipName.c_str();
    hostInitializeParams.pCounterAvailabilityImage = nullptr;
    cuptiProfilerHostInitialize(&hostInitializeParams);
    CUpti_Profiler_Host_Object* pHostObject = hostInitializeParams.pHostObject;

    CUpti_Profiler_Host_ConfigAddMetrics_Params configAddMetricsParams {CUpti_Profiler_Host_ConfigAddMetrics_Params_STRUCT_SIZE};
    configAddMetricsParams.pHostObject = pHostObject;
    configAddMetricsParams.ppMetricNames = g_metrics.data();
    configAddMetricsParams.numMetrics = g_metrics.size();
    cuptiProfilerHostConfigAddMetrics(&configAddMetricsParams);

    CUpti_Profiler_Host_GetConfigImageSize_Params getConfigImageSizeParams {CUpti_Profiler_Host_GetConfigImageSize_Params_STRUCT_SIZE};
    getConfigImageSizeParams.pHostObject = pHostObject;
    cuptiProfilerHostGetConfigImageSize(&getConfigImageSizeParams);
    g_configImage.resize(getConfigImageSizeParams.configImageSize);

    CUpti_Profiler_Host_GetConfigImage_Params getConfigImageParams = {CUpti_Profiler_Host_GetConfigImage_Params_STRUCT_SIZE};
    getConfigImageParams.pHostObject = pHostObject;
    getConfigImageParams.pConfigImage = g_configImage.data();
    getConfigImageParams.configImageSize = g_configImage.size();
    cuptiProfilerHostGetConfigImage(&getConfigImageParams);

    CUpti_Profiler_Host_GetNumOfPasses_Params getNumOfPassesParam {CUpti_Profiler_Host_GetNumOfPasses_Params_STRUCT_SIZE};
    getNumOfPassesParam.pConfigImage = g_configImage.data();
    getNumOfPassesParam.configImageSize = g_configImage.size();
    cuptiProfilerHostGetNumOfPasses(&getNumOfPassesParam);
    printf("Num of Passes: %d\n", getNumOfPassesParam.numOfPasses);

    CUpti_Profiler_Host_Deinitialize_Params deinitializeParams = {CUpti_Profiler_Host_Deinitialize_Params_STRUCT_SIZE};
    deinitializeParams.pHostObject = pHostObject;
    cuptiProfilerHostDeinitialize(&deinitializeParams);
    pHostObject = nullptr;
}

void EvaluateAndPrintForSample(size_t sampleIndex, CUpti_Profiler_Host_Object* pHostObject, std::vector<uint8_t>& counterDataImage)
{
    CUpti_PmSampling_CounterData_GetSampleInfo_Params getSampleInfoParams = {CUpti_PmSampling_CounterData_GetSampleInfo_Params_STRUCT_SI
    getSampleInfoParams.pPmSamplingObject = g_pPmSamplingObject;
    getSampleInfoParams.pCounterDataImage = counterDataImage.data();
    getSampleInfoParams.counterDataImageSize = counterDataImage.size();
    getSampleInfoParams.sampleIndex = sampleIndex;
    cuptiPmSamplingCounterDataGetSampleInfo(&getSampleInfoParams);
    printf("Sample Index: %zu, Start Timestamp: %llu, End Timestamp: %llu\n", sampleIndex, getSampleInfoParams.startTimestamp, getSample

    std::vector<double> metricValues(g_metrics.size());
    CUpti_Profiler_Host_EvaluateToGpuValues_Params evalauateToGpuValuesParams {CUpti_Profiler_Host_EvaluateToGpuValues_Params_STRUCT_SIZ
    evalauateToGpuValuesParams.pHostObject = pHostObject;
```

```
    evalauateToGpuValuesParams.pCounterDataImage = counterDataImage.data();
    evalauateToGpuValuesParams.counterDataImageSize = counterDataImage.size();
    evalauateToGpuValuesParams.ppMetricNames = g_metrics.data();
    evalauateToGpuValuesParams.numMetrics = g_metrics.size();
    evalauateToGpuValuesParams.rangeIndex = sampleIndex;
    evalauateToGpuValuesParams.pMetricValues = metricValues.data();
    cuptiProfilerHostEvaluateToGpuValues(&evalauateToGpuValuesParams);

    for (size_t i = 0; i < g_metrics.size(); ++i) {
        printf("\t%s: %f\n", g_metrics[i], metricValues[i]);
    }
    printf("\n");
}

void EvaluateAndPrintAllSamples(size_t numOfSamples)
{
    CUpti_Profiler_Host_Initialize_Params hostInitializeParams = {CUpti_Profiler_Host_Initialize_Params_STRUCT_SIZE};
    hostInitializeParams.profilerType = CUPTI_PROFILER_TYPE_PM_SAMPLING;
    hostInitializeParams.pChipName = g_chipName.c_str();
    hostInitializeParams.pCounterAvailabilityImage = nullptr;
    cuptiProfilerHostInitialize(&hostInitializeParams);
    CUpti_Profiler_Host_Object* pHostObject = hostInitializeParams.pHostObject;

    for (size_t sampleIndex = 0; sampleIndex < numOfSamples; ++sampleIndex) {
        EvaluateAndPrintForSample(sampleIndex, pHostObject, g_counterDataImage);
    }

    CUpti_Profiler_Host_Deinitialize_Params deinitializeParams = {CUpti_Profiler_Host_Deinitialize_Params_STRUCT_SIZE};
    deinitializeParams.pHostObject = pHostObject;
    cuptiProfilerHostDeinitialize(&deinitializeParams);
    pHostObject = nullptr;
}

// Helper function to initialize PM Sampling
void InitializeAndEnablePmSampling(int deviceIndex)
{
    // Initialize CUPTI Profiler
    CUpti_Profiler_Initialize_Params profilerInitializeParams = { CUpti_Profiler_Initialize_Params_STRUCT_SIZE };
    cuptiProfilerInitialize(&profilerInitializeParams);

    CUpti_Device_GetChipName_Params getChipNameParams = { CUpti_Device_GetChipName_Params_STRUCT_SIZE };
    getChipNameParams.deviceIndex = deviceIndex;
    cuptiDeviceGetChipName(&getChipNameParams);
    g_chipName = getChipNameParams.pChipName;
    printf("Chip Name: %s\n", g_chipName.c_str());

    // Enable PM sampling
    CUpti_PmSampling_Enable_Params enableParams = { CUpti_PmSampling_Enable_Params_STRUCT_SIZE };
    enableParams.deviceIndex = deviceIndex;
    cuptiPmSamplingEnable(&enableParams);
```

```
    g_pPmSamplingObject = enableParams.pPmSamplingObject;
}

void ConfigurePmSampling(uint64_t hardwareBufferSize, uint64_t samplingInterval)
{
    // Need to create the config image which will have the scheduling information for the metrics
    CreateConfigImage();

    // Set configuration
    CUpti_PmSampling_SetConfig_Params setConfigParams = { CUpti_PmSampling_SetConfig_Params_STRUCT_SIZE };
    setConfigParams.pPmSamplingObject = g_pPmSamplingObject;
    setConfigParams.configSize = g_configImage.size();
    setConfigParams.pConfig = g_configImage.data();
    setConfigParams.hardwareBufferSize = hardwareBufferSize;
    setConfigParams.samplingInterval = samplingInterval;
    setConfigParams.triggerMode = CUPTI_PM_SAMPLING_TRIGGER_MODE_GPU_SYSCLK_INTERVAL;
    cuptiPmSamplingSetConfig(&setConfigParams);
}

// Helper function to start PM sampling
void StartPmSampling()
{
    CUpti_PmSampling_Start_Params startParams = { CUpti_PmSampling_Start_Params_STRUCT_SIZE };
    startParams.pPmSamplingObject = g_pPmSamplingObject;
    cuptiPmSamplingStart(&startParams);
}

// Helper function to stop PM sampling
void StopPmSampling()
{
    CUpti_PmSampling_Stop_Params stopParams = { CUpti_PmSampling_Stop_Params_STRUCT_SIZE };
    stopParams.pPmSamplingObject = g_pPmSamplingObject;
    cuptiPmSamplingStop(&stopParams);
}

// Helper function to create counter data image
void CreateCounterDataImage(uint64_t maxSamplesInCounterDataImage)
{
    CUpti_PmSampling_GetCounterDataSize_Params getCounterDataSizeParams = { CUpti_PmSampling_GetCounterDataSize_Params_STRUCT_SIZE };
    getCounterDataSizeParams.pPmSamplingObject = g_pPmSamplingObject;
    getCounterDataSizeParams.numMetrics = g_metrics.size();
    getCounterDataSizeParams.pMetricNames = g_metrics.data();
    getCounterDataSizeParams.maxSamples = maxSamplesInCounterDataImage;
    cuptiPmSamplingGetCounterDataSize(&getCounterDataSizeParams);

    g_counterDataImage.resize(getCounterDataSizeParams.counterDataSize);
    CUpti_PmSampling_CounterDataImage_Initialize_Params initializeParams = { CUpti_PmSampling_CounterDataImage_Initialize_Params_STRUCT_S
    initializeParams.pPmSamplingObject = g_pPmSamplingObject;
    initializeParams.counterDataSize = g_counterDataImage.size();
    initializeParams.pCounterData = g_counterDataImage.data();
    cuptiPmSamplingCounterDataImageInitialize(&initializeParams);
```

```
}

// Helper function to decode and print sampling data
void DecodeAndPrintSamplingData()
{
    // Create counter data image which will store the decoded data from the hardware buffer
    constexpr uint64_t maxSamplesInCounterDataImage = 10000;
    CreateCounterDataImage(maxSamplesInCounterDataImage);

    // Decode sampling data
    CUpti_PmSampling_DecodeData_Params decodeParams = { CUpti_PmSampling_DecodeData_Params_STRUCT_SIZE };
    decodeParams.pPmSamplingObject = g_pPmSamplingObject;
    decodeParams.pCounterDataImage = g_counterDataImage.data();
    decodeParams.counterDataImageSize = g_counterDataImage.size();
    cuptiPmSamplingDecodeData(&decodeParams);

    // Get information about decoded data
    CUpti_PmSampling_GetCounterDataInfo_Params counterDataInfo = { CUpti_PmSampling_GetCounterDataInfo_Params_STRUCT_SIZE };
    counterDataInfo.pCounterDataImage = g_counterDataImage.data();
    counterDataInfo.counterDataImageSize = g_counterDataImage.size();
    cuptiPmSamplingGetCounterDataInfo(&counterDataInfo);
    printf("Number of completed samples: %zu\n", counterDataInfo.numCompletedSamples);

    // Print sample information (first 10 samples)
    size_t maxSamplesToShow = (counterDataInfo.numCompletedSamples > 10) ? 10 : counterDataInfo.numCompletedSamples;
    EvaluateAndPrintAllSamples(maxSamplesToShow);
}

void CleanupPmSampling()
{
    // Disable PM sampling
    CUpti_PmSampling_Disable_Params disableParams = { CUpti_PmSampling_Disable_Params_STRUCT_SIZE };
    disableParams.pPmSamplingObject = g_pPmSamplingObject;
    cuptiPmSamplingDisable(&disableParams);

    // Deinitialize profiler
    CUpti_Profiler_DeInitialize_Params profilerDeInitializeParams = { CUpti_Profiler_DeInitialize_Params_STRUCT_SIZE };
    cuptiProfilerDeInitialize(&profilerDeInitializeParams);
}
```

cpp

Full Screen

```
#include <pthread.h>
#include <atomic>

struct DecodeThreadArgs
{
    CUpti_PmSampling_Object* pPmSamplingObject;
    uint8_t* counterDataImage;
    size_t counterDataImageSize;
```

```
    size_t counterDataImageSize;
    std::atomic<bool>* stopFlag;
};

void* DecodeThread(void* args)
{
    DecodeThreadArgs* threadArgs = (DecodeThreadArgs*)args;
    while (!threadArgs->stopFlag->load())
    {
        CUpti_PmSampling_DecodeData_Params decodeParams = { CUpti_PmSampling_DecodeData_Params_STRUCT_SIZE };
        decodeParams.pPmSamplingObject = threadArgs->pPmSamplingObject;
        decodeParams.pCounterDataImage = threadArgs->counterDataImage;
        decodeParams.counterDataImageSize = threadArgs->counterDataImageSize;
        cuptiPmSamplingDecodeData(&decodeParams);

        // Process decoded data here

        // Reset counter data image for next batch
        CUpti_PmSampling_CounterDataImage_Initialize_Params resetParams = { CUpti_PmSampling_CounterDataImage_Initialize_Params_STRUCT_S
        resetParams.pPmSamplingObject = threadArgs->pPmSamplingObject;
        resetParams.counterDataSize = threadArgs->counterDataImageSize;
        resetParams.pCounterData = threadArgs->counterDataImage;
        cuptiPmSamplingCounterDataImageInitialize(&resetParams);

        usleep(10000); // 10ms sleep
    }
    return NULL;
}
```

2.4. 安装部署

2.4.1. Nsight System

```
(base) root@huawei:/usr/local/cuda-13.3/nsight-systems-2026.1.3/target-linux-x64# ls
CollectX           libcrypto.so      libcupti.so.11.7                  libnvperf_target.so              libToolsInjectionNCCL64.so      nsysdk                rules
CudaGpuInfoDumper  libcrypto.so.3    libcupti.so.11.8                  libPreRunProcessInjector.so      libToolsInjectionOpenMPI64.so   nsys-launcher         samples
eBPF               libcupti.so.10.2  libcupti.so.12.9                  libsqlite3.so.0                  libToolsInjectionOpenSHMEM64.so nvgpucs               sqlite3
exporter           libcupti.so.11.0  libcupti.so.13.3                  libssl.so                        libToolsInjectionProxy64.so     nvlog.config.template vulkan-layers
GpuMetrics         libcupti.so.11.1  libelf.so                         libssl.so.3                      libToolsInjectionPython64.so    NvTraceFormat
launcher           libcupti.so.11.2  libelf.so.1                       libToolsInjection64.so           libToolsInjectionUCP64.so       nvtx
libbpf.so          libcupti.so.11.3  libLauncher64.so                  libToolsInjectionCuDNN64_7.so    libz.so                         plugins
libbpf.so.1        libcupti.so.11.4  libLinuxKeyboardInterceptorProxy.so libToolsInjectionCuDNN64_8.so    libz.so.1                       python
libbpf.so.1.3.0    libcupti.so.11.5  libnvperf_grfx_host.so            libToolsInjectionMemoryAllocator.so libz.so.1.3.1                PythonFunctionsTrace
libcap.so.2        libcupti.so.11.6  libnvperf_host.so                 libToolsInjectionMPICH64.so      nsys                            reports
(base) root@huawei:/usr/local/cuda-13.3/nsight-systems-2026.1.3/target-linux-x64#
```

```
-rwxr-xr-x  1 root root  4001344 Jun  5 10:56 libcupti.so.13.3
lrwxrwxrwx  1 root root       11 Jun  5 10:56 libelf.so -> libelf.so.1
-rwxr-xr-x  1 root root   166408 Jun  5 10:56 libelf.so.1
-rwxr-xr-x  1 root root    14232 Jun  5 10:56 libLauncher64.so
-rwxr-xr-x  1 root root   121296 Jun  5 10:56 libLinuxKeyboardInterceptorProxy.so
-rwxr-xr-x  1 root root 33930288 Jun  5 10:56 libnvperf_grfx_host.so
-rwxr-xr-x  1 root root 33844272 Jun  5 10:56 libnvperf_host.so
-rwxr-xr-x  1 root root  6002864 Jun  5 10:56 libnvperf_target.so
-rwxr-xr-x  1 root root   984624 Jun  5 10:56 libPreRunProcessInjector.so
-rwxr-xr-x  1 root root 23343984 Jun  5 10:56 libsqlite3.so.0
lrwxrwxrwx  1 root root       11 Jun  5 10:56 libssl.so -> libssl.so.3
-rwxr-xr-x  1 root root  1274064 Jun  5 10:56 libssl.so.3
-rwxr-xr-x  1 root root 16370296 Jun  5 10:56 libToolsInjection64.so
-rwxr-xr-x  1 root root  2015608 Jun  5 10:56 libToolsInjectionCuDNN64_7.so
-rwxr-xr-x  1 root root  2038080 Jun  5 10:56 libToolsInjectionCuDNN64_8.so
-rwxr-xr-x  1 root root    98864 Jun  5 10:56 libToolsInjectionMemoryAllocator.so
-rwxr-xr-x  1 root root  1130792 Jun  5 10:56 libToolsInjectionMPICH64.so
-rwxr-xr-x  1 root root  1722888 Jun  5 10:56 libToolsInjectionNCCL64.so
-rwxr-xr-x  1 root root  1138984 Jun  5 10:56 libToolsInjectionOpenMPI64.so
-rwxr-xr-x  1 root root  1572520 Jun  5 10:56 libToolsInjectionOpenSHMEM64.so
-rwxr-xr-x  1 root root   732136 Jun  5 10:56 libToolsInjectionProxy64.so
-rwxr-xr-x  1 root root  2152744 Jun  5 10:56 libToolsInjectionPython64.so
-rwxr-xr-x  1 root root  1275848 Jun  5 10:56 libToolsInjectionUCP64.so
lrwxrwxrwx  1 root root        9 Jun  5 10:56 libz.so -> libz.so.1
lrwxrwxrwx  1 root root       13 Jun  5 10:56 libz.so.1 -> libz.so.1.3.1
-rwxr-xr-x  1 root root   100840 Jun  5 10:56 libz.so.1.3.1
-rwxr-xr-x  1 root root 82510880 Jun  5 10:56 nsys
drwxr-xr-x  3 root root       21 Jun  5 10:56 nsysdk
-rwxr-xr-x  1 root root  1926240 Jun  5 10:56 nsys-launcher
```

2.4.2. **Nsight Compute**

```
(base) root@huawei:/usr/local/cuda-13.3/nsight-compute-2026.2.0# ls
docs  EULA.txt  extras  host  ncu  ncu-ui  sections  target
(base) root@huawei:/usr/local/cuda-13.3/nsight-compute-2026.2.0# ls -al target/linux-desktop-glibc_2_11_3-x64/
total 104120
drwxr-xr-x 2 root root     4096 Jun  5 10:56 .
drwxr-xr-x 5 root root      112 Jun  5 10:56 ..
-rwxr-xr-x 1 root root 15949096 Jun  5 10:56 libcuda-injection.so
-rwxr-xr-x 1 root root   786352 Jun  5 10:56 libInterceptorInjectionTarget.so
-rwxr-xr-x 1 root root 33934384 Jun  5 10:56 libnvperf_host.so
-rwxr-xr-x 1 root root  6072496 Jun  5 10:56 libnvperf_target.so
-rwxr-xr-x 1 root root    13920 Jun  5 10:56 libTreeLauncherPlaceholder.so
-rwxr-xr-x 1 root root  3859792 Jun  5 10:56 libTreeLauncherTargetInjection.so
-rwxr-xr-x 1 root root  1477296 Jun  5 10:56 libTreeLauncherTargetUpdatePreloadInjection.so
-rwxr-xr-x 1 root root 39106000 Jun  5 10:56 ncu
-rwxr-xr-x 1 root root  4721096 Jun  5 10:56 nv-system-info-collector
-rwxr-xr-x 1 root root   663168 Jun  5 10:56 TreeLauncherSubreaper
-rwxr-xr-x 1 root root    14400 Jun  5 10:56 TreeLauncherTargetLdPreloadHelper
(base) root@huawei:/usr/local/cuda-13.3/nsight-compute-2026.2.0#
```

2.4.3. **CUPTI**

```
(base) root@huawei:/usr/local/cuda-13.3/extras/CUPTI/lib64# ls -al
total 141016
drwxr-xr-x 2 root root      235 Jun  5 10:56 .
drwxr-xr-x 6 root root       60 Jun  5 10:56 ..
-rwxr-xr-x 1 root root  1646312 Jun  5 10:56 libcheckpoint.so
lrwxrwxrwx 1 root root       14 Jun  5 10:56 libcupti.so -> libcupti.so.13
lrwxrwxrwx 1 root root       20 Jun  5 10:56 libcupti.so.13 -> libcupti.so.2026.2.0
-rwxr-xr-x 1 root root  5994368 Jun  5 10:56 libcupti.so.2026.2.0
-rw-r--r-- 1 root root 53155276 Jun  5 10:56 libcupti_static.a
-rwxr-xr-x 1 root root 33840176 Jun  5 10:56 libnvperf_host.so
-rw-r--r-- 1 root root 43036958 Jun  5 10:56 libnvperf_host_static.a
-rwxr-xr-x 1 root root  6011056 Jun  5 10:56 libnvperf_target.so
-rwxr-xr-x 1 root root   703936 Jun  5 10:56 libpcsamplingutil.so
(base) root@huawei:/usr/local/cuda-13.3/extras/CUPTI/lib64#
```

2.5. **竞分总结**

功能类别	NV	产品线	CANN工具	GAP分析
采集方式	ncu 命令行 + Nsight Compute GUI 一体；基于硬件性能计数器；section set（basic / detailed / full）分级采集	msProf / msOpProf 命令行采集 + MindStudio Insight 可视化；用 --aic-metrics 选择指标	NA	能力对应，NV 单工具体验更连贯
运行 / 重放模式	kernel replay / application replay / range replay，多次重放采全量指标	上板(真机) 与 仿真(simulator) 两种模式；支持可执行文件或算子 .o 二进制	NA	昇腾仿真模式无需真机即可分析
高层吞吐概览	GPU Speed of Light：计算(SM) 与内存吞吐相对理论峰值的百分比 + 细分贡献	计算负载分析 + 内存负载分析 + 计算内存热力图	NA	能力相当 — NV 显示的SOL更为直观
Roofline 分析	Roofline 图（算术强度 vs FLOP/s），含分层 roofline（L1 / L2 / DRAM 天花板）	Roofline 瓶颈分析图（Ops/Byte vs TOps/s），自动判定 Compute / Memory / Latency Bound，按单元/通路	NA	昇腾自动分类瓶颈更直白；NV 分层 roofline 更细
计算资源分析	Compute Workload Analysis：IPC、各 pipeline 利用率（FMA / ALU / Tensor 等)	pipeline ratio（cube / vector / scalar ratio）、Pipe 流水图	NA	1：NV的指标描述更为专业性 2：ASC 支持Pipe流水更见容易分析
内存资源分析	Memory Workload Analysis：内存图、L1 / L2 / DRAM、命中率、带宽	Cache 热力图(L2)、内存通路吞吐率波形图、MTE1/2/3 ratio	NA	维度对应

调度 / Warp 分析	Scheduler Statistics + Warp State Statistics（warp stall 原因周期采样）	Top Warp Stall Reason、核间负载分析	NA	NV 的 warp stall 采样体系更完善
占用率 Occupancy	Occupancy section（理论/实际占用率）+ 独立Occupancy Calculator	核间负载 / 负载均衡（架构上无严格 warp occupancy 概念）	NA	NV 占用率模型与计算器成熟
源码级分析	Source Counters + Source 页：CUDA-C / PTX / SASS 关联、逐行指标、stall 采样、分支效率	算子代码热点图 + 源码视图（源码 ↔ 指令集映射、耗时）	NA	NV 到 SASS 逐行更细
时间线 / 指令流水	偏单 kernel；含指令统计；跨核时间线属 Nsight Systems职责	MindStudio Insight：指令流水图 / 通算流水图 / Timeline（仿真），算子内时间线更强	NA	NV 界面使用更为流畅，体验更加优秀
基线对比 Baselines	Baselines：工具内多结果并排对比，量化优化收益	以数据文件为主，单算子基线并排对比能力较弱	NA	NV 优化前后对比更方便
引导式分析 / 规则	Rules 专家系统：自动检测瓶颈 + 按预计加速比排序建议；规则可用 Python 自定义	Roofline 自动 Bound 分类 + 提示；无通用可扩展规则引擎	NA	NV 自动化建议更系统、可扩展
扩展与自动化	Python report 接口、自定义 section / rule、CSV 导出、NVTX 标注	bin / json 数据文件 + Insight 导入；脚本化 / 开放度相对较低	NA	NV 二次开发 / CI 集成更成熟

3. 规格（含关键特性、精度与性能标准、资料与样例）

4. 兼容性设计（含跨芯片支持）

5. 易用性设计

6. 可调试性设计

7. 可维护性设计

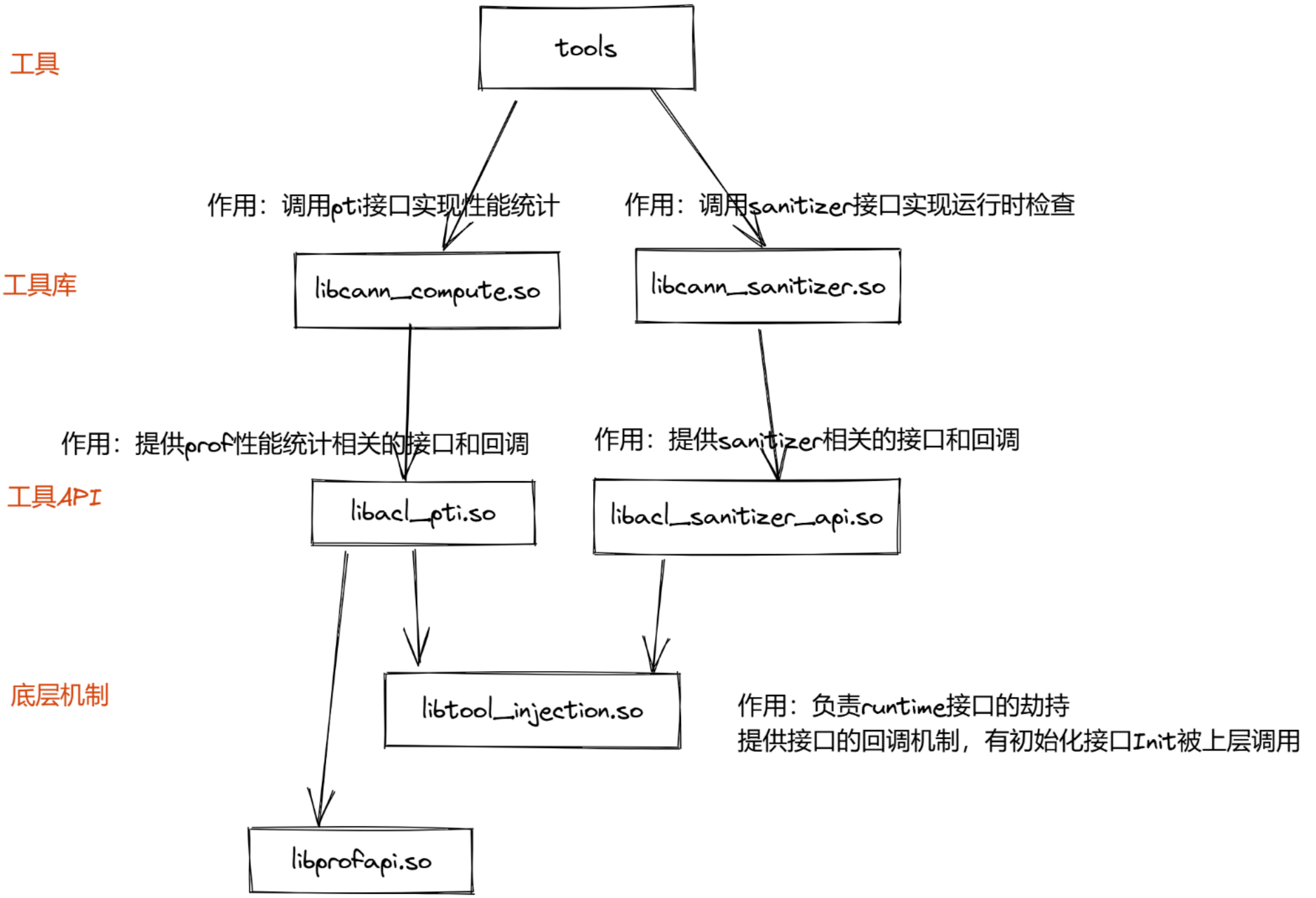
7.1. 采集数据完整性

drvier -> runtime -> prof 需要增加收到多少数据，读走多少数据

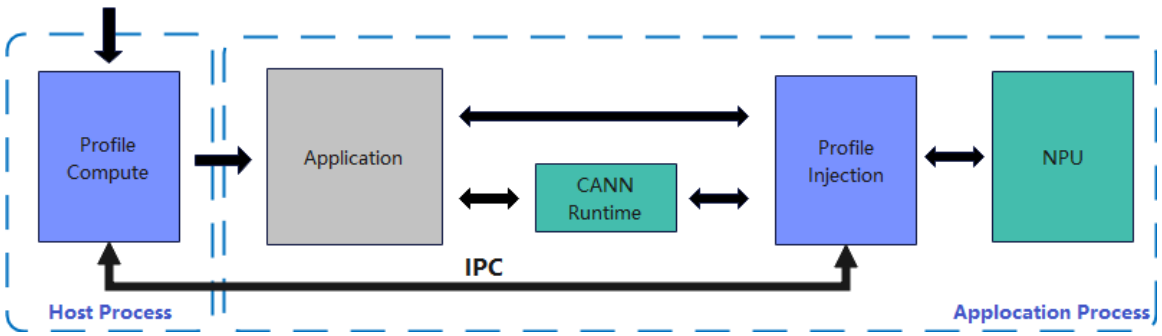
8. 可测试性设计

9. 其他DFX设计（可靠性、FunctionSafety、安全性）

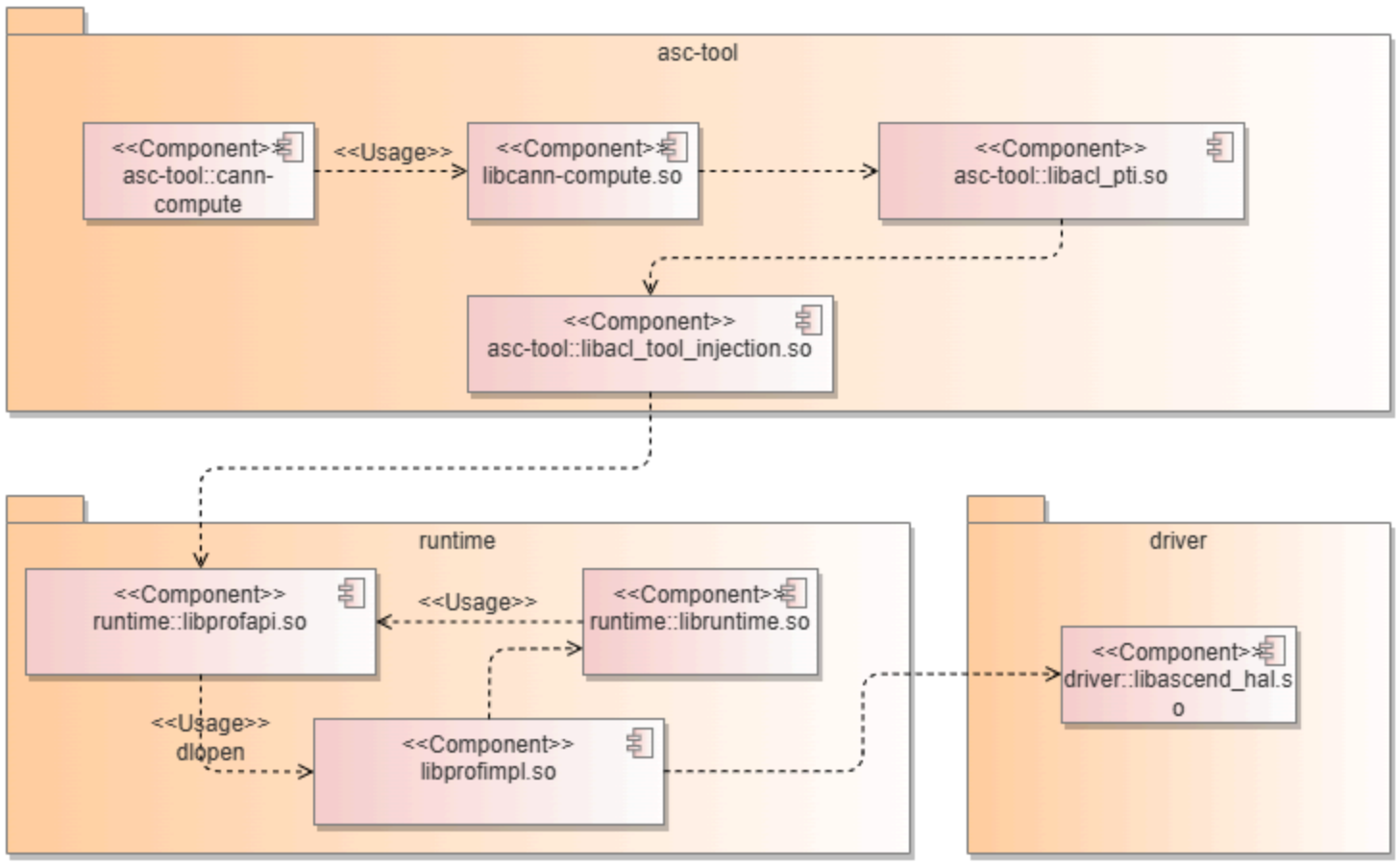
10. 架构图（含抽象分层）和API接口



10.1. 用例图



10.2. 部署视图



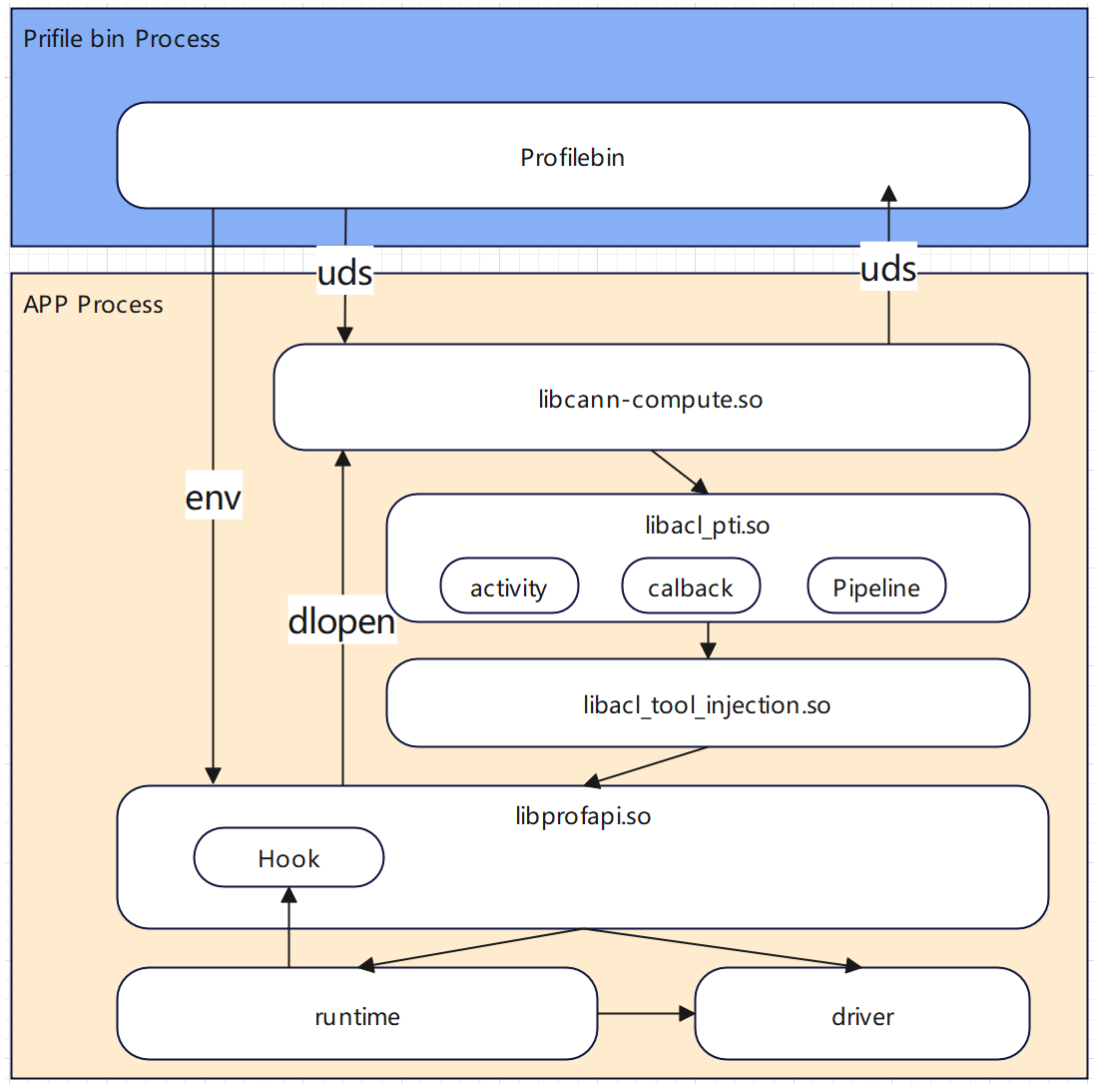
1：采集的device

clouddragon.huawei.com

10.3. 逻辑框架图

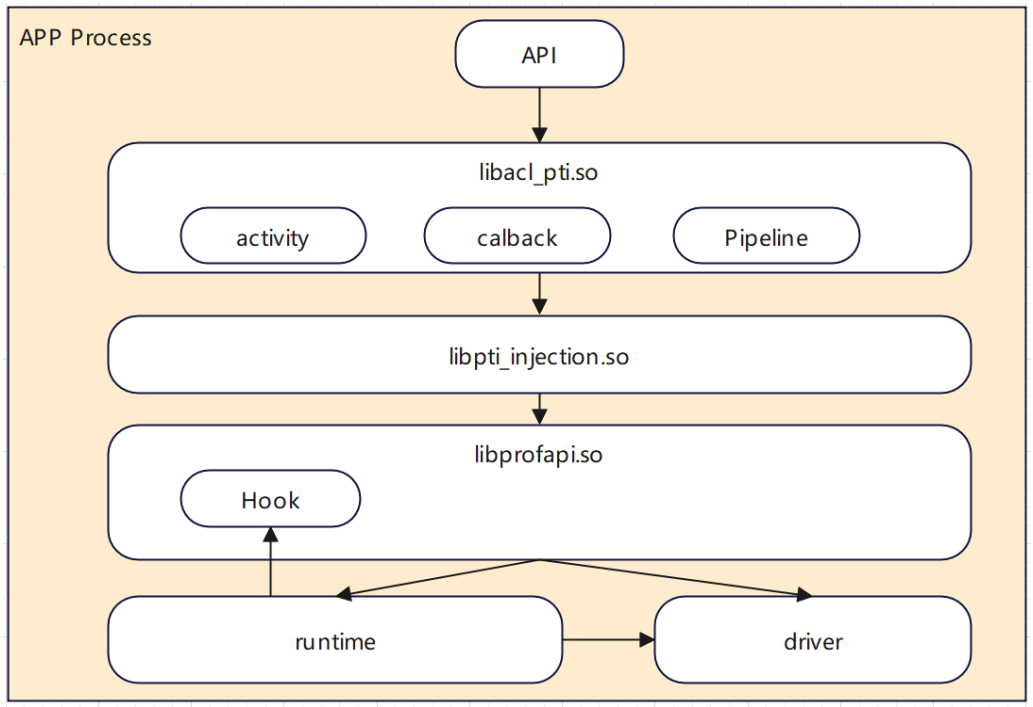
场景一：进程采集

tool option ./app



export ACL_API_INJECTION= “libcann-compute.so”

场景二：api采集



11. 关键功能实现设计

11.1. Profile基础特性

11.1.1. 功能概述

11.1.2. 实现思路

11.1.2.1. 二进制整体流程

cloudmodelingapi.tools.huawei.com

11.1.3. 实现设计

11.1.3.1. 命令行入口

```scss
1  单独工具方式：cann-compute [options] [program] [program-arguments]
```

命令行	取值	默认使能	备注说明
--help, -h	NA	no	Print this help message.

--set	basic	yes	Pipeline;PipeUtilization/L2Cache/Memory/MemoryL0/MemoryUB/ArithmeticUtilization
	detailed	no	暂不支持
	full	no	Pipeline;PipeUtilization/L2Cache/Memory/MemoryL0/MemoryUB/ArithmeticUtilization/ResourceConflictRatio/SourceCounters
	hccl	no	暂不支持
	roofline	no	暂不支持
--list-sets	NA	no	List all section sets found in the search paths.
--section	PipeUtilization	yes	
	Pipeline	yes	
	L2Cache	yes	
	Memory	yes	
	MemoryL0	yes	
	MemoryUB	yes	
	ArithmeticUtilization	no	
	ResourceConflictRatio	no	
	SourceCounters		展示每个指令的执行次数、谓词命中情况
	PmSampling		per mo
--cache-control	all/none		Control the behavior of the GPU caches during profiling. Allowed values: all
--list-sections	NA	no	List all sections found in the search paths.
--metric			暂不支持
--list-metrics			暂不支持

--query-metrics			暂不支持
replay-mode	kernel	yes	Replay individual kernel launches transparently during the execution of the application.
	application	no	Relaunch the entire application multiple times. Requires deterministic program execution.
	range	no	Replay ranges of kernel launches and API calls during the execution of the application.
	app-range	no	Profile ranges without API capture by relaunching the entire application multiple times. Requires deterministic program execution.
--import，-i	<input file>	no	Set the input file for reading profile results.
--export，-o	<output file>	no	Set the output file for writing the profile results. If not set, a temporary file will be used which is removed afterwards.
--force-overwrite，-f		no	Force overwriting all output, section or config files (any existing files will be overwritten).
--log-file	<log file>	no	暂不支持

11.1.3.2. **IPC传递机制**

对比竞品方式采用UDS（unix domain socket）
传输协议：

Magic(uint32_t)	Type(uint16_t)	resv0(uint16_t)	Length(uint32_t)	Value	说明
0x5A5B	0x0001				确认连接
	0x0002				Ctrl + C/ Flush数据
0x5A5B	0x0101				PMU数据

0x5A5B	0x0102				Pipeline数据

- 命令行信息发送 ->(export
- 报告结果的回传
- 算子性能采集报告打屏
- 算子性能采集进度条显示
- 动态采集需要uds的使能 -> ncu attch ncu

11.1.3.3. **Runtime Hook机制**

- LD_PRELOAD机制

shell

Full Screen

```shell
export LD_PRELOAD=xxx.so
```

机制	适用阶段	防护对象	副作用
dlsym(handle)	符号获取	仅 target_func 本身	无
RTLD_DEEPBIND	dlopen 时	库内所有内部调用	库内 libc 符号也本地优先
dlmopen(LM_ID_NEWLM)	dlopen 时	整库 + 独立 libc	内存/句柄开销
-static	链接时	全部	二进制大、不能用动态功能
-Bsymbolic-functions	链接时	库内函数调用	全库影响
visibility("hidden")	编译时	标记的内部接口	不可被外部 dlsym
setuid/setgid + AT_SECURE	运行时环境	整个 LD_PRELOAD	仅对提权执行生效

- Dispatch表方式
  - 公开的api 不直接走真实接口，走函数指针

cpp

Full Screen

```cpp

const aclrtDispatchTable* GetRtDispatchTable() {
  static auto* _v = &GetRtDispatchTableImpl<aclrtDispatchTable>();
  return _v;
```

```
}

aclError aclrtMalloc(void **devPtr, size_t size, aclrtMemMallocPolicy policy)
{
    return GetRtDispatchTable()->aclrtMalloc_fn(devPtr, size, policy)
}
```

➢ GetRtDispatchTableImpl如何实现

cpp Full Screen

```cpp
struct aclrtDispatchTable {
    aclrtMalloc_t aclrtMalloc_fn;
}

template <typename Tp> ToolsInit(Tp *tp)
{
    RegisterInjectionLibraryApiTable(mode, tp); // asce
}

template <typename Tp> Tp& GetRtDispatchTableImpl() {
    static auto dispatchTable = Tp{};
    // Change all the function pointers to point to the runtime implementation functions
    UpdateDispatchTable(&dispatchTable);

    // Profiler Registration, may wrap the function pointers
    ToolsInit(&dispatchTable);

    return dispatchTable;
}
```

➢ 注册时，把表项替换成自己的wrapper

cpp Full Screen

```cpp
void RegisterInjectionLibraryApiTable(const char*mode, Tp **tp)
{
    static const char *tools = get_env("ASCEND_INJECTION_MODE");
    if (tools != nullptr) {
        tp = hook_api(tp);
    }
}


```

➢ correlation id 生成

每个Hook的runtime接口入口都会生成一个 correlation id

```cpp
static std::atomic<uint64_t> g_correlation_id{0}

uint64_t generate_correlation_id() {
    return g_correlation_id.fetch_add(1, std::memory_order_relaxed);
}
```

- 提供接口Hook

```cpp
typedef int (*aclrtApiFunc)(void);

// Hook对应name的api，不存在api或已废弃删除的api会返回 无效Hook API错误
// 无效的API hook不成功，有效api可以hook成功，工具可以判断此错误码做处理，通常工具可以忽略此错误，因为对应的acl接口也不存在，业务不可能触发调用，自然
aclError aclrtApiInjectionSetFunc(const char *name, aclrtApiFunc func);

// 获取API对应的原生实现函数和当前hook的函数，不存在api或已废弃删除的api会返回失败
// Runtime初始化时originFunc == currentFunc
// 业务可以通过aclrtApiHookSetFunc修改currentFunc
aclError aclrtApiInjectionGetFunc(const char *name, aclrtApiFunc *originfunc, aclrtApiFunc *currentFunc)
```

Hook方案

runtime -> MsprofInjectionInitialize(aclrtApiInjectionSetFunc, aclrtApiInjectionGetFunc); (profapi.so)

需要Runtime HOOK的表 总共有138个
RuntimeHook的接口表
Hook性能对比结果

11.1.3.4. npu-compute

1：提供统一调用接口 aclptiInjectionInitialize

```java
void aclptiInjectionInitialize(void) // 接口不对外
{
    // 1: 初始化 Hook Init : acltoolProfilerHookInit()
    // 2: 启动thread 启动UDS server, server 接收client采集的具体内容
    // 3: 保存thread id
    // 4: 退出当前线程
}
```

```java
// 独立线程
void ComputeUdsServer(void *arg)
{
    // 1: 接收 uds client 命令
    // 2: 调用aclpti对应的采集接口
}
```

11.1.3.4.1. Metrics采集流程

```auto
1
```

11.1.3.4.2. Pipeline采集流程

```auto
1
```

11.1.3.5. acl_pti中的实现

11.1.3.5.1. Activity API

```scss
enum aclpriActivityApiFieldIds {}

aclptiResult aclptiActivityDisable(
    aclptiSubscriberHandle subscriber,
    aclptiActivityKind kind,
    aclptiActivityConfig *pActivityConfig,
)

aclptiResult aclptiActivityEnableAllSyncRecords(uint8_t enable)
aclptiResult aclptiActivityEnableAllocationSource(uint8_t enable)

aclptiResult aclptiActivityEnableAndDump(
    aclptiSubscriberHandle subscriber,
    aclptiActivityKind kind,
    aclptiActivityConfig *activityConfig,
)

aclptiResult acltiActivityEnableDeviceGraph(uint8_t enable)

aclptiResult aclptiActivityEnableHWTrace(uint8_t enable)

aclptiResult aclptiActivityEnableRuntimeApi(
```

```
    aclptiSubscriberHandle subscriber,
    aclptiCallbackId cbid,
    uint8_t enable,
)

aclptiResult aclptiActivityEnable(
    aclptiSubscriberHandle subscriber,
    aclptiActivityKind kind,
    aclptiActivityConfig *pActivityConfig,
)


aclptiResult aclptiActivityGetNextRecord(
    aclptiSubscriberHandle subscriber,
    uint8_t *buffer,
    size_t validBufferSizeBytes,
    aclptiActivity **record,
)

aclptiResult aclptiActivityGetNumDroppedRecords(
    aclptiSubscriberHandle subscriber,
    CUcontext context,
    uint32_t streamId,
    size_t *dropped,
)
```

11.1.3.5.2. **CallBack API**

java

Full Screen

```
aclptiResult aclptiSubscribe(
  aclptiSubscriberHandle *subscriber,
  aclptiCallbackFunc callback,
  void *userdata,
  aclptiSubscriberParams *pParams,
)
```

11.1.3.5.3. **RangeProfiling API**

scss

Full Screen

```
// aclpti_range_profiler.h
// 1:
aclptiResult aclptiRangeProfilerCounterDataGetRangeInfo(
    aclptiRangeProfilerCounterDataGetRangeInfoParams *pParams,
)

// 2:
```

```
aclptiResult aclptiRangeProfilerCounterDataImageInitialize(
    aclptiRangeProfilerCounterDataImageInitializeParams *pParams,
)

// 3:
aclptiResult aclptiRangeProfilerDecodeData(
    aclptiRangeProfilerDecodeDataParams *pParams,
)

// 4:
struct aclptiRangeProfilerDisableParams {
    size_t structSize;
    void *pPriv;
    aclptiRangeProfilerObject *pRangeProfilerObject
};
aclptiResult aclptiRangeProfilerDisable(
    aclptiRangeProfilerDisableParams *pParams,
)

// 5: Create a range profiler object and enable range profiling on the acl context
struct aclptiRangeProfilerEnableParams {
    size_t structSize;
    void *pPriv;
    aclContext ctx;
    aclptiRangeProfilerObject *pRangeProfilerObject
};
aclptiResult aclptiRangeProfilerEnable(
    aclptiRangeProfilerEnableParams *pParams,
)

// 6:
aclptiResult aclptiRangeProfilerGetCounterDataInfo(
    aclptiRangeProfiler_GetCounterDataInfo_Params *pParams,
)

// 7:
aclptiResult aclptiRangeProfilerGetCounterDataSize(
    aclptiRangeProfilerGetCounterDataSizeParams *pParams,
)

// 8:
aclptiResult aclptiRangeProfilerGetDevicePartitionInfo(
    aclptiRangeProfilerGetDevicePartitionInfoParams *pParams,
)

// 9:
aclptiResult aclptiRangeProfilerPopRange(
    aclptiRangeProfilerPopRangeParams *pParams,
)

// 10:
```

```
aclptiResult aclptiRangeProfilerPushRange(
    aclptiRangeProfilerPushRangeParams *pParams,
)

// 11:
aclptiResult aclptiRangeProfilerSetConfig(
    aclptiRangeProfilerSetConfigParams *pParams,
)

// 12:
aclptiResult aclptiRangeProfilerStart(
    aclptiRangeProfilerStartParams *pParams,
)

// 13:
aclptiResult aclptiRangeProfilerStop(
    aclptiRangeProfilerStopParams *pParams,
)
```

11.1.3.5.4. **Profiler Host API**

scss

Full Screen

```
aclptiResult aclptiProfilerHostConfigAddMetrics(
    aclptiProfilerHostConfigAddMetricsParams *pParams,
)

aclptiResult aclptiProfilerHostDeinitialize(
    aclptiProfilerHostDeinitializeParams *pParams,
)

aclptiResult aclptiProfilerHostEvaluateToGpuValues(
    aclptiProfilerHostEvaluateToGpuValuesParams *pParams,
)

aclptiResult aclptiProfilerHostGetBaseMetrics(
    aclptiProfilerHostGetBaseMetricsParams *pParams,
)

aclptiResult aclptiProfilerHostGetConfigImage(
    aclptiProfilerHostGetConfigImageParams *pParams,
)

aclptiResult aclptiProfilerHostGetConfigImageSize(
    aclptiProfilerHostGetConfigImageSizeParams *pParams,
)

aclptiResult aclptiProfilerHostGetMaxNumHardwareMetricsPerPass(
    aclptiProfilerHostGetMaxNumHardwareMetricsPerPassParams *pParams,
)
```

```
)

aclptiResult aclptiProfilerHostGetMetricProperties(
    aclptiProfilerHostGetMetricPropertiesParams *pParams,
)

aclptiResult aclptiProfilerHostGetMetricsInSinglePassSet(
    aclptiProfilerHostGetMetricsInSinglePassSetParams *pParams,
)

aclptiResult aclptiProfilerHostGetNumOfPasses(
    aclptiProfilerHostGetNumOfPassesParams *pParams,
)

aclptiResult aclptiProfilerHostGetRangeName(
    aclptiProfilerHostGetRangeNameParams *pParams,
)

aclptiResult aclptiProfilerHostGetSinglePassSets(
    aclptiProfilerHostGetSinglePassSetsParams *pParams,
)

aclptiResult aclptiProfilerHostGetSubMetrics(
    aclptiProfilerHostGetSubMetricsParams *pParams,
)

aclptiResult aclptiProfilerHostGetSupportedChips(
    aclptiProfilerHostGetSupportedChipsParams *pParams,
)

aclptiResult aclptiProfilerHostInitialize(
    aclptiProfilerHostInitializeParams *pParams,
)

aclptiResult aclptiProfilerHostSetDevicePartitionInfo(
    aclptiProfilerHostSetDevicePartitionInfoParams *pParams,
)
```

11.1.3.6. acl_tool_injection

11.1.3.6.1. 目的是什么

1: 保存Kernel二进制 算子.o -> Hash

java

Full Screen

```
aclrtBinaryLoadFromFileHook {
    // save kernel.o
}
aclrtBinaryLoadFromDataHook
aclrtBinaryUnLoadHook
```

2: 保存输入内存

```scss
aclrtMallocHook()  //  address +length  -> 再 malloc 一次 用于备份数据
aclrtMemcpyHook()  //
aclrtMemcpyAsyncHook
acltrFree
```

3: 保存KernelLaunch的参数

```auto
aclrtLaunchKernelWithHostArgsImpl
```

Hook接口

11.1.3.6.2. **hook实现**

```csharp
void acltoolProfilerHookInit()
{
    // 0: 以下呈现建议使用pthread_once调用 确保只调用一次
    // 1: MsprofGetInjectionFunc 获取Runtime Hook接口函数
    // 2: 使用Runtime Hook接口替换Profiler Hook表
}
```

1：调用MsprofGetInjectionFunc从libprofapi.so中获取已经注册的runtime 的hook接口

```cpp
//aclError aclrtApiInjectionSetFunc(const char *name, aclrtApiFunc func);
// aclError aclrtApiInjectionGetFunc(const char *name, aclrtApiFunc *originfunc, aclrtApiFunc *currentFunc)
void *MsprofGetInjectionFunc(uint32_t type); // profapi.so
```

2：劫持接口的后缀采用Hook命名

如：aclrtMalloc -> aclrtMallocHook

3：Hook Table的设置

```cpp
// Runtime API 枚举
typedef enum {
    ACL_RT_API_aclrtLaunchKernel     = 0,
    ACL_RT_API_aclrtMemcpy           = 1,
    ACL_RT_API_aclrtMalloc           = 4,
```

```
    // ... 按需扩展
    ACL_RT_API_MAX
} aclrtApiId;

std::map<string, aclrtApiId> aclrtApiNameMap = {
    { "aclrtLaunchKernel", ACL_RT_API_aclrtLaunchKernel },
    { "aclrtMemcpy", ACL_RT_API_aclrtMemcpy },
    { "aclrtMalloc", ACL_RT_API_aclrtMalloc },
};

typedef struct {
    void *entries[ACL_RT_API_MAX];
    void *orig[ACL_RT_API_MAX];
    pthread_rwlock_t lock;              // 读写锁，保护替换操作
} aclrtApiTable;
```

Runtime GetInjection接口中的将originfunc 的保存起来使用。

cpp Full Screen

```
aclError aclrtApiInjectionGetFunc(const char *name, aclrtApiFunc *originfunc, aclrtApiFunc *currentFunc)
```

scss Full Screen

```
aclrtMallocHook(...)
{
    ....hook pre process
    aclrtApiTable.orig[ACL_RT_API_aclrtMemcpy](...);
    ....hook post process
}
```

11.1.3.7. Profiling数据采集&解析

11.1.3.7.1. 基础信息采集 (需要考虑多device情况)

	参数	参数值	备注说明
software			
Host Version			
	Kernel	5.4.0-125-generic	
	OS	Ubuntu 20.04.6 LTS	

Device Version	Driver	7.0.t9.0.B886	/etc/ascend_install.info /usr/local/Ascend/driver/version.info
	CANN	9.1.0	env(ASCEND_HOME_PATH)/
hardware			
Host Info			
	Cpu Info	Intel(R) Xeon(R) CPU E5-2680 v4 @ 2.40GHz	
	Cpu Physical Count	2	
	Cpu Logical Count	46	
	Memory Total Size（MB）	461897260	
	Disk Total Size（GB）	2879978960	
Device Info			
	NPU Count	1	aclrtGetDeviceCount
	Chip Info	Ascend 950PR_9599 V100	aclrtGetSocName
	Arch Info	3510	
CPU Infomation			
	Control CPU Count	1 (1 * 1)	
	Control CPU Frequency (MHZ)	1500	
	AI CPU Count	6 (6 * 1)	
	AI CPU Frequency (MHZ)	1500	
AI Core Infomation			

	AI Core Count	36 (36 * 1)	
	AI Cube Count	36 (36 * 1)	
	AI Vector Count	72 (72 * 1)	
	AI Core Frequency (MHZ)	100, 100	
Memory Infomation			
	HBM Total (MB)	131072	
	HBM Used (MB)	5190.55	
	HBM Frequency (MHZ)	3200	
查询当前Device的浮点计算结果输出模式			aclrtGetDeviceSatMode
获取两个Device之间的网络拓扑关系			aclrtGetDevicesTopo
获取当前Device下所有Stream上任务的溢出状态，并将状态值拷贝到用户申请的Device内存中			aclrtGetOverflowStatus
查询两个Device之间的数据交互状态			aclrtDevicePeerAccessStatus
获取当前AI软件栈的运行模式			aclrtGetRunMode
获取batch模式			参数中获取
获取当前进程的Device资源限制			aclrtGetDeviceResLimit

11.1.3.7.2. 采集PMU流程及 PMU指标解析

- 采集伪码

perl

Full Screen

```
struct prof_start_para para
prof_drv_start(device id, channel id, ¶);

struct prof_poll_info channels[6] = {0};
while (g_start == 0) {
```

```
    ret = prof_channel_poll(channels, 6, 1);
    if (ret < 0 || ret > 6) {
        PROF_LOG("prof_channel_poll error, ret=%d", ret);
        return NULL;
    }
    if (ret == 0) {
        PROF_LOG("skip channel poll");
        continue;
    }
    for (uint32_t i = 0; i < ret; i++) {
        int ret1 = 0;
        do {
        PROF_LOG("==== device id: %u, channel id: %d", channels[i].device_id, channels[i].channel_id);
        ret1 = prof_channel_read(channels[i].device_id, channels[i].channel_id, buf, PROF_READ_BUF_SIZE);
        if (ret1 > 0) {
            PROF_LOG("BIU prof_channel %d read length = %d", channels[i].channelid, ret1);
        } else {
            PROF_LOG("BIU prof_channel %d read failed ret %d", channels[i].channel_id, ret1);
        }
        } while (ret1 > 0);
    }
}

ret = prof_stop(device id, channe id);
```

- 采集的参数配置

采集类型	采集配置结构体
Stars Task Time	cpp Full Screen

```cpp
using StarsSocLogConfigT = struct TagStarsSocLogConfig {
    uint32_t acsq_task;          // 1-enable,2-disable
    uint32_t accPmu;            // 1-enable,2-disable
    uint32_t cdqm_reg;           // 1-enable,2-disable
    uint32_t dvpp_vpc_block;     // 1-enable,2-disable
    uint32_t dvpp_jpegd_block;   // 1-enable,2-disable
    uint32_t dvpp_jpede_block;   // 1-enable,2-disable
    uint32_t ffts_context_task;  // 1-enable,2-disable
    uint32_t ffts_block;         // 1-enable,2-disable
    uint32_t sdma_dmu;           // 1-enable,2-disable
    uint32_t tag;                // 0-enable immediately, 1-enable delay
    uint32_t blockShinkFlag;     // 1-enable,2-disable
};
```

Stars Task PMU	cpp Full Screen

```cpp
using AccProfileConfigT = struct TagAccProfileConfig {
    uint32_t type;                          // bit0:task base | bit1:sample base | bit2:blk task | bit3
    uint32_t period;                        // sample base
    uint32_t coreMask;                      // sample base
    uint32_t eventNum;                      // public
    uint16_t event[10];                     // public
};

using StarsAccProfileConfigT = struct TagStarsAccProfileConfig {
    uint32_t cfgMode;            // 0-none, 1-aic, 2-aiv, 3-aic&aiv
    AccProfileConfigT aiEventCfg[3];
    uint32_t tag;                // 0-enable immediately, 1-enable delay
    uint32_t blockShinkFlag;     // 1-enable, 2-disable
    uint32_t aicScale;           // 0-all, 1-partial
};
```

- 采集的指标及解析公式

stars log数据

Task Start/End 记录到soc_log buf 单条acsq log输出为32Byte

bit / no	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0x6bd3（固定值）																sqe_type						CNT									func_type=6'b00_0000(task start) func_type=6'b00_0001(task end)	
1	taskID																rt_streamID															
2	system_counter_low																															
3	system_counter_high																															
4	acsqID(低7bit有效)											accID（仅end log有accID）					RES0															
5	RES1																															
6	RES2																															
7	RES3																															

AIC/AIV block log， 记录到soc_log buf 单条block log 为32byte

no \ bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0x6bd3（和软件约定具体值）																sqe_type						CNT						func_type=6'b10_0100 func_type=6'b10_0101						
1	taskID																rt_streamID															
2	system_counter_low																															
3	system_counter_high																															
4	cxtID																res							mst		cxt_type						
5	threadID																res									core_id					core_type	
6	blkID																sublkID															
7	RES3																															

aic/aiv pmu 数据

AIC AIV task profiling 单笔128Byte

AIC AIV TASK_PROFILING

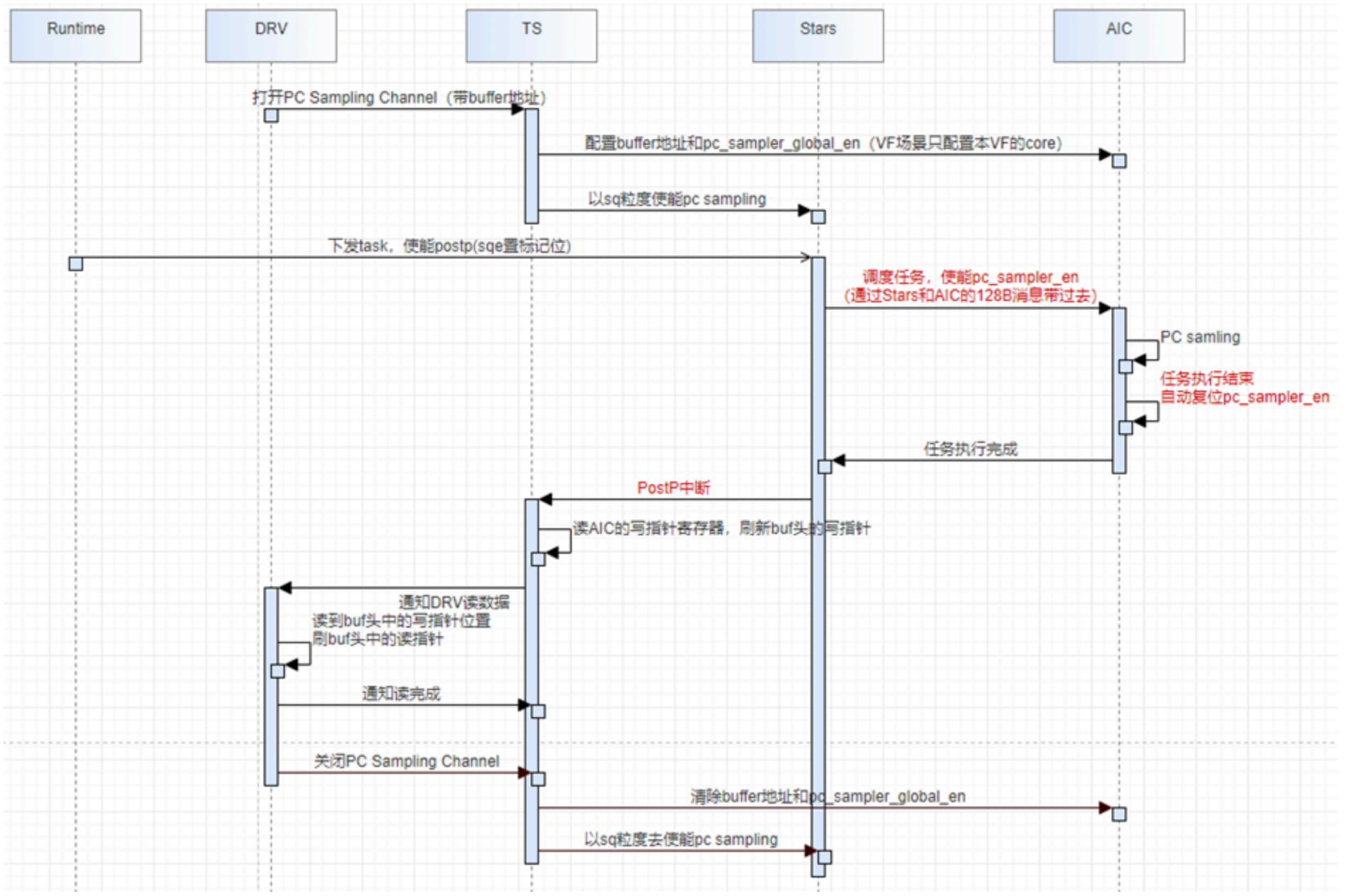
no \ bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0x6bd3（固定值）																res						CNT						func_type=6'b10_1010				
1	taskID																rt_streamID															
2	total_cyc_lo																															
3	total_cyc_hi																															
4	cxtID																res					OV	res	mst		cxt_type						
5	threadID																RES									core_id					core_type	
6	blkID																sublkID															
7	RES3																															
8	pmu_cnt0_lo																															
9	pmu_cnt0_hi																															
10	pmu_cnt1_lo																															
11	pmu_cnt1_hi																															
12	pmu_cnt2_lo																															
13	pmu_cnt2_hi																															
14	pmu_cnt3_lo																															
15	pmu_cnt3_hi																															
16	pmu_cnt4_lo																															
17	pmu_cnt4_hi																															
18	pmu_cnt5_lo																															
19	pmu_cnt5_hi																															
20	pmu_cnt6_lo																															
21	pmu_cnt6_hi																															
22	pmu_cnt7_lo																															
23	pmu_cnt7_hi																															
24	pmu_cnt8_lo																															
25	pmu_cnt8_hi																															
26	pmu_cnt9_lo																															
27	pmu_cnt9_hi																															
28	system_counter_low for task start																															
29	system_counter_high for task start																															
30	system_counter_low for task end																															
31	system_counter_high for task end																															

PMU值及公式

11.1.3.7.3. 采集BiuPerf&PC Sampling流程 及 解析

- 采集伪码：

https://gitcode.com/wk0911/asc/blob/main/prof.asc



cpp

Full Screen

```cpp
using InstrProfileConfigT = struct TagInstrProfileConfig {
    uint32_t period;              // cycle
    uint32_t biuPcSamplilngMode;  // 0: biu instr mode, 1: pc sampling
    uint32_t groupType            // 0: aic, 1: aiv0, 2: aiv1
    uint32_t groupNo              // group 编号
};
```

- 解析方式：
  - BiuPerf解析

Demo trace		
14	{4'h0, blkID[7:0]}	syscnt[15:0]
14	{4'h1, blkID[15:8]}	syscnt[31:16]
14	{4'h2, reserved 0}	syscnt[47:32]
14	{4'h3, reserved 0}	syscnt[63:48]
15	12'b0000_0000_0001	10
0	12'd1234	1
15	12'b0000_0001_0001	100
4	12'd1234	1(因為我們 一拍 trace 不了 兩件事)
15	12'b0000_0001_0001	65535(因為 timestamp delta saturated)
15	12'b0000_0000_0001	14465
15	12'b0000_0001_0001	20

31:28 (ctrl)	27:16 (12bits)	15:0 (16bits)
0000	12 bit region ID, dispatch time	Timestamp delta
0001	12bit region, execute time, for vector	Timestamp delta
0010	12bit region, execute time, for cube	Timestamp delta
0011	12bit region, execute time, for mte1	Timestamp delta
0100	12bit region, execute time, for mte2	Timestamp delta
0101	12bit region, execute time, for mte3	Timestamp delta
1010	12bit region, execute time, for fixp	Timestamp delta
0110-1001, 1011-1101	reserved	
1110	{4'h0, blkID[7:0]}	syscnt[15:0]
1110	{4'h1, blkID[15:8]}	syscnt[31:16]
1110	{4'h2, reserved 0}	syscnt[47:32]
1110	{4'h3, reserved 0}	syscnt[63:48]
1111	idle/busy state of different pipes	TimeStamp delta

bit	respective pipe idle/busy
0	SU
1	vec* (might be reserved)
2	cube*
3	mte1*
4	mte2
5	mte3
6	fixp
7-15	reserved

- PC Sampling解析

The state could be “active” and 8 other different stall reasons, namely:

1) IBuf empty
2) Stall cycles (nop cycles)
3) Scoreboard not ready
4) Register bank conflict
5) Resource conflict
6) Warp level sync
7) Divergence stack spill
8) Others (e.g. warp is still in L2, or any case that is not listed above)

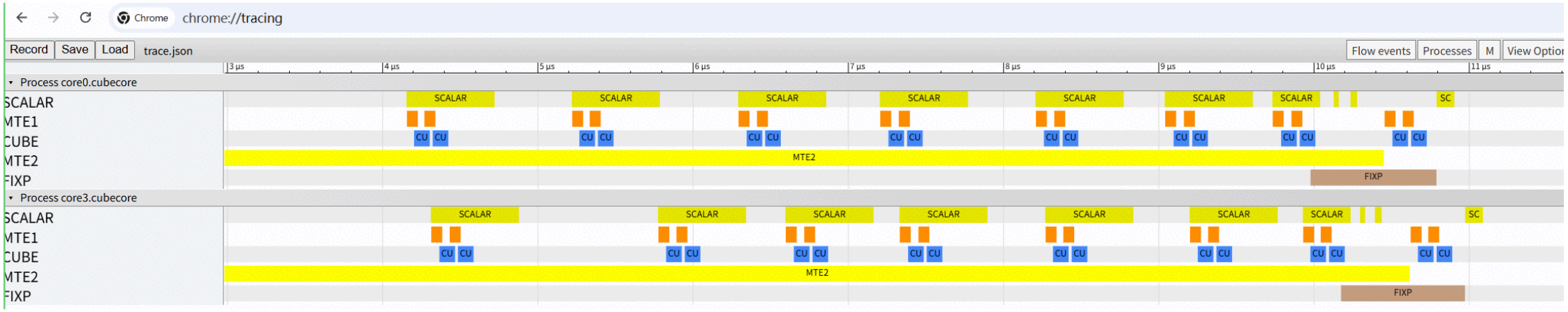
Entry	PC	state 0 cnt	state 1 cnt	state 2 cnt	state 3 cnt	state 4 cnt	state 5 cnt	state 6 cnt	state 7 cnt	state 8 cnt
0	0x12345	1	0	4	0	0	0	0	0	0
...	....									
N	0x12345	44	47	123	0	21	0	0	0	0
	0x12345	45	47	127	0	21	0	0	0	0

- 代码末尾插入：bisheng-tune

cpp

Full Screen

```cpp
extern __attribute__((noinline)) __attribute__((weak)) __aicore__
void __compute_before_kernel_end(__gm__ uint8_t *memInfo, uint64_t pc, uint32_t bid)
{
    asm volatile("bar.all");
    asm volatile(".rept 32\n\tNOP \n\t.endr");      // 确保MTE变为IDLE
    asm("DFX_REGION.S %0" :: "l" (0xd88));          // 8个DFX_REGION指令确保刷满32BYTE
    asm volatile("nop");
    asm("DFX_REGION.S %0" :: "l" (0xd99));
    asm volatile("nop");
    asm("DFX_REGION.S %0" :: "l" (0xdaa));
    asm volatile("nop");
    asm("DFX_REGION.S %0" :: "l" (0xdbb));
    asm volatile("nop");
    asm("DFX_REGION.S %0" :: "l" (0xdcc));
    asm volatile("nop");
    asm("DFX_REGION.S %0" :: "l" (0xddd));
    asm volatile("nop");
    asm("DFX_REGION.S %0" :: "l" (0xdee));
    asm volatile("nop");
    asm("DFX_REGION.S %0" :: "l" (0xdff));
    asm volatile(".rept 3500\n\tNOP \n\t.endr");     // 增加时延，以确保buffer刷入总线
    asm("DFX_REGION.S %0" :: "l" (0xdff));
    asm volatile("nop");
    return;
}
```



trace.json(12.01 KB)

11.1.3.7.4. **Profiling的prof_api的动态库中提供接口**

cpp

Full Screen

```cpp
Struct MsprofConfigAttr{
    MsprofConfigAttrId id;
    MsprofConfigAttValue value;
}

Enum MsprofConfigAttrId {
  MSPROF_AICOREMETRICS;
  ...
}
union MsprofConfigValue {
    uint32_t aicoreMetrics[10];  // 0xFFFFFFFF标记无效 PMU，后续不要采集这些指标。
    struct Pipeline(on, interval)
}

struct MsprofConfigInfo{
    MsprofConfigAttr*attrs;
    size_t numAttrs;
};

#define  MAX_DUMP_PATH_LEN 1024
#define  MAX_SAMPLE_CONFIG_LEN 4096
struct MsprofConfig {
    uint64_t profSwitch;                        // 参数开关
    uint64_t profSwitchHi;                      // 参数开关
    uint32_t devNums;
    uint32_t devIdList[MSPROF_MAX_DEV_NUM + 1]; // device id
    uint32_t modelId;
    uint32_t type;
    uint32_t cacheFlag;
    uint32_t storageLimit;
    uint32_t metrics;
```

```
    uintptr_t fd;
    char dumpPath[MAX_DUMP_PATH_LEN];
    char sampleConfig[MAX_SAMPLE_CONFIG_LEN];
    MsprofConfigInfo configInfo;
};

//1： Profiling 启动接口  type = PROF_CONFIG_COMPTUE (8)
int MsprofStart(uint32_t type, void *config, uint32_t len);

// 2：Profiling 结束接口 type = PROF_CONFIG_COMPTUE (8)
int MsprofStop(uint32_t type, void *config, uint32_t len);

// 3: Profiling Uploader接口
typedef struct {
    bool isLastChunk;
    size_t offset;
    int32_t chunkModule;
    int32_t deviceId;
    enum RawDataType type;
    size_t chunkSize;
    char chunk[256]; // 需要确认biuperf采集是数据大小是否满足？
} MsprofRawData;

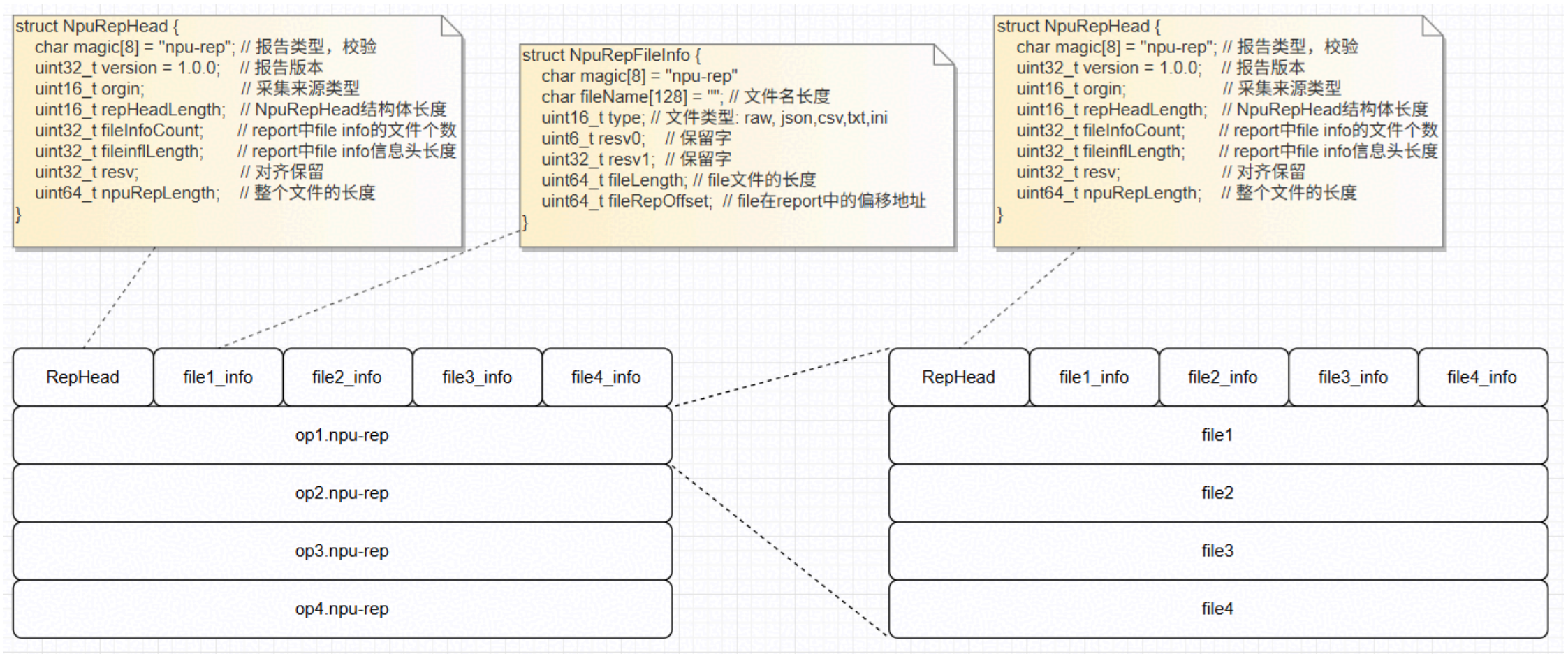
typedef int32_t(*MsprofRawDataCallback)(MsprofRawData* rawData);
int MsprofRegisterDataCallback(uint32_t type, void* func);
```

11.1.3.7.5. **仿真数据采集**

11.1.3.7.6. **解析数据关联**

11.1.3.8. **解析文件落盘格式 (金瑛琪)**

采集名称格式：report_<timestamp>_<rand-id>.npu-rep
vscode npu-sim --format=npu-rep -o xxx.npu-rep ./app



- NpuRepHead: report 的Head描述信息

字段	字节	字段说明	取值说明
magic	8	报告魔术字用于校验	“npu-rep”
version	4	报告版本	高16位为major，低16位中的高8位为minor，低8位为patch
origin	2	采集的来源	1：compute，2：simulator，3：sanitizer
repHeadLength	2	NpuRepHead结构体长度	sizeof（NpuRepHead）
fileInfoCount	4	report中file info的文件总个数	
fileInfoLength	4	report中file info信息头单个长度	sizeof(NpuRepFileInfo)
resv1	4	保留字，字节对齐	NA
npuRepLength	8	整个文件的总长度	

- NpuRepFileInfo：report中file info描述信息

字段	字节	字段说明	取值说明
magic	8	报告魔术字用于校验	"npu-rep"
fileName	128	文件名，需要包含'\0'	"kernel0"
fileType	2	文件类型	1: npu-rep，2:json，3:jsonl, 4: csv，5: sqlite3， 6: protobuf,
resv0	2	对齐保留字	0
resv1	4	对齐保留字	
fileLength	8	file文件的长度	
fileRepOffset	8	file在report中的偏移地址	

11.1.3.8.1. **硬件数据采集，HardwareInfo.jsonl**

json

Full Screen

```
{"category":"Host Info","cpu_physical_count":2,"cpu_logical_count":46,"memory_total_size_MB":461897260,"disk_total_size_GB":2879978960}
{"category":"Device Info","npu_count":1,"chip_info":"Ascend 950PR_9599 V100","arch_info":"3510"}
{"category":"CPU Information","control_cpu_count":1,"ai_cpu_count":6,"ai_cpu_frequency_MHZ":1500}
{"category":"AI Core Information","ai_core_count":36,"ai_cube_count":36,"ai_vector_count":72,"ai_core_frequency_MHZ":[100,100]}
{"category":"Memory Information","hbm_total_MB":131072,"hbm_used_MB":5190.55,"hbm_frequency_MHZ":3200}
```

11.1.3.8.2. **ArithmeticUtilization.csv**

block_id	sub_block_id	aic_time(us)	aic_total_cycles	aic_cube_ratio	aic_cube_fp16_ratio	aic_cube_int8_ratio	aic_cube_total_instr_number	aic_cube_fp_instr_number	aic_cube_int_instr_number	aiv_time(us)	aiv_total_cycles	aiv_vec_ratio	aiv_vec_vf_ratio	aiv_vec_sfu_ratio	aiv_vec_simt_vf_ratio
0	cube0	7.264849	11987	0.170852	0.170852	0	4	4	0	NA	NA	NA	NA	NA	NA
0	vector0	NA	NA	NA	NA	NA	NA	NA	NA	8.665455	14298	0.023849	0.021052	0.002798	0
0	vector1	NA	NA	NA	NA	NA	NA	NA	NA	8.652727	14277	0.023885	0.021083	0.002802	0

字段名	呈现名	**字段解释**
block_id	blcok id	Task运行切分数量，对应Task运行时配置的核数。
sub_block_id		Block上Vector\Cube核的ID。
aic_time(us)		该Task被分配到每个AI Cube Core计算单元上后，每个AI Cube Core计算单元上的执行时间，单位us。
aic_total_cycles		该Task被分配到每个AI Cube Core计算单元上后，每个AI Cube Core计算单元上的执行的cycle总数。

aic_cube_ratio		代表Cube单元指令的cycle数在total cycle数中的占用比。
aic_cube_fp_ratio		代表Cube fp类型指令的cycle数在total cycle数中的占用比。
aic_cube_int_ratio		代表Cube int类型指令的cycle数在total cycle数中的占用比。
aic_cube_total_instr_number		代表Cube指令的总条数，包括fp和int类型。
aic_cube_fp_instr_number		代表Cube fp类型指令的总条数。
aic_cube_int_instr_number		代表Cube int类型指令的总条数。
aiv_time(us)		该Task被分配到每个AI Vector Core计算单元上后，每个AI Vector Core计算单元上的执行时间，单位us。
aiv_total_cycles		该Task被分配到每个AI Vector Core计算单元上后，每个AI Vector Core计算单元上的执行的cycle总数。
aiv_vec_ratio		代表Vec单元指令的cycle数在total cycle数中的占用比。
aiv_vec_vf_ratio		代表Vec vf类型指令的cycle数在total cycle数中的占用比。
aiv_vec_sfu_ratio		代表Vec sfu类型指令的cycle数在total cycle数中的占用比。
aiv_vec_simt_vf_ratio		代表Vec simt vf类型指令的cycle数在total cycle数中的占用比。

11.1.3.8.3. **L2Cache.csv**

block_id	sub_block_id	aic_time(us)	aic_total_cycles	aic_write_cache_hit	aic_write_cache_miss_allocate	aic_r0_read_cache_hit	aic_r0_read_cache_miss_allocate	aic_r1_read_cache_hit	aic_r1_read_cache_miss_allocate	aic_write_hit_rate(%)	aic_read_hit_rate(%)	aic_total_hit_rate(%)	aiv_time(us)	aiv_total_cycles	aiv_write_cache_hit	aiv_write_cache_miss_allocate	aiv_r0_read_cache_hit	aiv_r0_read_cache_miss_allocate	aiv_r1_read_cache_hit	aiv_r1_read_cache_miss_allocate	aiv_write_hit_rate(%)	aiv_read_hit_rate(%)	aiv_total_hit_rate(%)
0	vector0	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	5.092727	8403	20	8	32	19	28	18	71.42857	61.85567	64
1	vector0	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	5.172727	8535	20	8	27	17	33	20	71.42857	61.85567	64
2	vector0	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	5.271515	8698	20	8	30	18	29	20	71.42857	60.82474	63.2
3	vector0	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	5.093333	8404	20	8	32	18	28	19	71.42857	61.85567	64
4	vector0	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	5.141212	8483	20	8	31	18	29	19	71.42857	61.85567	64
5	vector0	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	5.299394	8744	20	8	30	19	30	18	71.42857	61.85567	64
6	vector0	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	5.161212	8516	20	8	29	18	31	19	71.42857	61.85567	64
7	vector0	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	5.240606	8647	20	8	31	19	29	18	71.42857	61.85567	64

字段名	呈现名	**字段解释**
block_id		Task运行切分数量，对应Task运行时配置的核数。

sub_block_id		Task运行使用的每个block名称和序号。
aic_time(us)		该Task被分配到每个AI Cube Core计算单元上后，每个AI Cube Core计算单元上的执行时间，单位us。
aic_total_cycles		该Task被分配到每个AI Cube Core计算单元上后，每个AI Cube Core计算单元上的执行的cycle总数。
aiv_time(us)		该Task被分配到每个AI Vector Core计算单元上后，每个AI Vector Core计算单元上的执行时间，单位us。
aiv_total_cycles		该Task被分配到每个AI Vector Core计算单元上后，每个AI Vector Core计算单元上的执行的cycle总数。
ai*_read_close_hit		读close cache命中次数。
ai*_read_close_miss		读close cache缺失次数。
ai*_read_close_victim		读close cache驱逐次数。
ai*_read_far_hit		读far cache命中次数。
ai*_read_far_miss		读far cache缺失次数。
ai*_read_far_victim		读far cache驱逐次数。
ai*_read_hit_rate(%)		读cache命中率。
ai*_write_close_hit		写close cache命中次数。
ai*_write_close_miss		写close cache缺失次数。
ai*_write_close_victim		写close cache驱逐次数。
ai*_write_far_hit		写far cache命中次数。
ai*_write_far_miss		写far cache缺失次数。
ai*_write_far_victim		写far cache驱逐次数。
ai*_write_hit_rate(%)		写cache命中率。

11.1.3.8.4. **Memory.csv**

block_id	sub_block_id	aic_time(us)	aic_total_cycles	aic_l1_read_bw(GB/s)	aic_l1_write_bw(GB/s)	aic_main_mem_read_bw(GB/s)	aic_main_mem_write_bw(GB/s)	aic_mte1_instructions	aic_mte1_ratio	aic_mte2_instructions	aic_mte2_ratio	aic_mte3_instructions	aic_mte3_ratio	aiv_time(us)	aiv_total_cycles	aiv_ub_to_gm_bw(GB/s)	aiv_gm_to_ub_bw(GB/s)	aiv_main_mem_read_bw(GB/s)	aiv_main_mem_write_bw(GB/s)	aiv_mte2_instructions	aiv_mte2_ratio	aiv_mte3_instructions	aiv_mte3_ratio	read_main_memory_datas(KB)	write_main_memory_datas(KB)	GM_to_L1_datas(KB)	GM_to_L1_bw_usage_rate(%)	L0C_to_L1_datas(KB)	L0C_to_L1_bw_usage_rate(%)	L0C_to_GM_datas(KB)	L0C_to_GM_bw_usage_rate(%)	GM_to_UB_datas(KB)	GM_to_UB_bw_usage_rate(%)	UB_to_GM_datas(KB)	UB_to_GM_bw_usage_rate(%)
0	cube0	7.264849	11987	17.230213	17.230213	17.364824	11.486809	10	0.073913	6	0.440227	1	0.000167	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	193.5	128	193.5	0.33338	0	0	128	0.275675	NA	NA	NA	NA
0	vector0	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	8.665455	14298	11.845771	20.236526	20.236526	11.845771	3	0.033501	3	0.025739	225.5	132	NA	NA	NA	NA	NA	NA	225.5	0.387661	132	0.280947
0	vector1	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	8.652727	14277	11.845771	20.236526	20.236526	11.845771	3	0.031869	3	0.025862	225.5	132	NA	NA	NA	NA	NA	NA	225.5	0.387661	132	0.280947

字段名	字段解释
block_id	Task运行切分数量，对应Task运行时配置的核数。
sub_block_id	Task运行使用的每个block名称和序号。
aic_time(us)	该Task被分配到每个AI Cube Core计算单元上后，每个AI Cube Core计算单元上的执行时间，单位us。
aic_total_cycles	该Task被分配到每个AI Cube Core计算单元上后，每个AI Cube Core计算单元上的执行的cycle总数。
aiv_time(us)	该Task被分配到每个AI Vector Core计算单元上后，每个AI Vector Core计算单元上的执行时间，单位us。
aiv_total_cycles	该Task被分配到每个AI Vector Core计算单元上后，每个AI Vector Core计算单元上的执行的cycle总数。
aiv_ub_to_gm_bw(GB/s)	代表ub向gm写入的数据量对应total cycle的带宽速率，单位GB/s。
aiv_gm_to_ub_bw(GB/s)	代表gm向ub写入的数据量对应total cycle的带宽速率，单位GB/s。
aic_l1_read_bw(GB/s)	代表本算子中l1单元读取其他所有单元数据时，对应的total cycle的带宽速率，单位GB/s。
aic_l1_write_bw(GB/s)	代表本算子中l1单元写入其他所有单元数据时，对应的total cycle的带宽速率，单位GB/s。
ai*_main_mem_read_bw(GB/s)	代表主存储器读取其他所有单元数据时，对应的total cycle的带宽速率，单位GB/s。
ai*_main_mem_write_bw(GB/s)	代表主存储器写入其他所有单元数据时，对应的total cycle的带宽速率，单位GB/s。
aic_mte1_instructions	代表MTE1类型指令条数。
aic_mte1_ratio	代表MTE1类型指令的cycle数在total cycle数中的占用比。

ai*_mte2_instructions	代表MTE2类型指令条数。
ai*_mte2_ratio	代表MTE2类型指令的cycle数在total cycle数中的占用比。
ai*_mte3_instructions	代表MTE3类型指令条数。
ai*_mte3_ratio	代表MTE3类型指令的cycle数在total cycle数中的占用比。
read_main_memory_datas(KB)	读主存储器数据总量。
write_main_memory_datas(KB)	写主存储器数据总量。
GM_to_L1_datas(KB)	GM到L1的数据搬运量。
L0C_to_L1_datas(KB)	L0C到L1的数据搬运量。
L0C_to_GM_datas(KB)	L0C到GM的数据搬运量。
GM_to_UB_datas(KB)	GM到UB的数据搬运量。
UB_to_GM_datas(KB)	UB到GM的数据搬运量。
GM_to_L1_bw_usage_rate(%)	GM到L1通路带宽使用率。
L0C_to_L1_bw_usage_rate(%)	L0C到L1通路带宽使用率。
L0C_to_GM_bw_usage_rate(%)	L0C到GM通路带宽使用率。
GM_to_UB_bw_usage_rate(%)	GM到UB通路带宽使用率。
UB_to_GM_bw_usage_rate(%)	UB到GM通路带宽使用率。

11.1.3.8.5. **MemoryL0.csv**

block_id	sub_block_id	aic_time(us)	aic_total_cycles	aic_l0a_read_bw(GB/s)	aic_l0a_write_bw(GB/s)	aic_l0b_read_bw(GB/s)	aic_l0b_write_bw(GB/s)	aic_l0c_read_bw_cube(GB/s)	aic_l0c_write_bw_cube(GB/s)	aiv_time(us)	aiv_total_cycles
0	vector0	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	5.092727	8403
1	vector0	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	5.172727	8535
2	vector0	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	5.271515	8698
3	vector0	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	5.093333	8404
4	vector0	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	5.141212	8483
5	vector0	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	5.299394	8744
6	vector0	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	5.161212	8516
7	vector0	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	5.240606	8647

字段名	字段解释
block_id	Task运行切分数量，对应Task运行时配置的核数。
sub_block_id	Task运行使用的每个block名称和序号。
aic_time(us)	该Task被分配到每个AI Cube Core计算单元上后，每个AI Cube Core计算单元上的执行时间，单位us。
aic_total_cycles	该Task被分配到每个AI Cube Core计算单元上后，每个AI Cube Core计算单元上的执行的cycle总数。
aiv_time(us)	该Task被分配到每个AI Vector Core计算单元上后，每个AI Vector Core计算单元上的执行时间，单位us。
aiv_total_cycles	该Task被分配到每个AI Vector Core计算单元上后，每个AI Vector Core计算单元上的执行的cycle总数。
aic_l0a_read_bw(GB/s)	代表本算子中l0a读取其他所有单元数据时，对应的total cycle的带宽速率，单位GB/s。
aic_l0a_write_bw(GB/s)	代表本算子中l0a写入其他所有单元数据时，对应的total cycle的带宽速率，单位GB/s。
aic_l0b_read_bw(GB/s)	代表本算子中l0b读取其他所有单元数据时，对应的total cycle的带宽速率，单位GB/s。
aic_l0b_write_bw(GB/s)	代表本算子中l0b写入其他所有单元数据时，对应的total cycle的带宽速率，单位GB/s。
aic_l0c_read_bw_cube(GB/s)	代表Cube从l0c读取的数据量对应total cycle的带宽速率，单位GB/s。

aic_l0c_write_bw_cube(GB/s)	代表Cube向l0c写入的数据量对应total cycle的带宽速率，单位GB/s。

11.1.3.8.6. **MemoryUB.csv**

block_id	sub_block_id	aic_time(us)	aic_total_cycles	aiv_time(us)	aiv_total_cycles	aiv_ub_read_bw_vector(GB/s)	aiv_ub_write_bw_vector(GB/s)	aiv_ub_read_bw_gm(GB/s)	aiv_ub_write_bw_gm(GB/s)
0	cube0	7.264849	11987	NA	NA	NA	NA	NA	NA
0	vector0	NA	NA	8.665455	14298	1.592897	1.435851	11.84577	20.23653
0	vector1	NA	NA	8.652727	14277	1.592897	1.435851	11.84577	20.23653

字段名	**字段解释**
block_id	Task运行切分数量，对应Task运行时配置的核数。
sub_block_id	Task运行使用的每个block名称和序号。
aic_time(us)	该Task被分配到每个AI Cube Core计算单元上后，每个AI Cube Core计算单元上的执行时间，单位us。
aic_total_cycles	该Task被分配到每个AI Cube Core计算单元上后，每个AI Cube Core计算单元上的执行的cycle总数。
aiv_time(us)	该Task被分配到每个AI Vector Core计算单元上后，每个AI Vector Core计算单元上的执行时间，单位us。
aiv_total_cycles	该Task被分配到每个AI Vector Core计算单元上后，每个AI Vector Core计算单元上的执行的cycle总数。
aiv_ub_read_bw_vector(GB/s)	代表Vector从UB读取的数据量对应total cycle的带宽速率，单位GB/s。
aiv_ub_write_bw_vector(GB/s)	代表Vector向UB写入的数据量对应total cycle的带宽速率，单位GB/s。
aiv_ub_write_bw_gm(GB/s)	代表GM向UB写入的数据量对应total cycle的带宽速率，单位GB/s。
aiv_ub_read_bw_gm(GB/s)	代表GM向UB读取的数据量对应total cycle的带宽速率，单位GB/s。

11.1.3.8.7. **OpBasicInfo.csv**

Op Name	Op Type	Task Duration(us)	Block Dim	Mix Block Dim	Device Id	Pid	Current Freq	Rated Freq
matmul_leakyrelu_custom_0_mix_aic	mix	157.600006	2	4	0	521012	800	1650

字段名	字段解释
Op Name	算子名称。
Op Type	算子类型。
Task Duration(us)	Task耗时，包含调度到AI处理器的时间、AI处理器上的执行时间以及结束响应时间，单位us。
Block Dim	Task运行切分数量，对应Task运行时核数，开发者设置的算子执行逻辑核数。
Mix Block Dim	部分算子在Cube Core和Vector Core上同时执行，主AI处理器的blockDim在“Block Dim”字段中描述，可理解为Cube Core的数量，而从AI处理器的blockDim在本字段中描述，则可理解为Vector Core的数量。显示为N/A表示为非Mix融合算子。此参数仅适用于Atlas A3 训练系列产品/Atlas A3 推理系列产品和Atlas A2 训练系列产品/Atlas A2 推理系列产品以及Ascend 950 系列产品。
Device ID	运行时使用AI处理器的ID。
PID	算子运行时的进程号。
Current Freq	AI处理器当前运行的频率。
Rated Freq	AI处理器的理论频率。

11.1.3.8.8. **PipeUtilization.csv**

aic_total_cycles	aic_cube_time(us)	aic_cube_ratio	aic_scalar_time(us)	aic_scalar_ratio	aic_mte1_time(us)	aic_mte1_ratio	aic_mte2_time(us)	aic_mte2_ratio	aic_mte3_time(us)	aic_mte3_ratio	aic_mte3_active_bw(GB/s)	aic_fixpipe_time(us)	aic_fixpipe_ratio	aic_fixpipe_active_bw(GB/s)	aic_icache_miss_rate	aiv_time(us)	aiv_total_cycles	aiv_vec_time(us)	aiv_vec_ratio	aiv_scalar_time(us)	aiv_scalar_ratio	aiv_mte2_time(us)	aiv_mte2_ratio	aiv_mte2_active_bw(GB/s)	aiv_mte3_time(us)	aiv_mte3_ratio	aiv_mte3_active_bw(GB/s)	aiv_icache_miss_rate	aic_mte1_active_bw(GB/s)	aic_mte2_active_bw(GB/s)
272662	27.342	0.0797	161.6	0.4711	15.78	0.046	61.505	0.1793	2.0467	0.006	3614	84.673	0.2469	28.833	0.0049	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	232.07	59.696
NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	85.998	141896	3.6012	0.0105	7.557	0.022	7.043	0.0205	173.32	8.8133	0.0257	138.79	0.033	NA	NA
NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	164.31	271118	3.5661	0.0104	7.9879	0.0233	6.9479	0.0203	175.69	8.9085	0.026	137.31	0.0338	NA	NA

字段名	字段解释
block_id	Task运行切分数量，对应Task运行时配置的核数。
sub_block_id	Task运行使用的每个block名称和序号。

aic_time(us)	该Task被分配到每个AI Cube Core计算单元上后，每个AI Cube Core计算单元上的执行时间，单位us。
aic_total_cycles	该Task被分配到每个AI Cube Core计算单元上后，每个AI Cube Core计算单元上的执行的cycle总数。
aiv_time(us)	该Task被分配到每个AI Vector Core计算单元上后，每个AI Vector Core计算单元上的执行时间，单位us。
aiv_total_cycles	该Task被分配到每个AI Vector Core计算单元上后，每个AI Vector Core计算单元上的执行的cycle总数。
aiv_vec_time(us)	代表vec类型指令（向量类运算指令）耗时。
aiv_vec_ratio	代表vec类型指令（向量类运算指令）的cycle数在total cycle数中的占用比。
aic_cube_time(us)	代表Cube类型指令（fp16及s16矩阵类运算指令）耗时。
aic_cube_ratio	代表Cube类型指令（fp16及s16矩阵类运算指令）的cycle数在total cycle数中的占用比。
ai*_scalar_time(us)	代表scalar类型指令（标量类运算指令）耗时。
ai*_scalar_ratio	代表scalar类型指令（标量类运算指令）的cycle数在total cycle数中的占用比。
aic_fixpipe_time(us)	代表fixpipe类型指令（L0C->GM/L1搬运类指令）耗时。
aic_fixpipe_ratio	代表fixpipe类型指令（L0C->GM/L1搬运类指令）的cycle数在total cycle数中的占用比。
aic_mte1_time(us)	代表MTE1类型指令（L1->L0A/L0B搬运类指令）耗时，不包括搬运等待时间。
aic_mte1_ratio	代表MTE1类型指令（L1->L0A/L0B搬运类指令）的cycle数在total cycle数中的占用比。
ai*_mte2_time(us)	代表MTE2类型指令（GM->AICORE搬运类指令）耗时。
ai*_mte2_ratio	代表MTE2类型指令（GM->AICORE搬运类指令）的cycle数在total cycle数中的占用比。
ai*_mte3_time(us)	代表MTE3类型指令（AICORE->GM搬运类指令）耗时。

ai*_mte3_ratio	代表MTE3类型指令（AICORE->GM搬运类指令）的cycle数在total cycle数中的占用比。
ai*_icache_miss_rate	代表ICache缺失率，即未命中instruction的L1 cache，数值越小越好。
aic_mte3_active_bw(GB/s)	代表MTE3类型指令（AICORE->DDR CUBE搬运类指令）的活跃带宽。
aiv_mte3_active_bw(GB/s)	代表MTE3类型指令（AICORE->DDR AIV搬运类指令）的活跃带宽。
aic_fixpipe_active_bw(GB/s)	代表fixpipe类型指令（L0C->OUT/L1搬运类指令）的活跃带宽。
aiv_mte2_active_bw(GB/s)	代表MTE2类型指令（DDR->AICORE AIV搬运类指令）的活跃带宽。
aic_mte1_active_bw(GB/s)	代表Cube单元MTE1的活跃带宽，具体涉及L1->L0A、L1->L0B这2个部分的通路数据。
aic_mte2_active_bw(GB/s)	代表Cube单元MTE2的活跃带宽，具体涉及GM->L1、GM->L0A、GM->L0B这3条通路的数据。

11.1.3.8.9. **ResourceConflictRatio.csv**

block_id	sub_block_id	aic_time(us)	aic_total_cycles	aic_cube_wait_ratio	aic_mte1_wait_ratio	aic_mte2_wait_ratio	aic_mte3_wait_ratio	aiv_time(us)	aiv_total_cycles	aiv_vec_stu_cflt_ratio	aiv_vec_ldu_cflt_ratio	aiv_vec_sfu_cflt_ratio	aiv_vec_wait_ratio	aiv_mte2_wait_ratio	aiv_mte3_wait_ratio
0	cube0	7.264849	11987	0.56044	0.55068	0	0	NA	NA	NA	NA	NA	NA	NA	NA
0	vector0	NA	NA	NA	NA	NA	NA	8.665455	14298	0	0.002588	0	0.842845	0.809904	0.791439
0	vector1	NA	NA	NA	NA	NA	NA	8.652727	14277	0	0.002592	0	0.853401	0.820901	0.802409

字段名	**字段解释**
block_id	Task运行切分数量，对应Task运行时配置的核数。
sub_block_id	Task运行使用的每个block名称和序号。
aic_time(us)	该Task被分配到每个AI Cube Core计算单元上后，每个AI Cube Core计算单元上的执行时间，单位us。
aic_total_cycles	该Task被分配到每个AI Cube Core计算单元上后，每个AI Cube Core计算单元上的执行的cycle总数。
aic_cube_wait_ratio	代表Cube单元被阻塞的cycle数在所有指令执行cycle数中占比。

aic_mte1_wait_ratio	代表MTE1被阻塞的cycle数在所有指令执行cycle数中占比。
aic_mte2_wait_ratio	代表MTE2被阻塞的cycle数在所有指令执行cycle数中占比。
aic_mte3_wait_ratio	代表MTE3被阻塞的cycle数在所有指令执行cycle数中占比。
aiv_time(us)	该Task被分配到每个AI Vector Core计算单元上后，每个AI Vector Core计算单元上的执行时间，单位us。
aiv_total_cycles	该Task被分配到每个AI Vector Core计算单元上后，每个AI Vector Core计算单元上的执行的cycle总数。
aiv_vec_stu_cflt_ratio	代表Vector执行的指令set值时阻塞的cycle数在所有指令执行cycle数中占比。
aiv_vec_ldu_cflt_ratio	代表Vector执行的指令load值时阻塞的cycle数在所有指令执行cycle数中占比。
aiv_vec_sfu_cflt_ratio	代表Vector执行的指令读、写冲突阻塞的cycle数在所有指令执行cycle数中占比。
aiv_vec_wait_ratio	代表Vector单元被阻塞的cycle数在所有指令执行cycle数中占比。
aiv_mte2_wait_ratio	代表MTE2被阻塞的cycle数在所有指令执行cycle数中占比。
aiv_mte3_wait_ratio	代表MTE3被阻塞的cycle数在所有指令执行cycle数中占比。

11.1.3.8.10. **Timeline**

Trace(json) 支持 -> Chrome Trace
Pipeline :
CoreId + Cube()
CoreId + Vector()
物理核与逻辑核的映射。

11.1.3.8.11. **Report.json**

1：计算公式， 依据：task pmu

2：task pmu 放进来， csv的字段提供数据

SOL/平均带宽。

- 报告展示

1：整体耗时，算力情况，带宽，核利用率
2：核数，aic 频率、带宽大小、NPU ARCH等信息

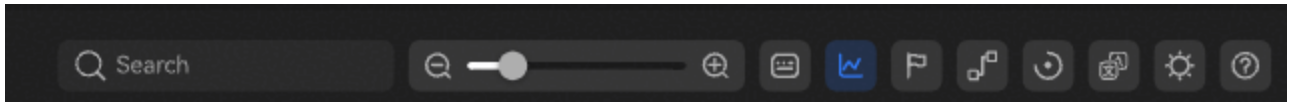
11.1.3.8.12.

11.1.3.8.13. **Timeline流水图**

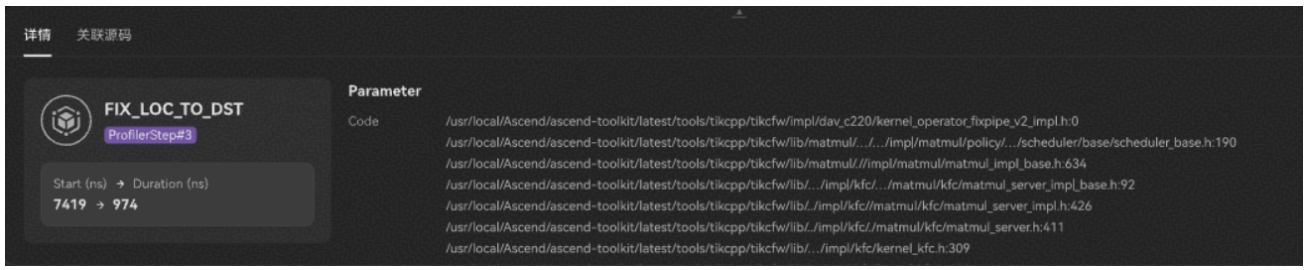


- P

1：试图中 按照 100% 还是实际Cycle查看？ 优选百分比 (时钟周期、吞吐量、百分比)
2：timeline中右键菜单内容？ （是否包含所有的界面邮件）
3：鼠标在流水线上停留了，能够显示 name和duration信息
4：图标位置：



5：详情页内容会调整，由 @汪凯 提供内容信息（基础信息采集），原峰给出UI设计
6：源码页面保留，数据源头由 @汪凯 提供
7：点击指令，查看堆栈，并支持跳转到对应的代码行源码信息（源码详情页，非编辑器，不可编辑）



8：当前只做kernel侧的源码与反汇编指令对应关系
9：仿真页面UI补充 @褚原峰
10：主瓶颈数据来源（同时包含算力达成，带宽达成，核负载均衡数据来源）： @汪凯

https://octo.hdesign.huawei.com/developerPreview/developer/index.html#edit&uniqueId=Cbt3Yr1Wzd6zfmVfNkJOYQ-50712&pageId=938628

11.1.3.9. **cache control**

- --cache-control=all （默认） 每轮replay前清空缓存，稳定指标

cpp

Full Screen

```cpp
// A5芯片可以直接使用，rsv 传nullptr
aclError aclrtDeviceL2CacheFlush(void *rsv);

// 其他以往芯片
aclError aclrtMemInvalidate(void* devPtr, size_t size);
aclError aclrtMemFlush(void* devPtr, size_t size);
```

- --cache-control=none 不刷缓存，模拟真实热缓存场景

cpp

Full Screen

```cpp
aclError aclrtMallocCached(void** devPtr, size_t size, aclrtMemMallocPolicy policy);
```

11.1.4. **接口设计**

11.1.4.1

https://wiki.huawei.com/domains/11731/wiki/95008/WIKI2026063011658345

11.1.5 分配需求

11.2. 可视化界面数据关联

11.2.1. 命令参数使用和说明

11.2.1.1. 编译命令行说明

11.2.1.2. 命令行参数使用

- 默认命令行模式：

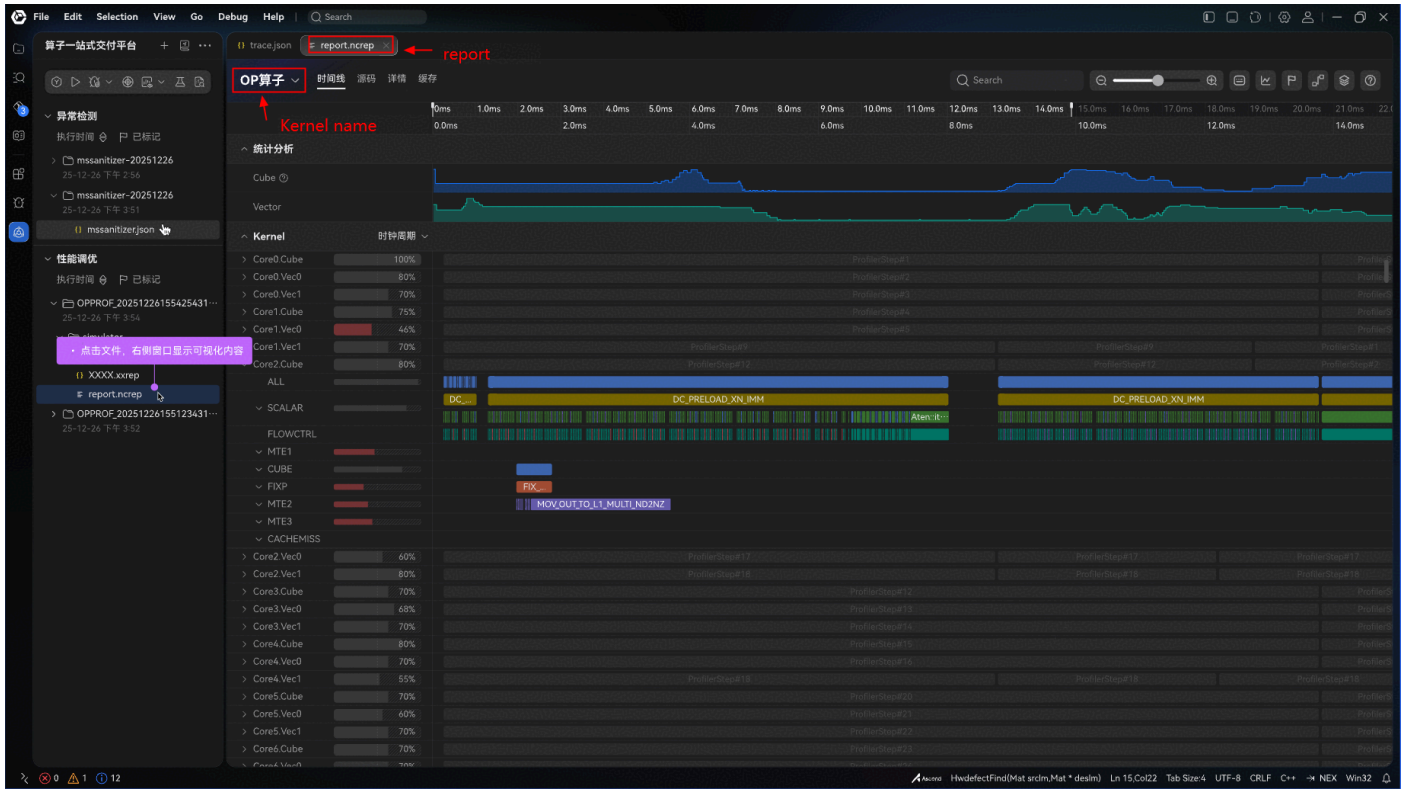
```lua
npu-compute ./app
```

- 按照配置页提供命令参数

11.2.2. 可视化显示原始稿

可视化原稿设计：https://octo-g.hdesign.huawei.com/developerPreview/developer/index.html#edit&uniqueId=Cbt3Yr1Wzd6zfmVfNkJOYQ-50712&pageId=1001695

11.2.3. 入口信息



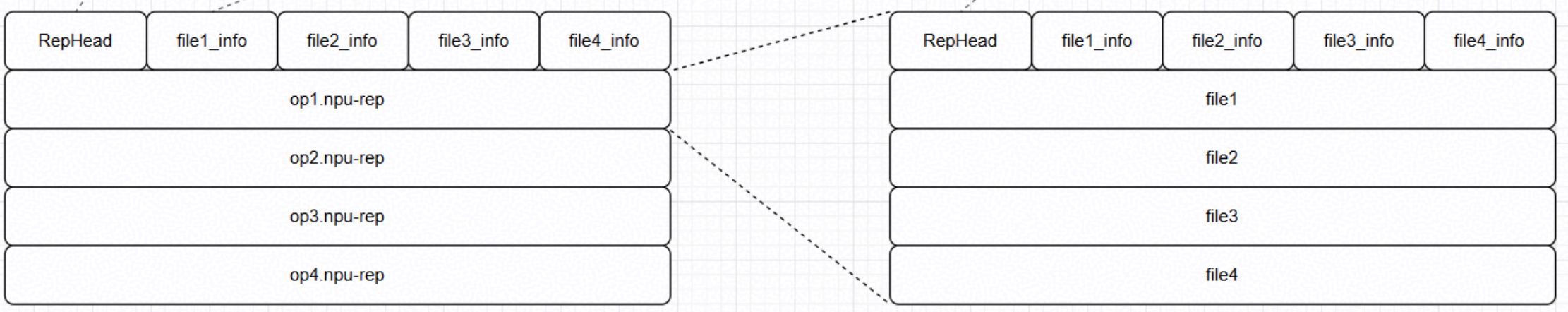
1：report：report_<timestamp>_<rand id>.npu-rep

2：OP算子：主要npu-rep中包含的npu-rep文件个数的选择

```
struct NpuRepHead {
    char magic[8] = "npu-rep"; // 报告类型，校验
    uint32_t version = 1.0.0;    // 报告版本
    uint16_t orgin;              // 采集来源类型
    uint16_t repHeadLength;   // NpuRepHead结构体长度
    uint32_t fileInfoCount;      // report中file info的文件个数
    uint32_t fileinflLength;     // report中file info信息头长度
    uint32_t resv;               // 对齐保留
    uint64_t npuRepLength;    // 整个文件的长度
}
```

```
struct NpuRepFileInfo {
    char magic[8] = "npu-rep"
    char fileName[128] = ""; // 文件名长度
    uint16_t type; // 文件类型: raw, json,csv,txt,ini
    uint6_t resv0;    // 保留字
    uint32_t resv1;  // 保留字
    uint64_t fileLength; // file文件的长度
    uint64_t fileRepOffset;  // file在report中的偏移地址
}
```

```
struct NpuRepHead {
    char magic[8] = "npu-rep"; // 报告类型，校验
    uint32_t version = 1.0.0;    // 报告版本
    uint16_t orgin;              // 采集来源类型
    uint16_t repHeadLength;   // NpuRepHead结构体长度
    uint32_t fileInfoCount;      // report中file info的文件个数
    uint32_t fileinflLength;     // report中file info信息头长度
    uint32_t resv;               // 对齐保留
    uint64_t npuRepLength;    // 整个文件的长度
}
```



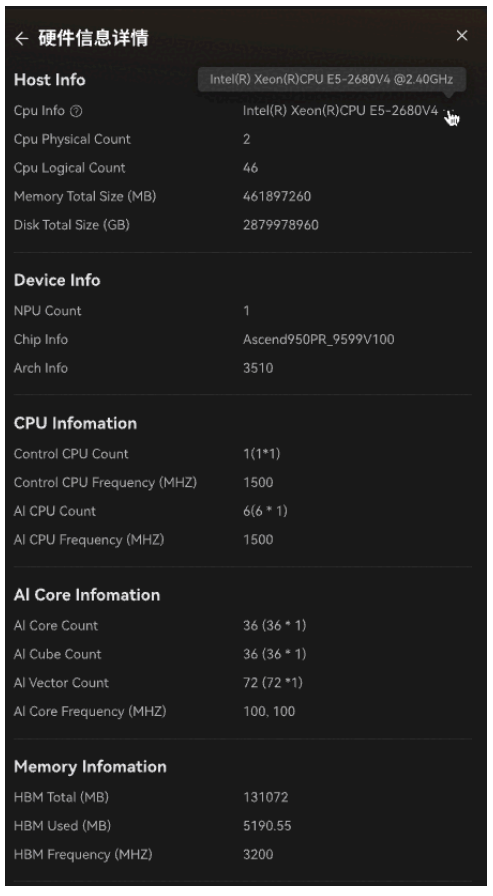
11.2.4. 报告统计信息



序号	显示	示意	字段	数据来源
1	进程ID	进程：1234	PID	OpBasicInfo.csv
2	算子类型	算子类型：mix	OpType	OpBasicInfo.csv
3	Blocks	Blocks：10	Block Dim	OpBasicInfo.csv

4	整体耗时	整体耗时 4.6 us	Task Duration（us）	OpBasicInfo.csv
5	算力情况			
6	输入带宽		aic_main_mem_read_bw	Memory.csv
			aiv_main_mem_read_bw	Memory.csv
7	输出带宽		aic_main_mem_write_bw	Memory.csv
			aiv_main_mem_write_bw	Memory.csv
8	Aicore并行使用率			
9	负载均衡度			

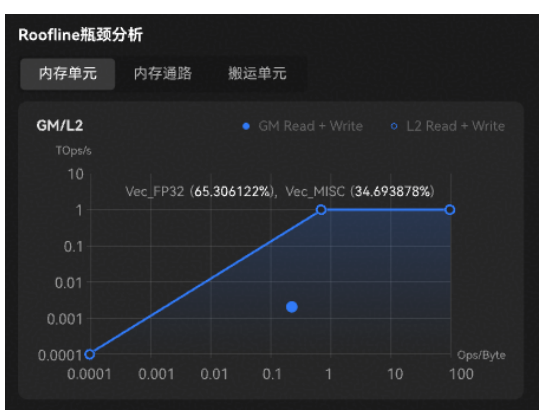
11.2.4.1. **硬件详情**



HardwareInfo.jsonl


```json
{"category":"Host Info","cpu_physical_count":2,"cpu_logical_count":46,"memory_total_size_MB":461897260,"disk_total_size_GB":2879978960}
{"category":"Device Info","npu_count":1,"chip_info":"Ascend 950PR_9599 V100","arch_info":"3510"}
{"category":"CPU Information","control_cpu_count":1,"ai_cpu_count":6,"ai_cpu_frequency_MHZ":1500}
{"category":"AI Core Information","ai_core_count":36,"ai_cube_count":36,"ai_vector_count":72,"ai_core_frequency_MHZ":[100,100]}
{"category":"Memory Information","hbm_total_MB":131072,"hbm_used_MB":5190.55,"hbm_frequency_MHZ":3200}
```


11.2.5. **Roofline 瓶颈分析**

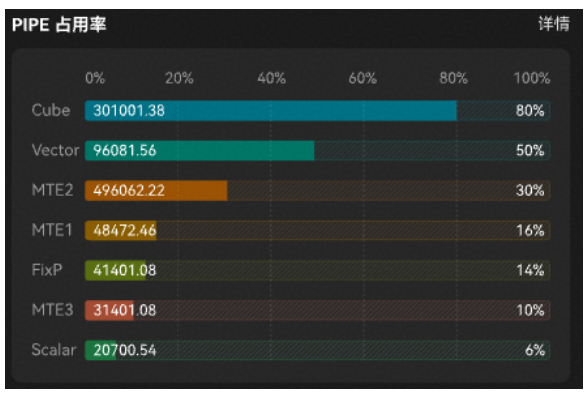


--section roofline

summary {"roofilne": 理论值, "值"}

11.2.6. **计算负载分析**

🎯 需要针对OpType 为MIX 算子， 需要分Cube / Vector，icache rate



- Cube 占用率

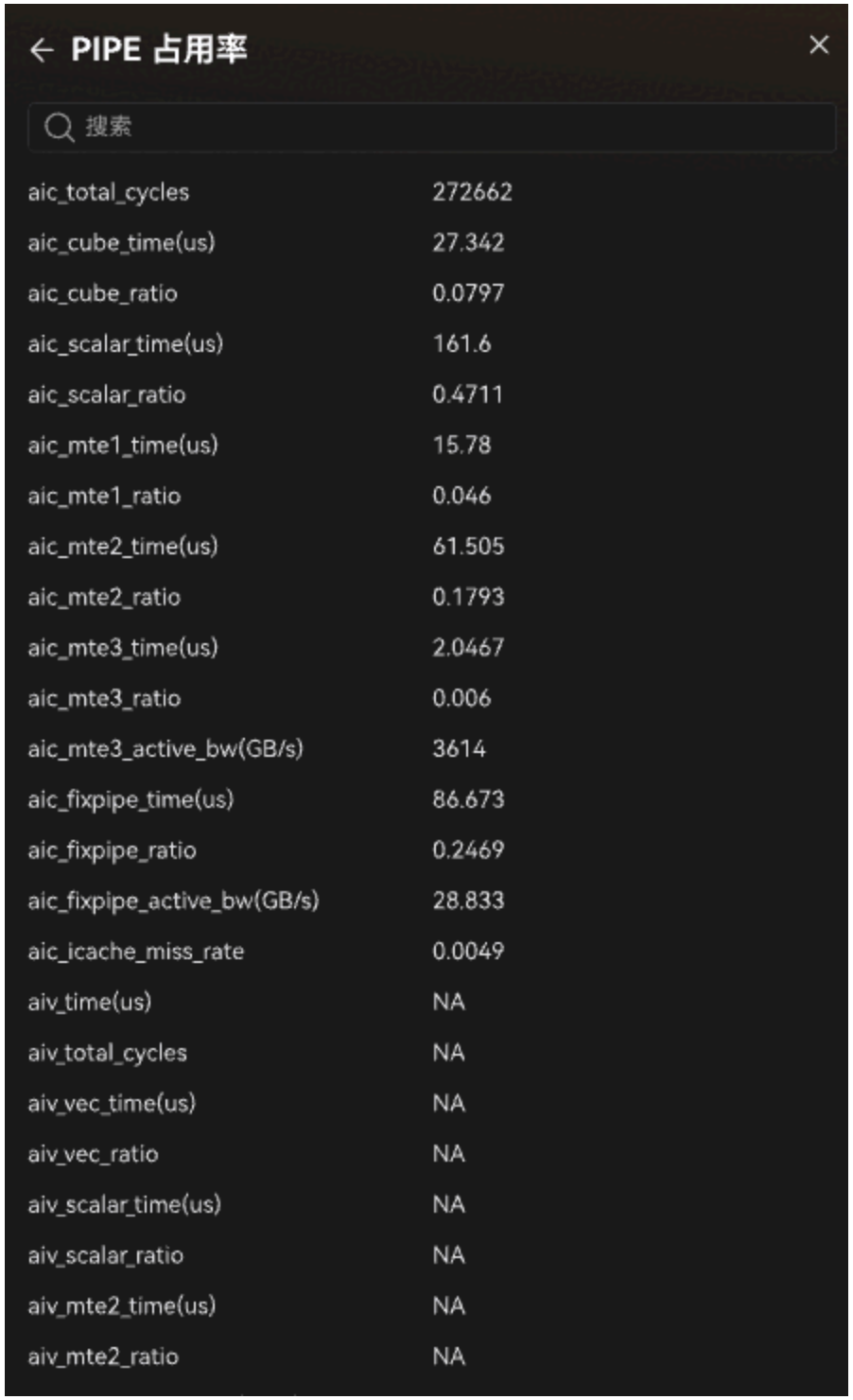
序号	显示	字段	数据来源

1	Cube	aic_cube_ratio	PipeUtilization.csv
2	MTE2	aic_mte2_ratio	PipeUtilization.csv
3	MTE1	aic_mte1_ratio	PipeUtilization.csv
4	FIXP	aic_fixpipe_ratio	PipeUtilization.csv
5	Scalar	aic_scalar_ratio	PipeUtilization.csv
6	ICache Miss	aic_icache_miss_rate	PipeUtilization.csv

- Vector 占用率

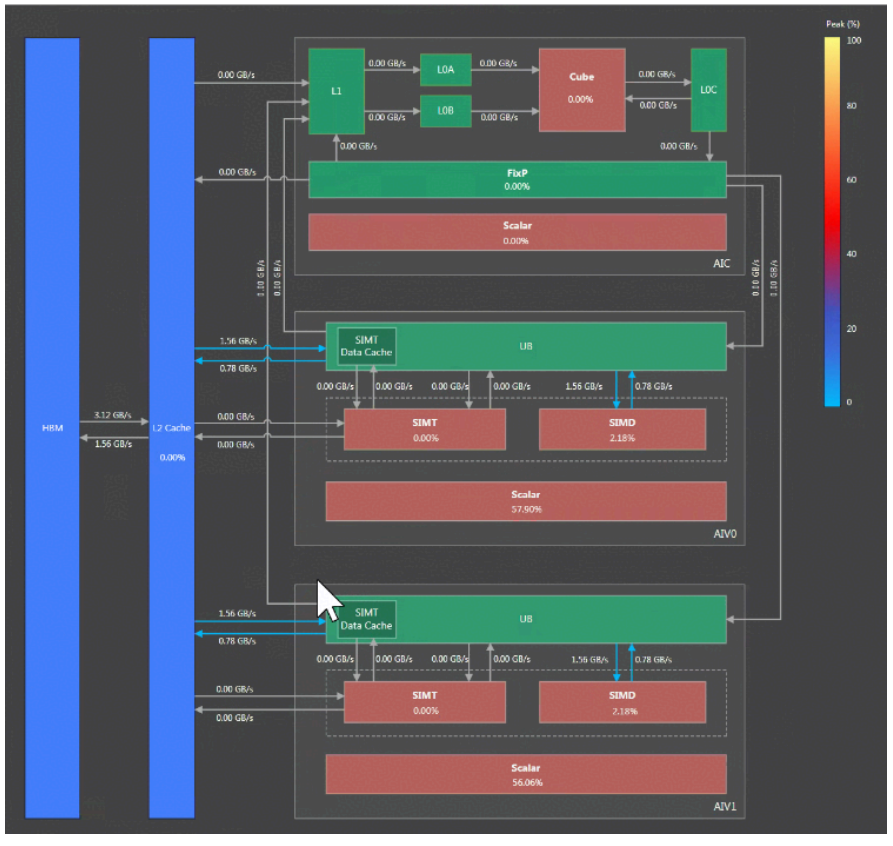
序号	显示	字段	数据来源
1	Vector	aiv_vec_ratio	PipeUtilization.csv
2	MTE2	aiv_mte2_ratio	PipeUtilization.csv
3	MTE3	aiv_mte3_ratio	PipeUtilization.csv
4	Scalar	aiv_scalar_ratio	PipeUtilization.csv
5	ICache Miss	aiv_icache_miss_rate	PipeUtilization.csv

11.2.6.1. **PipeUtilization详细信息 （制作成页签） （缺少：ArithmeticUtilization, ResourceConflictRatio）**



11.2.7. **内存负载分析**

🎯 需要针对双Die 场景，存在Remote内存获取，是否有右击的详情



1：NA的参数值不做显示, 0值照常显示。

2：有值的参数需要高亮显示，存在色阶对应百分比问题，需要确定理论值的100%

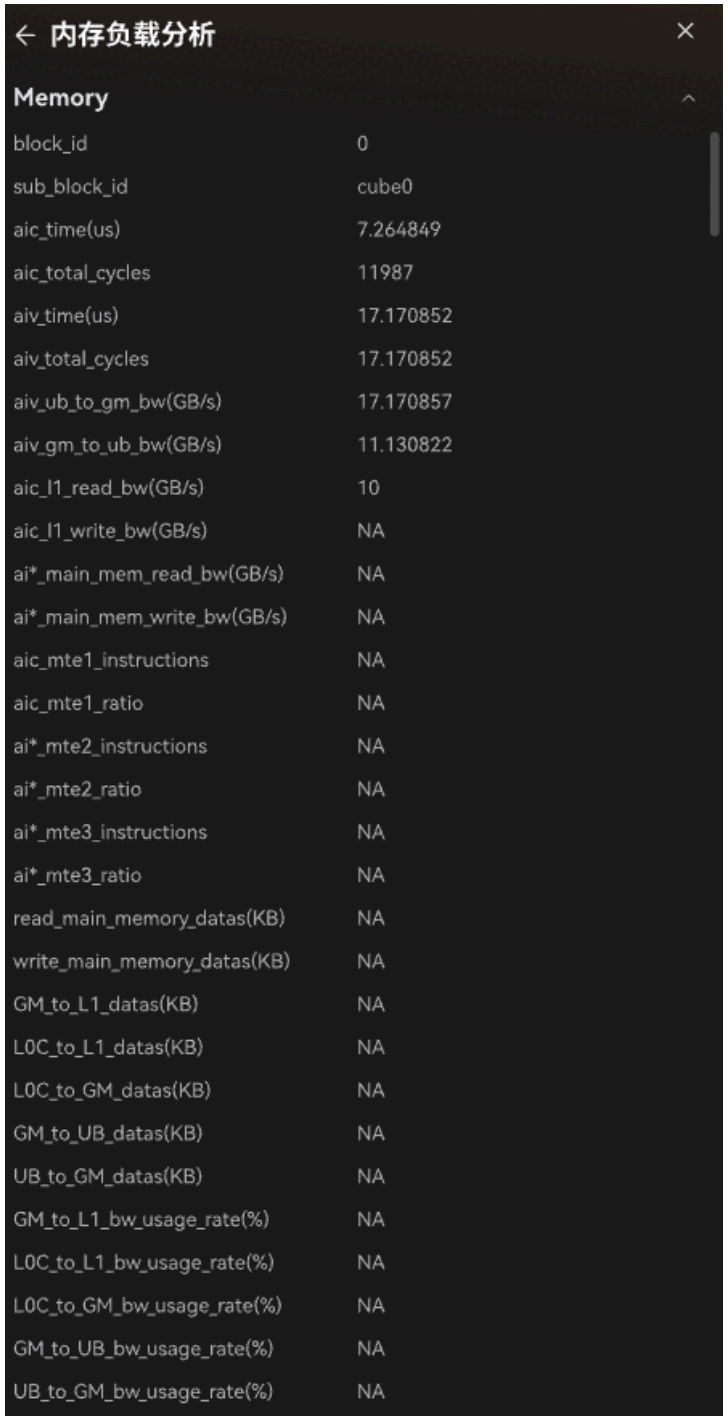
3：L2Cache存在远端访问和近端访问数据，close， far

序号	显示	字段	理论值	数据来源
	GM <-L2	ai*_main_mem_read_bw		Memory.csv
	GM -> L2	ai*_main_mem_write_bw		Memory.csv
	L2->L1	aic_l1_read_bw(GB/s)		Memory.csv
	L2<-L1	aic_l1_write_bw(GB/s)		Memory.csv
	L1 -> L0A	aic_l0a_read_bw(GB/s)		MemoryL0.csv
	L1 -> L0B	aic_l0b_read_bw(GB/s)		MemoryL0.csv
	L0A -> Cube	aic_l0a_write_bw(GB/s)		MemoryL0.csv
	L0B -> Cube	aic_l0b_write_bw(GB/s)		MemoryL0.csv
	L0C -> Cube	aic_l0c_read_bw_cube(GB/s)		MemoryL0.csv (待确定)

	Cube -> L0C	aic_l0c_write_bw_cube(GB/s)		MemoryL0.csv
	L0C -> L1	L0C_to_L1_datas		待确定
	L0C -> L2	L0C_to_GM_datas		待确定
	L0C -> UB			待确定
	UB -> L2	aiv_ub_read_bw_gm(GB/s)		MemoryUB.csv
	L2 -> UB	aiv_ub_write_bw_gm(GB/s)		MemoryUB.csv
	Vec -> UB	aiv_ub_read_bw_vector(GB/s)		MemoryUB.csv
	UB -> Vec	aiv_ub_write_bw_vector(GB/s)		MemoryUB.csv
	L2Cache Hit Rate			L2Cache.csv

11.2.7.1. **内存负载详细信息（制作成页签）**

- Memory： Memory.csv->L2Cache->Memory L0->Memory UB


内存负载分析
Memory
block_id 0
sub_block_id cube0
aic_time(us) 7.264849
aic_total_cycles 11987
aiv_time(us) 17.170852
aiv_total_cycles 17.170852
aiv_ub_to_gm_bw(GB/s) 17.170857
aiv_gm_to_ub_bw(GB/s) 11.130822
aic_l1_read_bw(GB/s) 10
aic_l1_write_bw(GB/s) NA
ai*_main_mem_read_bw(GB/s) NA
ai*_main_mem_write_bw(GB/s) NA
aic_mte1_instructions NA
aic_mte1_ratio NA
ai*_mte2_instructions NA
ai*_mte2_ratio NA
ai*_mte3_instructions NA
ai*_mte3_ratio NA
read_main_memory_datas(KB) NA
write_main_memory_datas(KB) NA
GM_to_L1_datas(KB) NA
L0C_to_L1_datas(KB) NA
L0C_to_GM_datas(KB) NA
GM_to_UB_datas(KB) NA
UB_to_GM_datas(KB) NA
GM_to_L1_bw_usage_rate(%) NA
L0C_to_L1_bw_usage_rate(%) NA
L0C_to_GM_bw_usage_rate(%) NA
GM_to_UB_bw_usage_rate(%) NA
UB_to_GM_bw_usage_rate(%) NA

序号	显示	字段	数据来源

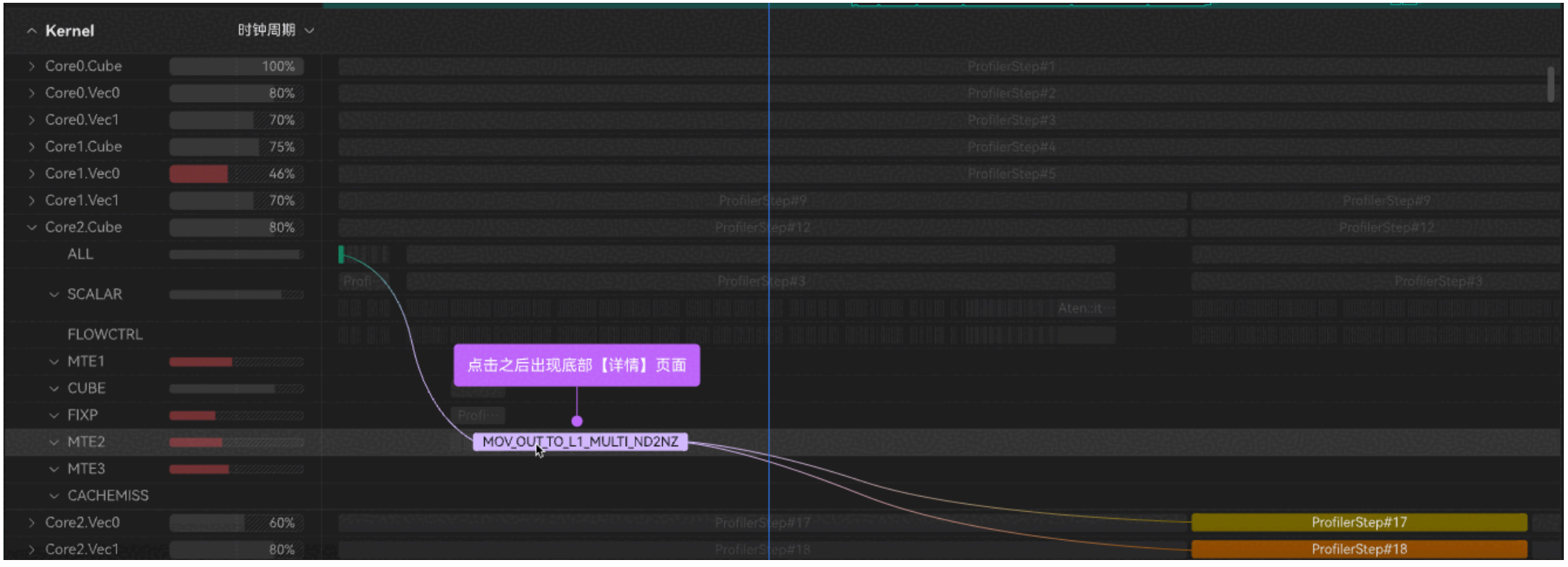
- L2Cache：L2Cache.csv

序号	显示	字段	数据来源

11.2.8. **统计分析**



11.2.9. **Kernel中Block级的Timeline**



序号	显示	字段	数据来源

11.2.9.1. **流水中详情信息**

左边start-> duration-> end
中间Code：None，930前不提供



序号	显示	字段	数据来源

11.3. 性能规格设计

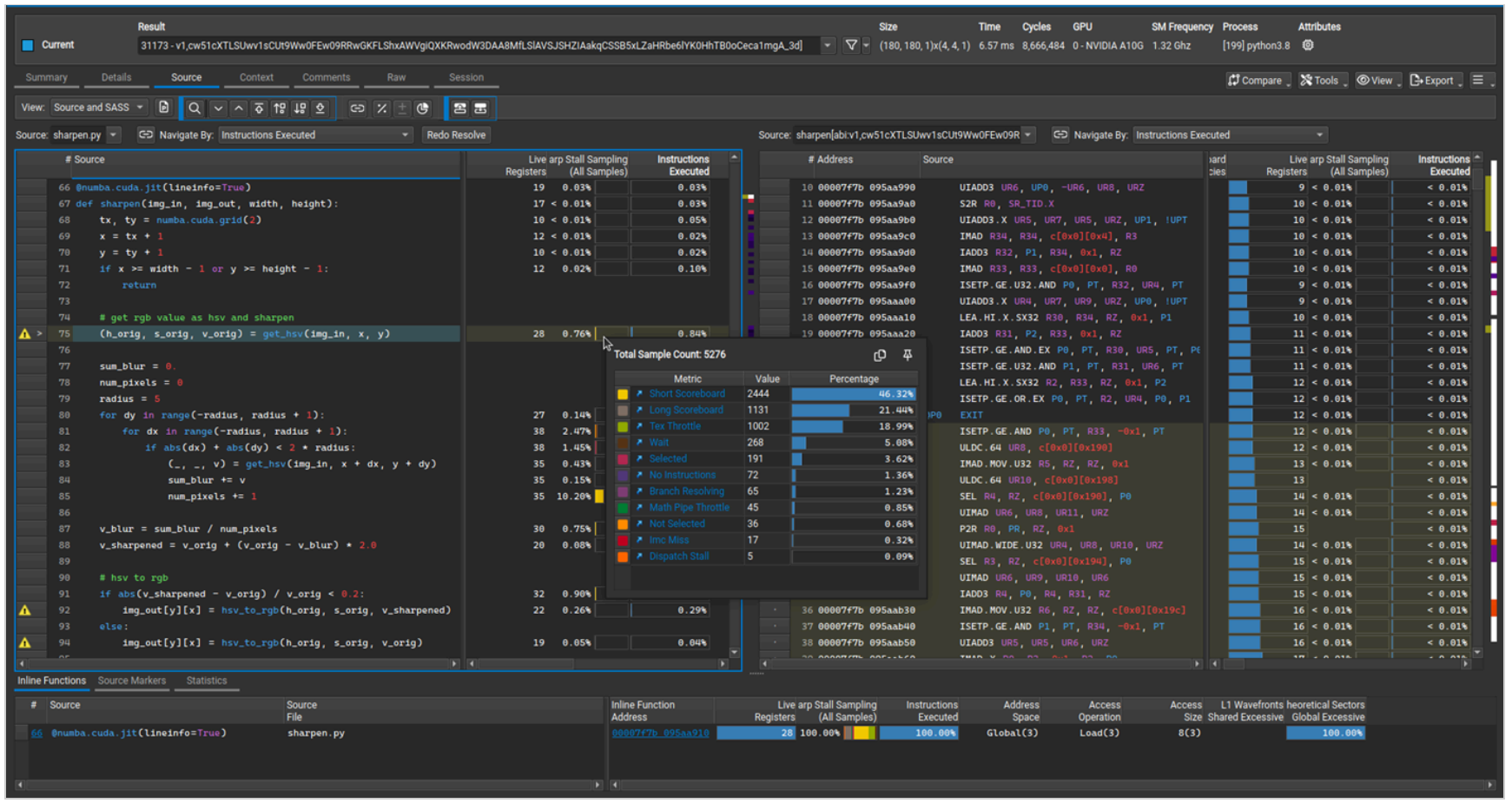
11.4. 代码关联及可视化方案

11.4.1. 功能概述

- 为了能够快速定位定界性能问题，需要支持分析每一行代码的状态
- 依据竞品能力支持代码行显示主要有

1)、source，激活的寄存器，sampling stall 次数， 执行执行次数

2)、汇编地址，汇编代码，激活的寄存器，sampling stall 次数， 执行执行次数



时间线 源码 详情 缓存

计算核： ALL 源码： [redacted]1_matmul_kernellaunch/MatmulInvocationNeo/build/auto_gen/ascendc_kernels_npu/auto_gen_m...

行数：，关联指令数：0 仅查看关联指令

#	源码	执行指令数	L2Cache命中率	处理数据量...
31	GM_ADDR workspace_param;			
32	GM_ADDR workspace_usr;			
33	#if defined(HAVE_TILING)			
34	workspace_param = workspace;			
35	#else			
36	workspace_param = tilingGm;			
37	#endif			
38	if (workspace_param == nullptr) {	6	NA	NA
39	return;			
40	}			
41	AscendC::SetSysWorkspaceForce(workspace_param);	6	NA	NA
42	workspace_usr = AscendC::GetUserWorkspace(workspace_param);			
43	#if defined(REGIST_MATMUL_OBJ) || defined(__MIX_CORE_MACRO__)			
44	if constexpr (g_coreType == AscendC::AIC) {			
45	matmul::clearWorkspace(workspace_param);	71	0%	32896
46	#ifdef ASCENDC_TIME_STAMP_ON			
47	AscendC::AscendCTimeStamp(static_cast<uint32_t>(AscendC::TimeStampId::TIME_STAMP_WRAP_			
48	#endif			
49	}			
50	#endif			
51	#if defined(HAVE_TILING)			
52	workspace = workspace_usr;			
53	#else			
54	tilingGm = workspace_usr;			
55	#endif			
56	#endif			
57	matmul_custom_origin(a, b, c, workspace, tilingGm);	56574	53.35%	6291968
58	#if defined(ASCENDC_DUMP) && defined(ASCENDC_DEBUG)			
59	AscendC::WriteBackOverflow(overflow_status);			
60	#endif			
61	}	51	NA	NA
62				
63	#endif			
64	#include "inner_interface/inner_kernel_operator_intf.h"			
65				

#	地址	指令队列	源码	执行指令数	L2Cache命中率	处理数据量(Bytes)
1	0x124000000000	SCALAR	MOV	1	NA	NA
2	0x124000000004	SCALAR	MOVK	1	NA	NA
3	0x124000000008	SCALAR	MOV	1	NA	NA
4	0x12400000000c	SCALAR	ADD.s64	1	NA	NA
5	0x124000000010	SCALAR	MOV	1	NA	NA
6	0x124000000014	SCALAR	MOV	1	NA	NA
7	0x124000000018	SCALAR	AND.b64	1	NA	NA
8	0x12400000001c	SCALAR	MOV	1	NA	NA
9	0x124000000020	SCALAR	MADD.s64	1	NA	NA
10	0x124000000024	SCALAR	SUB.s64	1	NA	NA
11	0x124000000028	SCALAR	MOV	1	NA	NA
12	0x12400000002c	SCALAR	ADD.s64	1	NA	NA
13	0x124000000030	SCALAR	LD.b64	1	NA	NA
14	0x124000000034	SCALAR	LDP.b64	1	NA	NA
15	0x124000000038	SCALAR	MOV	1	NA	NA
16	0x12400000003c	SCALAR	MOV	1	NA	NA
17	0x124000000040	SCALAR	CMP.EQ.s64	1	NA	NA
18	0x124000000044	FLOWCTRL	PRELOAD	1	NA	NA
19	0x124000000048	FLOWCTRL	JUMPC	1	NA	NA
20	0x12400000004c	SCALAR	MOV	1	NA	NA
21	0x124000000050	FLOWCTRL	JUMPCMP.EQ.s64	1	NA	NA
22	0x124000000054	FLOWCTRL	JUMP	1	NA	NA
23	0x124000000058	SCALAR	MOV	0	NA	NA
24	0x12400000005c	SCALAR	MOVK	0	NA	NA
25	0x124000000060	SCALAR	MOVK	0	NA	NA
26	0x124000000064	FLOWCTRL	JUMP	0	NA	NA
27	0x124000000068	SCALAR	SUB.s64	0	NA	NA
28	0x12400000006c	SCALAR	SHR.u64	0	NA	NA
29	0x124000000070	SCALAR	SHL.b64	0	NA	NA
30	0x124000000074	SCALAR	ADD.s64	1	NA	NA
31	0x124000000078	SCALAR	MOV	1	NA	NA
32	0x12400000007c	SCALAR	ST.b64	1	NA	NA
33	0x124000000080	SCALAR	MOV	1	NA	NA
34	0x124000000084	SCALAR	INSERT	1	NA	NA
35	0x124000000088	SCALAR	MOV	1	NA	NA

Timeline | Source | Details

Core : ALL

Source : /home/xxxx/xxx/smoke/c310_smoke-master/illegal_read_and_write/simt_gm_float/kernel.cpp

Line : , Related Instructions Count : 0

Only Related Instructions

#	Source	Instructions E...	GPR Count	Process Bytes
1	/*			
2	* Copyright (c) Huawei Technologies Co., Ltd. 2025-2025. All rights reserved.			
3	*/			
4				
5	#include "kernel_operator.h"			
6	using namespace AscendC;			
7				
8	template <typename T>			
9	__simt_vf__ __aicore__ LAUNCH_BOUND(2048) inline void SimtCompute(__gm__ T* dst, __gm__ T* src0, __gm_			
10	{	2	1	NA
11	int idx = AscendC::Simt::GetThreadIdx<0>();	4	2	NA
12	if (idx < 100) {	2	2	NA
13	dst[0] = src0[0] + src1[2];	8	4	NA
14	dst[3] = src0[3] + src1[8];	8	4	NA
15	dst[5] = src0[8] + src1[7];	8	5	NA
16	dst[1000] = src0[8 + idx] + src1[1023];	10	3	NA
17	}			
18	}	2	0	NA
19				
20	template <typename T>			
21	__aicore__ inline void SimtComputeShell(GM_ADDR x, GM_ADDR y, GM_ADDR z, const int size)			
22	{			
23	__gm__ T* dst = (__gm__ T*)x;			
24	__gm__ T* src0 = (__gm__ T*)y;			
25	__gm__ T* src1 = (__gm__ T*)z;			
26	AscendC::Simt::VF_CALL<SimtCompute<T>>(AscendC::Simt::Dim3{3, 6, 2}, dst, src0, src1, size);	60	0	NA
27	}			
28				
29	extern "C" __global__ __aicore__ void vec_add(GM_ADDR src0, GM_ADDR src1, GM_ADDR out, GM_ADDR gm_til			
30	{	40	0	NA
31	SimtComputeShell<float>(out, src0, src1, 32768);	60	0	NA
32	}	2	0	NA
33				
34	extern "C" void vec_add_do(uint32_t blockDim, void *l2ctrl, void *stream, uint8_t *src0, uint8_t *src1			
35	{			

#	Address	Pipe	Source	Instructions Exe...	GPR Count	GPR Status	Process Bytes
4	0x1000444000c4	SCALAR	MOVI	2	0		NA
5	0x1000444000c8	SCALAR	DCCI	2	0		NA
6	0x1000444000cc	SCALAR	END	2	0		NA
7	0x1000444000d0	VECTOR	S2R	2	1		NA
8	0x1000444000d8	VECTOR	S2R	2	2		NA
9	0x1000444000e0	VECTOR	SHFI	2	2		NA
10	0x1000444000e8	VECTOR	ISETP.i	2	2		NA
11	0x1000444000f8	VECTOR	IADD.i	0	3		NA
12	0x100044400108	VECTOR	FMUL.i	2	3		NA
13	0x100044400118	VECTOR	LDG	2	3		NA
14	0x100044400120	VECTOR	LDG	2	4		NA
15	0x100044400128	VECTOR	FADD	2	4		NA
16	0x100044400130	VECTOR	STG	2	3		NA
17	0x100044400138	VECTOR	LDG	2	3		NA
18	0x100044400140	VECTOR	LDG	2	4		NA
19	0x100044400148	VECTOR	FADD	2	4		NA
20	0x100044400150	VECTOR	STG	2	3		NA
21	0x100044400158	VECTOR	LDG	2	3		NA
22	0x100044400160	VECTOR	LDG	2	4		NA
23	0x100044400168	VECTOR	FADD	2	5		NA
24	0x100044400170	VECTOR	STG	2	3		NA
25	0x100044400178	VECTOR	LDG	2	3		NA
26	0x100044400180	VECTOR	LDG	2	2		NA
27	0x100044400188	VECTOR	FADD	2	2		NA
28	0x100044400190	VECTOR	STG	2	1		NA
29	0x100044400198	VECTOR	END	2	0		NA
30	0x1000444001a0	VECTOR	LDG	0	1		NA
31	0x100044400000	SCALAR	MOV_SPR2X	2	0		NA
32	0x100044400004	SCALAR	MOVK	2	0		NA
33	0x100044400008	SCALAR	DC_PRELOADI	2	0		NA
34	0x10044440000c	SCALAR	MOVI	2	0		NA
35	0x100044400010	SCALAR	MOVK	2	0		NA
36	0x100044400014	SCALAR	MOV_SPR2X	2	0		NA
37	0x100044400018	SCALAR	MOV_SPR2X	2	0		NA
38	0x10044440001c	SCALAR	ADD.s64	2	0		NA

Assembly Line	Execution Count ...	Stall Samples ...	Instruction Type
0x10c SET_FLAG_V.MTE3 #0	8		PUSHQ
0x110 WAIT_FLAGI.MTE3.Vector #0	8		MTE3
0x114 ADD.s64 X14 [WO], X11 [RO], X13 [RO]	8		SCALAR
0x118 MOV_UB_TO_OUT_ALIGN_V2 X14 [RO], X9 [RO], X2 [RO], X1 [RO]	8		MTE3
0x11c SET_FLAGI.MTE2.MTE3 #0	8		MTE3
0x120 WAIT_FLAGI.MTE2.MTE3 #0	8		MTE2
0x124 ADD.s64 X13 [WO], X13 [RO], X9 [RO]	8		SCALAR
0x128 CMP.s64 X13 [RO], X10 [RO]	8		SCALAR
0x12c JUMPCI #-32	8		FC
0x130 **Basic Block 2:**pred: [], succ: []			
0x130 BAR.All	1		FC
0x134 MOVI X0 [WO], #0	1		
0x138 DCCI X0 [RO]	1		
0x13c MOVI X15 [WO], #0	1		
0x140 DCCI X15 [RO]	1		
0x144 END	1		
0x200 **Function gelu_kernel.vector.thread:**			
0x200 **Basic Block 0:**pred: [], succ: []			

- Scalar Active (0)
- Pipe Active (0)
- Memory Stall (0)
- Execution Stall (0)
- Synchronization Stall (990)
- Queue Full Stall (0)
- Instruction Fetch Stall (0)
- Other Stall (0)

Source Line	Bank Conflicts
25	
26 `tmp(tid*factor) = 0;`	3
27	
28 `__syncthreads();`	
29 `for (uint32_t i = 0; i < ntime; i++) {`	
H 30 `tmp(tid*factor) += tid * factor;`	60
31 `__syncthreads();`	
32 `}`	
33	
34 `global_tensor_a(gid) = tmp(tid*factor);`	3
35 `});`	
	60

Assembly Line	Bank Conflicts	Instruction Type
0x240 STS R3, R1, P7	3	RVECST
0x248 BAR_THREAD_BLOCK		RVECSU
0x250 LDS R3, R1, P7	3	RVECLD
0x258 LEA R3, R0, R3, #3, P7, P7		RVECEX
0x260 STS R3, R1, P7	3	RVECST
0x268 BAR_THREAD_BLOCK		RVECSU
0x270 LDS R3, R1, P7	3	RVECLD
0x278 LEA R3, R0, R3, #3, P7, P7		RVECEX
0x280 STS R3, R1, P7	3	RVECST
0x288 BAR_THREAD_BLOCK		RVECSU
0x290 LDS R3, R1, P7	3	RVECLD
	60	

Performance hints

Source Line	Program counter	Hint text
H 35	0x1dc	*Low warp occupancy detected (3.12%). Consider using more threads to achieve better performance.*
H 30	Not specified	*UB bank conflicts detected. Try to optimize local memory accesses, prefer sequential patterns instead of strided.*

Source Line	Cache Line ...	Cache Line Utilization ...
22		
23 `Simt::ParallelEXE(num_threads, [=]() mutable {`		
24 `int32_t tid = static_cast<int32_t>(Simt::GetTh`		
25 `int32_t gid = static_cast<int32_t>(block_idx *`		
26		
27 `for (uint32_t i = gid; i < num_threads; i += n`		
28 `int32_t idx = global_tensor_index.GetValue(i`		
29 `global_tensor_b(i) = global_tensor_a(idx);`	4.5	56.25
30 `}`		
31 `});`		
32 `return;`		
	5	56

Assembly Line	Cache Line Count	Cache Line Utilization
0x300 LOP3 #234, R12, R10, R4, R127, P7		
0x310 LDG R10, R12, P7	1	100
0x318 LEA R12, R10, R6, #2, P0, P7		
0x320 LEA_HI_X R13, R10, R7, #2, P0, P7		
H 0x328 LDG R10, R12, P7	8	12.5
0x330 LEA R12, R0, R8, #2, P0, P7		
0x338 LEA_HI_X R13, R0, R9, #2, P0, P7		
0x340 IADD R0, R0, R1, P7		
0x348 ISETP P7, P0, P0, P3, P7, P7		
0x350 STG R10, R12, P7	1	100
0x358 BRANCH #-136 , P0		
	5	56

Performance Hints

Source Line	Instruction Address	Hint Message
H 29	0x328	*Inefficient global memory operation detected. Consider to use sequential memory accesses to achieve better performance.*

Source Line	Execution Time...	Warp Occupancy...
1125 `#ifdef SPLIT_CORE_CUBE`		
1126 `g_cubeTPipePtr = this;`		
1127 `#elif defined(SPLIT_CORE_VEC)`		
1128 `g_vecTPipePtr = this;`	2866	47
1129 `#else`		
1130 `g_tPipePtr = this;`		
1131 `#endif`		
1132 `g_tpipeImpl.isDestroy = false;`		
1133 `}`		
1134		
1135 `template <class T> __aicore__ inline bool TPipe:`		
1136 `{`		
	2866	47

Assembly Line	Execution Time ...	Warp Occupancy (%)
0x118 MOV_SPR2X X2, PC	4	
0x11c MOVI X3, #28	4	
0x120 NOP	0	
0x124 NOP	0	
0x128 SHLI.b64 X3, #2	4	
0x12c ADD.s64 X3, X3, X2	4	
H 0x130 VF_SIMT #114, X3 , X1 : <<<512, 1, 1>>>,	2217	47
0x138 BAR.All	0	
0x13c BAR.All	0	
0x140 MOVI X2, #1	4	
0x144 MOVI X1, #0	4	
0x148 MOVK #3, X2, #61440	4	
	2866	47

Performance Hints

Source Line	Instruction Address	Hint Message
H 1128	0x130	*Low warp occupancy detected (47.33%). Consider to use more warps and eliminate job imbalance for better performance.*

Source Line		Execution Time ...	Branch Efficiency (%)
26	float f = 6.0f;		
27	float x = float(gid) / 1024.0f;		
28	float x2 = x * x;		
H 29	float xn = x * x2;	96	50
30	int fact = 4.0;		
31			
32	for (uint32_t i = 1; i < 9; i++) {		
H 33	x = x - xn / f;	992	50
H 34	xn = - (xn * x2);	224	50
35	f = f * (fact) * (fact + 1.0f);		
36	fact = fact + 2.0f;		
37	}		
38	tmp(gid) = x;		
39	} else {		
40	float f = 2.0f;		
		992	**50**

Assembly Line	Execution Time ...	Branch Efficiency (%)
0x3b0 MOVI R9, #3485045862, P7	28	50
0x3b8 FMUL R6, R1, R4, P7	36	50
0x3c0 FDIV R4, R4, R5, P7	72	50
0x3c8 MOVI R5, #3270508544, P7	28	50
0x3d0 FMUL R7, R1, R6, P7	36	50
0x3d8 FDIV R5, R6, R5, P7	72	50
0x3e0 MOVI R6, #3315433472, P7	28	50
0x3e8 FMUL R8, R1, R7, P7	40	50
0x3f0 FDIV R6, R7, R6, P7	72	50
0x3f8 MOVI R7, #3367055360, P7	28	50
0x400 FADD R3, R3, R4, P7	32	50
0x408 FMUL R4, R1, R8, P7	36	50
0x410 FDIV R7, R8, R7, P7	72	50
0x418 MOVI R8, #3424142656, P7	28	50
0x420 FADD R3, R3, R5, P7	36	50
	992	**50**

Performance Hints

Source Line	Instruction Address	Hint Message
H 38	0x1dc	*Low warp occupancy detected (12.39%). Consider to use more warps and eliminate job imbalance for better performance.*
H 34	Not specified	*Inefficient usage of warps detected (some threads in a warp do nothing due to condition). Consider to change the condition to use all the threads inside a warp.*
H 33	Not specified	*Inefficient usage of warps detected (some threads in a warp do nothing due to condition). Consider to change the condition to use all the threads inside a warp.*
H 29	Not specified	*Inefficient usage of warps detected (some threads in a warp do nothing due to condition). Consider to change the condition to use all the threads inside a warp.*
H 413	Not specified	*Inefficient usage of warps detected (some threads in a warp do nothing due to condition). Consider to change the condition to use all the threads inside a warp.*

11.4.2. **实现设计**

11.4.2.1. **数据采集流程**

- Instruction excuted

通过bbcount的插桩，获取bbcount的下的pc和执行的次数

怎么调用接口

http://corc.rnd.huawei.com/docs/hibit/UserGuide/HibitAPI.html

- simt pc sampling stall
- live register
- register status

获取算子.o之后需要通过llvm-objdump -d -l -S aicore.o 直接获取反汇编文件，示例数据：

https://gitcode.com/wk0911/npu-tools/blob/main/npu-compute/obj/objwithsource.txt

采集的数据按照PC地址放置。pc调用栈信息，通过llvm-symbolizer -f -e aicore.o --inlining=true --output-style=JSON 0x00 0x01 0x450


```json
[
{"Address":"0x0","ModuleName":"op_585649.aicore.o",
    "Symbol":[{"Column":0,"Discriminator":0,"FileName":"/home/wk/w00968123/asc/op_585649/op_585649.asc","FunctionName":"Hello_World_5856
{"Address":"0x1","ModuleName":"op_585649.aicore.o",
    "Symbol":[{"Column":0,"Discriminator":0,"FileName":"/home/wk/w00968123/asc/op_585649/op_585649.asc","FunctionName":"Hello_World_5856
{"Address":"0x450","ModuleName":"op_585649.aicore.o",
    "Symbol":[
        {"Column":41,"Discriminator":0,"FileName":"/usr/local/Ascend/cann-9.3.0/asc/impl/utils/debug/npu_arch_3510/asc_aicore_printf_uti
        {"Column":5,"Discriminator":0,"FileName":"/usr/local/Ascend/cann-9.3.0/asc/include/basic_api/../../impl/basic_api/dav_3510/../..
        {"Column":5,"Discriminator":0,"FileName":"/usr/local/Ascend/cann-9.3.0/asc/include/basic_api/../../impl/basic_api/dav_3510/../..
        {"Column":5,"Discriminator":0,"FileName":"/usr/local/Ascend/cann-9.3.0/asc/include/basic_api/../../impl/basic_api/dav_3510/../..
        {"Column":5,"Discriminator":0,"FileName":"/usr/local/Ascend/cann-9.3.0/asc/include/basic_api/../../impl/basic_api/dav_3510/../..
        {"Column":5,"Discriminator":0,"FileName":"/usr/local/Ascend/cann-9.3.0/asc/include/basic_api/../../impl/basic_api/dav_3510/../..
        {"Column":5,"Discriminator":0,"FileName":"/usr/local/Ascend/cann-9.3.0/asc/include/basic_api/../../impl/basic_api/dav_3510/../..
        {"Column":5,"Discriminator":0,"FileName":"/usr/local/Ascend/cann-9.3.0/asc/include/basic_api/../../impl/basic_api/dav_3510/../..
        {"Column":5,"Discriminator":0,"FileName":"/usr/local/Ascend/cann-9.3.0/asc/include/basic_api/../../impl/basic_api/dav_3510/../..
    ]
}
]
```


11.4.2.2. **数据保存格式**

按照pc 为key，保存其他的参数信息

11.4.2.3. **界面呈现**

- C/C++ 界面

类别	说明	备注
Source	C/C++代码	
Instruction excuted	指令执行次数	
Instruction time	指令执行耗时	
sampling stall	stall的信息	stall reason 分类统计
warp occupancy	warp占用率	
Cache line count	cache line次数	

Cache line Utilization	cache line利用率	
live register	激活的寄存器	
branch efficiency	分支效率	

- 汇编界面

类别	说明	备注
Address	指令地址	
Assembly	汇编代码	
Instruction excuted	指令执行次数	
Instruction time	指令执行耗时	
Instruction type	指令类型	
sampling stall	stall的信息	stall reason 分类统计
warp occupancy	warp占用率	
Cache line count	cache line次数	
Cache line Utilization	cache line利用率	
live register	激活的寄存器	
register status	寄存器状态	
branch efficiency	分支效率	

- npu-rep的文件中：objdump.txt

shell Full Screen

```
0000000000000000 <_Z16gather_2d_customIfjEvPT_PT0_S1_jj$local>:
_Z16gather_2d_customIfjEvPT_PT0_S1_jj():
       0:                           02004880      MOV           X0, PARA_BASE
       4:                           07c10000      MOVK          X0, #0, #3
.ParamSummary__Z16gather_2d_customIfjEvPT_PT0_S1_jj():
       8:                           08c00000      DC_PRELOADI   [X0], #0
```

```
0000000000000000 <_Z16gather_2d_customIfjEvPT_PT0_S1_jj$local>:
_Z16gather_2d_customIfjEvPT_PT0_S1_jj():
       0:  02004880    MOV         X0, PARA_BASE
       4:  07c10000    MOVK        X0, #0, #3
.ParamSummary__Z16gather_2d_customIfjEvPT_PT0_S1_jj():
       8:  08c00000    DC_PRELOADI [X0], #0
       c:  073afea0    MOVI        X29, #65184
      10:  077b000f    MOVK        X29, #15, #1
      14:  029e3880    MOV         X15, SYS_VA_BASE
.ParamInfo__Z16gather_2d_customIfjEvPT_PT0_S1_jj_0():
      18:  02224880    MOV         X17, PARA_BASE
      1c:  003bd781    ADD.s64     X29, X29, X15
      20:  021f0880    MOV         X15, COREID
      24:  07207fff    MOVI        X16, #32767
      28:  026202b4    SHRI.u64    X17, #52
      2c:  00def80a    AND.b64     X15, X15, X16
      30:  08231002    ADDI        X17, X17, #2
```

Address

Assembly

11.5. **Mertrics采集**

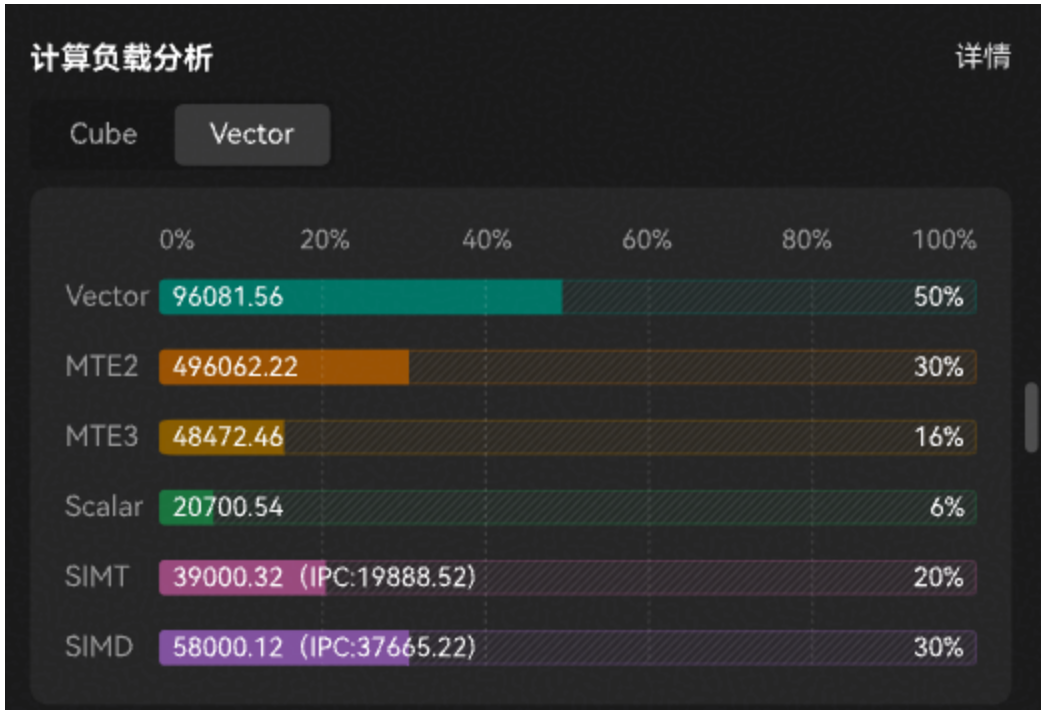
11.5.1. **aicore 采集**

https://clouddrive.huawei.com/p/296b28f1f462c8a616e5d83d4d6b3e2e

指标	含义	PMU值	PMU名称	计算公式	备注

aiv_vec_ratio		1280 + 1	pmu_idc_aiv_vec_busy_o	pmu_idc_aiv_vec_busy_o / aiv_total_cycle	
aiv_simd vfratio		1280 + 2	pmu_idc_aiv_vec_instr_simd_vf_busy_o	pmu_idc_aiv_vec_instr_simd_vf_busy_o / aiv_total_cycle	
simd vf ipc		1280 + 43	idu_pmu_ins_iss_cnt	idu_pmu_ins_iss_cnt/pmu_idc_aiv_vec_instr_simd_vf_busy_o	
simt vf ratio		1280 + 4	pmu_idc_aiv_vec_instr_simt_vf_busy_o	pmu_idc_aiv_vec_instr_simt_vf_busy_o / aiv_total_cycle	
aiv_simt_vf_ipc		1280 + 201	wsu_pmu_exu_ins_issue	(wsu_pmu_exu_ins_issue + wsu_pmu_dvg_ins_issue + wsu_pmu_dcu_ins_issue) / pmu_idc_aiv_vec_instr_simt_vf_busy_o / 4	
		1280 + 202	wsu_pmu_dvg_ins_issue		
		1280 + 203	wsu_pmu_dcu_ins_issue		

11.5.2. **aicore参数显示**



11.6. **通算融合算子**

11.6.1. **采集方式**

--section communication
由于通算融合存在多个硬件kernel的下发，单纯的执行aicore kernel时存在问题，需要支持application replay能力。

- application replay 主要是将程序执行多次，每次采集不同metrics的数据。
- 配置profiling 采集程序程序

针对aicpu数据需要开启：aicpu=on
针对ccu需要开启：使用task_time=l1 (level1)

- 需要采集msprof的数据

采集aicpu数据：

#	字段	含义
1	waitNotifyStartTime	开始等待通信参数
2	kfcAlgExeStartTime	开始通信算法执行
3	sendTaskStartTime	开始下发任务
4	waitActiveStartTime	开始等待激活
5	activeStartTime	开始激活处理
6	waitExeEndStartTime	开始等待任务执行结束
7	rtsqExeEndTime	任务执行结束

采集CCU数据

CCU Mission任务的

bit / no	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	taskID																streamId															
1	Start Loopgroup syscnt[15:0]																LoopGroup InstId															
2	Start Loopgroup syscnt[47:16]																															
3	End Loopgroup syscnt[15:0]																Start Loopgroup syscnt[63:48]															
4	End Loopgroup syscnt[47:16]																															
5	SetCKEBit InstId																End Loopgroup syscnt[63:48]															
6	SetCKEBit Start Syscnt[31:0]																															
7	SetCKEBit Start Syscnt[63:31]																															
8	rel_syscnt_1																rel_syscnt_0															
9	rel_syscnt_3																rel_syscnt_2															
10	rel_syscnt_5																rel_syscnt_4															
11	rel_syscnt_7																rel_syscnt_6															
12	rel_syscnt_9																rel_syscnt_8															
13	rel_syscnt_11																rel_syscnt_10															
14	rel_syscnt_13																rel_syscnt_12															
15	rel_syscnt_15																rel_syscnt_14															

◆ Rt_streamID：该域段表示RunTime的streamID，STARS带过来的信息，只在profiling记录时使用

◆ TaskID：表示SQE唯一性的标识符，STARS带过来的信息，只在profiling记录时使用

◆ SetCKEBit InstID：SetCKEBit的Instruction ID

◆ SetCKEBit Startsyscnt：SetCKEBit执行的起始时间

◆ rel_syscnt_0：SetCKEBit中Maskn bit0置1相对SetCKEBit Startsyscnt的差值

◆ rel_syscnt_1~15：SetCKEBit中Maskn bit1~bit15置1相对SetCKEBit Startsyscnt的差值

◆ Loopgroup InstID：Loopgroup的Instruction ID

channel记录

通道平均/最小/最大传输延迟，可换算成成带宽

CCU 带宽任务的Profile

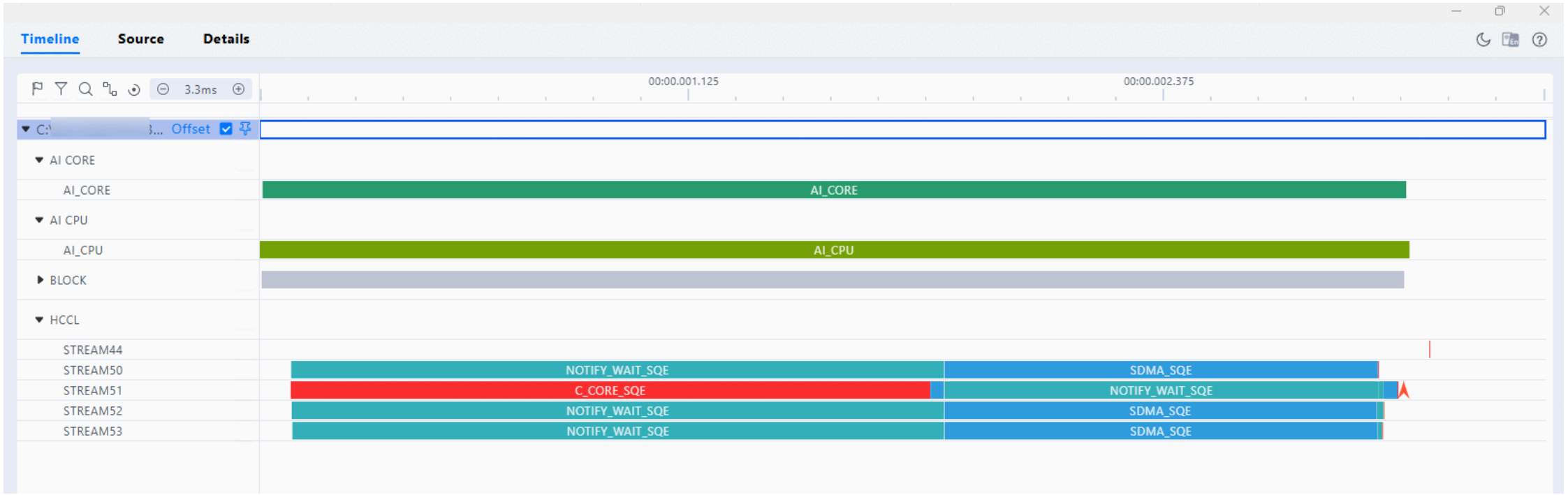
bit / no	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	channel_0_syscnt[31:0]																															
1	channel_0_syscnt[63:31]																															
2	channel_0_max																															
3	channel_0_min																															
4	channel_0_avg																															
5~634	channel_1~126_syscnt1[31:0]																															
5~634	channel_1~126_syscnt1[63:31]																															
5~634	channel_1~126_max																															
5~634	channel_1~126_min																															
5~634	channel_1~126_avg																															
635	channel_127_syscnt1[31:0]																															
636	channel_127_syscnt1[63:31]																															
637	channel_127_max																															
638	channel_127_min																															
639	channel_127_avg																															

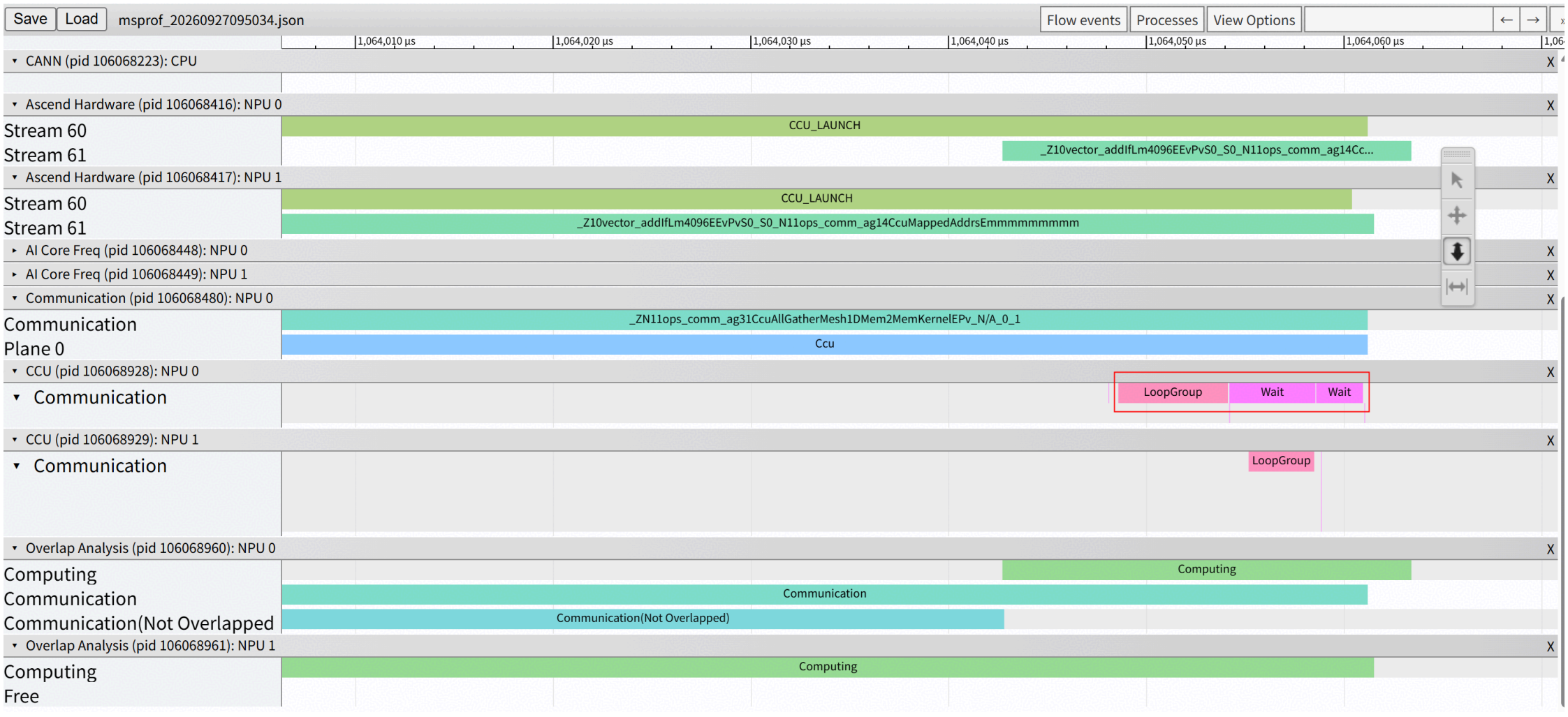
msprof 上报的数据：

AICPU: MsprofAicpuHcclTaskInfo

CCU：CCUTaskInfo、CCUWaitSignalInfo，CCUGroupInfo

11.6.2. **数据呈现**

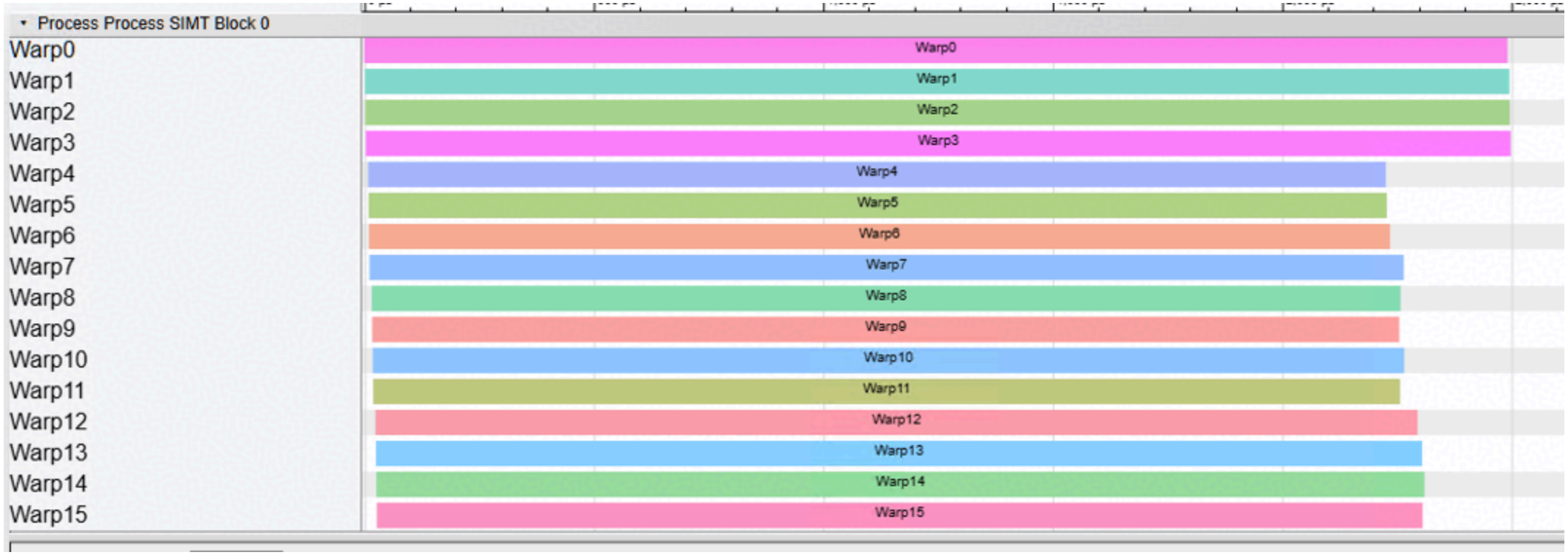

Timeline
Source
Details
3.3ms
00:00.001.125
00:00.002.375
Offset
AI CORE
AI_CORE
AI_CORE
AI CPU
AI_CPU
AI_CPU
BLOCK
HCCL
STREAM44
STREAM50
NOTIFY_WAIT_SQE
SDMA_SQE
STREAM51
C_CORE_SQE
NOTIFY_WAIT_SQE
STREAM52
NOTIFY_WAIT_SQE
SDMA_SQE
STREAM53
NOTIFY_WAIT_SQE
SDMA_SQE


Save
Load
msprof_20260927095034.json
Flow events
Processes
View Options
1,064,010 µs
1,064,020 µs
1,064,030 µs
1,064,040 µs
1,064,050 µs
1,064,060 µs
CANN (pid 106068223): CPU
Ascend Hardware (pid 106068416): NPU 0
Stream 60
CCU_LAUNCH
Stream 61
_Z10vector_addIfLm4096EEvPvS0_S0_N11ops_comm_ag14Cc...
Ascend Hardware (pid 106068417): NPU 1
Stream 60
CCU_LAUNCH
Stream 61
_Z10vector_addIfLm4096EEvPvS0_S0_N11ops_comm_ag14CcuMappedAddrsEmmmmmmmmm
AI Core Freq (pid 106068448): NPU 0
AI Core Freq (pid 106068449): NPU 1
Communication (pid 106068480): NPU 0
Communication
_ZN11ops_comm_ag31CcuAllGatherMesh1DMem2MemKernelEPv_N/A_0_1
Plane 0
Ccu
CCU (pid 106068928): NPU 0
Communication
LoopGroup
Wait
Wait
CCU (pid 106068929): NPU 1
Communication
LoopGroup
Overlap Analysis (pid 106068960): NPU 0
Computing
Computing
Communication
Communication
Communication(Not Overlapped)
Communication(Not Overlapped)
Overlap Analysis (pid 106068961): NPU 1
Computing
Computing
Free

11.7. **SIMT调优**

11.7.1. **SIMT Wrap occupancy**

通过 hibit 插桩：http://corc.rnd.huawei.com/docs/hibit/UserGuide/HibitAPI.html#simt-warp-occupancy



11.8. **Roofline**

11.8.1. **理论性能计算**

- p_peak理论算力计算口径（frequency 单位 MHz，结果 TFLOPS）：

执行器	**数据类型**	**公式**
cube	fp16/bf16	cube_num × 4096 × 2 × frequency / 1000
cube	tf32/hf32	cube_num × 2048 × 2 × frequency / 1000
cube	int8/fp8	cube_num × 8192 × 2 × frequency / 1000
vector	fp16/bf16	vector_num × 128 × 2 × frequency / 1000
vector	fp32	vector_num × 64 × 2 × frequency / 1000
vector	int32	vector_num × 64 × 2 × frequency / 1000

🎯 说明：cube 一拍完成 16×16×16 个乘法操作（2 表示乘加）；vector 一拍 128（fp16）/64（fp32、int32）次加法（2 表示双发）。

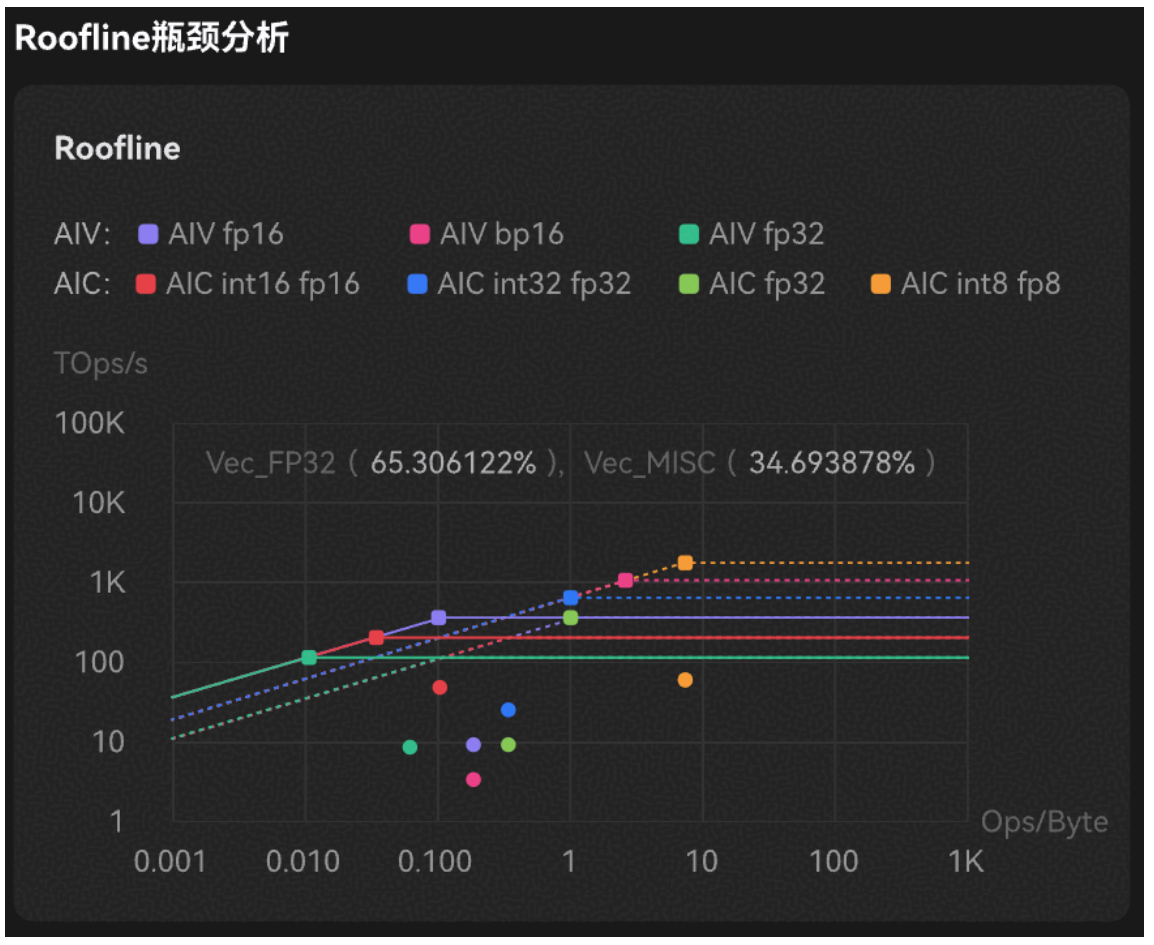
- b_peak规格定义最大带宽：1.6T/s

11.8.2. **数据呈现**

yaml

Full Screen

```yaml
{
  "category": "Roofline",
  "aiv_simd_b8": {"p_peak": 27033600000000,"b_peak": 1600,"w": 16777216,"t": 0.00005,"q": 50331648},
  "aiv_simt_b8": {"p_peak": 6758400000000, "b_peak": 800, "w": 16777216, "t": 0.0001,"q": 50331648 },
  "aiv_simd_b16": {"p_peak": 27033600000000,"b_peak": 1600,"w": 16777216,"t": 0.0001,"q": 100663296 },
  "aiv_simt_b16": {"p_peak": 13516800000000,"b_peak": 800,"w": 16777216,"t": 0.0002,"q": 100663296},
  "aiv_simd_b32": {"p_peak": 13516800000000,"b_peak": 1600,"w": 16777216,"t": 0.0002,"q": 201326592},
  "aiv_simt_b32": {"p_peak": 13516800000000,"b_peak": 800,"w": 16777216,"t": 0.0004,"q": 201326592},
  "aic_b8": {"p_peak": 865075200000000,"b_peak": 1600,"w": 17179869184,"t": 0.00004,"q": 12582912},
  "aic_b16": {"p_peak": 432537600000000,"b_peak": 800,"w": 17179869184,"t": 0.00008,"q": 25165824},
  "aic_b32": {"p_peak": 27033600000000,"b_peak": 1600,"w": 2147483648,"t": 0.00012,"q": 12582912}
}
```



绘图时，横坐标计算强度 I=w/q，纵坐标实际性能 P=w/t，带宽上限需先转换为 B=b_peak×10⁹ bytes/s，理论 Roofline 为 min(p_peak, b_peak×I)。以上各点均满足 P≤p_peak 且 q/t≤B；Vector 示例偏访存受限，Cube 示例偏计算受限

11.9. acl_pti接口设计

11.9.1. 接口直接引用Hook流程

cpp

Full Screen

```cpp
// acl_pti中调用acl_tool_injection中的接口，acl_tool_injection接口调用MsprofSetInjectionFunc
// (runtime接口存在未初始化等问题需要先注册接口等runtime调用) void aclhookInitialize(void)
__attribute__((constructor(101))) static void aclptiRegisterHookFunc() {
    acltoolInjectionInitialize() {
        MsprofSetInjectionFunc(PROF_HOOK_INIT, aclhookInitialize);
    }
}

int32_t MsprofSetInjectionFunc(type func) {
    // 按照type: PROF_HOOK_INIT 保存func
}

void MsprofInjectionInitialize(void)
{
    // 优先判断ACL_API_INJECTION环境变量，如果有调用acltoolInitialize
    // 如果没有，检查有没有PROF_HOOK_INIT函数注册，如果有调用函数初始化
    // 如果都没有，直接返回
}

// runtime 接口存在二进制先注册，然后再初始化问题，这个需要考虑注册接口的劫持时机。
```

12. 参考文档

http://corc.rnd.huawei.com/docs/hibit/UserGuide/Hiprof.html